

Best Practices for Using Claude

From Personal Mastery to Enterprise Scale

*What Anthropic Recommends, What Practitioners Discover,
and Where They Converge*

Brandon Behring

Version 2.8
March 2026

Content verified against Anthropic documentation as of March 2026.
Licensed under CC BY 4.0

Contents

Preamble	1
I. Foundations	3
1. Principles and Mental Model	5
1.1. What Claude Code Actually Is	5
1.2. Four Engineering Principles	5
1.2.1. Never Fail Silently	5
1.2.2. Simplicity Over Complexity	6
1.2.3. Immutability by Default	6
1.2.4. Fail Fast with Diagnostics	6
1.3. The Codebase Is the Curriculum	7
1.4. Quick Reference	7
2. Your First CLAUDE.md	9
2.1. What CLAUDE.md Does	9
2.1.1. Hierarchical Loading	9
2.2. Generate It: <code>/init</code>	9
2.3. What to Include	9
2.4. What to Exclude	10
2.5. Your First Deny Rules	11
2.6. Quick Reference	11
3. Your First Working Session	12
3.1. The Session Loop	12
3.2. Three Prompting Basics	12
3.2.1. Be Specific About What, Not How	12
3.2.2. Tell Claude to Think Step by Step	13
3.2.3. Include Verification Criteria	13
3.3. Context Hygiene: The Basics	13
3.3.1. Two Commands to Know Now	13
3.3.2. The Two-Failure Rule	14
3.4. Course-Correcting	14
3.5. Plan Mode: Explore Before You Build	14
3.6. A Complete Example	15
3.7. Quick Reference	15
II. Personal Practice	17
4. Prompting That Works	19
4.1. Building a Precision Vocabulary	19
4.2. Structuring Complex Prompts	19
4.2.1. XML Tags for Multi-Component Prompts	19

4.2.2. Rich Inputs	20
4.3. Controlling Scope	21
4.4. Extended Thinking	22
4.4.1. Effort Controls	22
4.4.2. When to Enable vs. Disable	22
4.5. Cost Optimization	22
4.5.1. Prompt Caching	22
4.5.2. Batch API	23
4.5.3. Model Selection	23
4.5.4. Fast Mode (Research Preview)	23
4.6. Visual: Cost Optimization Decision Tree	24
4.6.1. Model Capabilities Reference	24
5. Thinking Together	26
5.1. Hypothesis-Driven Debugging	26
5.2. Tests as Thinking Tools	27
5.2.1. Property-Based Tests as Assumption Detectors	28
5.3. Surfacing Hidden Assumptions	28
5.4. Getting Genuine Comparisons, Not Validation	29
5.5. Quick Wins That Compound	30
5.5.1. Docstrings as Triple-Duty Documentation	30
5.5.2. Type Hints as Contracts	31
5.5.3. Error Messages That Debug Themselves	31
5.5.4. Code Archaeology	31
5.5.5. README-Driven Development	31
5.6. Let Claude Interview You	31
5.7. Architecture Decision Records	32
5.8. Quick Reference	33
5.8.1. Building a Prompt Pattern Library	33
6. Context as Currency	34
6.1. Why Context Degrades	34
6.2. The Full Context Toolkit	34
6.3. The Status Line: Zero-Cost Monitoring	35
6.4. The Two-Failure Rule (Revisited)	36
6.4.1. Customizing Compaction	36
6.5. Strategy for 1M Context Windows	37
6.5.1. PreCompact Hook for State Preservation	37
6.5.2. Two-Phase Compaction	37
6.5.3. PostCompact Hook for State Restoration	38
6.5.4. Decision Framework for 1M	38
6.6. Context Curation for Large Codebases	39
6.7. The Compaction Protocol	40
6.8. Session Management	41
6.8.1. Session Navigation	41
6.8.2. Durable Artifacts: The Primary Persistence Strategy	42

6.9. Auto-Memory and Cross-Session Persistence	42
6.9.1. What to Store in Memory	43
6.9.2. What NOT to Store	43
6.10. Plan Documents for Large Tasks	44
6.11. Quick Reference	44
7. The Edit-Test-Commit Loop	46
7.1. Verification Is King	46
7.1.1. The 6-Layer Validation Architecture	46
7.1.2. The Missing Layer: Domain Correctness	47
7.2. Phase-Appropriate Standards	48
7.2.1. Making Phase Transitions Explicit	48
7.3. Test-First with Claude	49
7.4. Verification Criteria in CLAUDE.md	49
7.5. Quality Gates via Hooks	49
7.6. Quick Reference	50
8. Extending Claude: Commands, Skills, Hooks, and MCP	51
8.1. Choosing the Right Mechanism	51
8.2. Custom Slash Commands	52
8.3. Skills: Domain Knowledge and Workflows	53
8.3.1. File Structure and Precedence	53
8.3.2. Frontmatter Reference	53
8.3.3. Basic Skill Example	54
8.3.4. Dynamic Context	54
8.3.5. Forked Context: Skill as Subagent	55
8.3.6. Budget and Permissions	55
8.4. Hooks: Deterministic Automation	55
8.4.1. Available Lifecycle Events (25 Total)	55
8.4.2. Handler Types	58
8.4.3. Notification Hooks	58
8.4.4. Debugging Hooks, Skills, and MCP Servers	59
8.5. MCP: External Tool Integration	59
8.5.1. Practical MCP Walkthrough	60
8.6. Plugins: Community Extensions	61
8.7. IDE Integrations	62
8.8. Sandboxing: Safe Autonomy	63
8.9. CLI Tools: Context-Efficient Integration	63
8.10. Visual: The Extension Hierarchy	63
III. Advanced Craft	65
9. CLAUDE.md Architecture	67
9.1. The Hub-and-Spoke Pattern	67
9.2. Tier Classification	67
9.3. Conditional Rules	68
9.3.1. Advanced Rules Patterns	68

9.4. Settings: 5-Level Precedence	69
9.5. CLAUDE.md Length and Attention	69
9.6. Output Styles	69
9.7. Visual: The Configuration Hierarchy	70
10. Agents, Delegation, and Parallel Work	71
10.1. Subagents: Isolated Context Workers	71
10.1.1. Subagent Frontmatter Reference	72
10.1.2. Built-In Subagents	72
10.1.3. Continuing Subagent Work	72
10.2. When to Delegate	72
10.3. Designing Effective Agent Tasks	73
10.4. Git Worktrees: Isolated File Systems	74
10.4.1. Subagent Worktrees	74
10.5. The Writer/Reviewer Pattern	75
10.6. Agent Teams (Research Preview)	75
10.7. The <code>/batch</code> Skill: Parallel Codebase Changes	75
10.8. Quick Reference	76
11. Starting and Refactoring Projects	78
11.1. Greenfield: From Zero to Production	78
11.1.1. Day 1: Foundation	78
11.1.2. Week 1: Patterns Emerge	78
11.1.3. Month 1: Stabilization	78
11.2. Brownfield: Introducing Claude to Existing Code	79
11.2.1. Start with Understanding, Not Action	79
11.2.2. Incremental Adoption	79
11.3. The Incremental Refactoring Protocol	79
11.3.1. Characterization Tests	79
11.3.2. Golden Master Testing for Pipelines	80
11.4. Visual: The Incremental Refactoring Cycle	81
11.5. The Quality Flywheel	81
12. Anti-Patterns and Recovery	82
12.1. Context Overload	82
12.2. The Kitchen Sink Session	83
12.3. Over-Correcting	83
12.4. The Permanent Prototype	83
12.5. Pipeline Direction Violations	84
12.6. The Verification Gap	84
12.7. The Infinite Exploration	85
12.8. Big-Bang Refactoring	85
12.9. Systematic Diagnosis: The Four Failure Layers	86
12.10 Testing Your CLAUDE.md	87

IV. Team & Enterprise	88
13. Automation and Pipelines	90
13.1. Choosing Your Automation Mode	90
13.2. Headless Mode	91
13.2.1. Fan-Out Pattern	92
13.2.2. Permissions in Headless Mode	93
13.2.3. Bare Mode for Scripted Calls	93
13.3. Claude Agent SDK	93
13.4. Pipeline Design Patterns	94
13.4.1. Traffic-Light Dashboards	94
13.4.2. Pipeline Directionality	94
13.5. Scheduled Tasks	95
13.6. GitHub Actions Integration	95
13.7. Remote Control and Web Sessions	96
14. Team Patterns and Governance	98
14.1. Shared CLAUDE.md in Version Control	98
14.2. Managed Settings for IT	98
14.3. Onboarding New Team Members	100
14.4. Shared Extensions at Scale	100
14.5. Code Review Patterns with Claude	100
14.5.1. Pre-PR Self-Review	101
14.5.2. Reviewer-Assist	101
14.6. Quick Reference	101
15. Enterprise Deployment	102
15.1. Security and Compliance	102
15.1.1. Identity and Access Management	102
15.1.2. Data Protection	102
15.2. Deployment Scenario: 50-Person Rollout	103
15.3. Cost Optimization at Scale	103
15.3.1. API Pricing Reference	103
15.4. Enterprise Deployments at Scale	104
15.5. Competitive Positioning	104
15.6. Government and Regulated Industries	104
15.7. When to Deploy: Decision Matrix	105
A. The Claude Maturity Model	106
A.1. Five Levels of Claude Adoption	106
A.2. Upgrade Checklists	106
A.2.1. L1 → L2: From Conversational to Configured	106
A.2.2. L2 → L3: From Configured to Systematic	106
A.2.3. L3 → L4: From Systematic to Autonomous	107
A.2.4. L4 → L5: From Autonomous to Enterprise	107
A.3. Progression Strategy	107
A.4. Visual: Maturity Pyramid	108

B. Quick-Start Templates	109
B.1. Minimal CLAUDE.md (10 lines)	109
B.2. Production CLAUDE.md (50 lines)	109
B.3. Settings Template	110
B.4. Hook Configuration	110
B.5. Skill Template	111
B.6. Subagent Definition Template	111
B.7. Rules File with Paths	111
B.8. Custom Command Template	112
C. Command and Hook Reference	113
C.1. Built-In Commands	113
C.2. Bundled Skills	115
C.3. Session Management Flags	116
C.4. Keyboard Shortcuts	117
C.5. Hook Events (25 Total)	117
C.6. Hook Types (4 Total)	119
C.7. Permission Modes (6 Total)	119
C.8. Settings Precedence (5 Levels)	119
C.9. Environment Variables	119
D. Sources and Further Reading	121
D.1. Claude Code Documentation	121
D.2. Prompt Engineering and API	121
D.3. Enterprise and Governance	122
D.4. Partnership Announcements	122
D.5. Safety and Research	123
D.6. Community Resources	123
D.7. Further Reading	123
E. Glossary	124
F. What's New	127

Preamble

This handbook teaches you to use Claude Code effectively—not by listing features, but by walking through the workflows where each feature earns its value.

Two Sources of Knowledge

Every practice is drawn from one of two sources and labeled accordingly:

[Official]

Anthropic’s documented recommendation, verified against current documentation with source URL. Example: [Official] Run `/init` to generate a starter CLAUDE.md.¹

[Practitioner]

Discovered through extensive daily use across 20+ projects spanning data science, ML, and software engineering. Example: [Practitioner] After two failed corrections, `/clear` and write a better initial prompt.

[Convergence]

Both sources agree independently. Example: [Convergence] Verification criteria in CLAUDE.md from day one.

How This Book Is Organized

The handbook follows your adoption journey through four parts, each corresponding to a level of the Claude Maturity Model (Chapter A):

Part I: Foundations

(Part I) — What Claude Code is, your first CLAUDE.md, your first session. *Takes you from L1 (Conversational) to L2 (Configured).*

Part II: Personal Practice

(Part II) — Advanced prompting, context management, testing, extensions. *Takes you from L2 to L3 (Systematic).*

Part III: Advanced Craft

(Part III) — Configuration architecture, agents, project lifecycle, anti-patterns. *Takes you from L3 toward L4 (Autonomous).*

Part IV: Team & Enterprise

(Part IV) — Automation, team governance, enterprise deployment. *Scaling from L4 to L5 (Enterprise).*

Who This Is For

Primary audience: Data scientists and ML engineers who have used Claude Code at least once and want to go from “it works sometimes” to “it works reliably.” No prior configuration experience assumed.

Secondary audience: DS/ML team leads evaluating Claude for their teams. Chapter 15 is specifically designed to be shared with decision-makers.

¹<https://code.claude.com/docs/en/best-practices>

How to Read

New to Claude Code?

Start at Chapter 1 and read straight through Part I.

Already configured?

Start at Chapter 4 (Part II). Skim Part I for anything you missed.

Experienced user?

Jump to Chapter 12. Fix what is broken. Then read Chapter 9 for scaling patterns.

DS/ML team lead?

Read Chapter 15 and Chapter A, plus Chapter 5 and Chapter 7 for DS-specific workflow value.

Spiral Structure

Key concepts appear at increasing depth as you progress:

- **CLAUDE.md**: basics in Chapter 2, architecture in Chapter 9
 - **Context**: basics in Chapter 3, full management in Chapter 6
 - **Testing**: overview in Chapter 7, characterization tests in Chapter 11
 - **Extensions**: overview in Chapter 8, agents and delegation in Chapter 10
 - **Collaboration**: thinking partner in Chapter 5, writer/reviewer in Chapter 10
- Each revisit explicitly connects back to the earlier treatment.

A Note on Pace

AI tooling moves fast. Content is verified as of March 2026. Check the repository for updates, and consult Chapter D for primary source URLs.

Part I.

Foundations

You are at Level 1 (Conversational)—using Claude Code as a chatbot with file access. By the end of this part, you will be at Level 2 (Configured): Claude will know your project, respect your boundaries, and follow your conventions without being told twice.

1. Principles and Mental Model

You have just installed Claude Code. Before your first prompt, take five minutes to understand what you are working with—not a chatbot, not an autocomplete engine, but an agent with a specific architecture and specific constraints. The principles in this chapter will inform every decision in this handbook.

1.1. What Claude Code Actually Is

Claude Code runs an **agent loop**: it reads your prompt, reasons about the task, selects a tool (read a file, edit code, run a command), observes the result, and decides what to do next. This cycle repeats until the task is complete or Claude asks for clarification.

Three properties define the system:

Context window

Everything Claude knows about your task—your prompt, files it has read, tool results, conversation history—occupies a finite window. When the window fills, older information fades. Context is not infinite memory; it is a workspace.

Tool use

Claude does not type code into a file. It calls tools: Read, Edit, Write, Bash, Glob, Grep. Each tool call consumes context (the request and the result) and produces observable effects. Understanding this matters because every tool call has a cost in context tokens.

Configuration layers

Claude's behavior is shaped by four layers: `CLAUDE.md` (project knowledge), rules (conditional context), settings (permissions and behavior), and output styles (communication patterns). These layers are not suggestions—they are the operating system through which Claude interprets your project.

Why the Mental Model Matters

Developers who treat Claude as a chatbot prompt it like one: vague requests, no verification, no context management. Developers who understand the agent loop prompt it like a system: clear inputs, verification criteria, deliberate context control. The same tool produces dramatically different results depending on which mental model the developer holds.

1.2. Four Engineering Principles

These four principles underlie every recommendation in this handbook. They apply to code written by AI, code written by humans, and code written by both together.

1.2.1. Never Fail Silently

Every error must be explicitly reported with recovery options. Silent failures are the most expensive kind of bug: they produce incorrect results

Official

Claude Code is an agentic coding tool that operates directly in your terminal.

<https://code.claude.com/docs/en/overview>

that look correct, propagate through downstream systems undetected, and surface only when the damage is difficult to reverse.

Before & After

Before:

```
# CLAUDE.md
Handle errors gracefully.
```

After:

```
# CLAUDE.md
If you encounter an error, report it immediately
with the error message, your analysis of the root cause,
and 2-3 options for resolution.
Never work around errors silently.
```

In AI-assisted development, this principle has an additional dimension: Claude must be configured to surface problems rather than work around them. A vague instruction to “handle errors” invites Claude to catch and suppress. An explicit instruction to “report errors with analysis” makes every failure visible and actionable.

Tip

The “after” version is longer but produces better outcomes. Precision in error handling instructions compounds across every session.

1.2.2. Simplicity Over Complexity

Short functions. Flat structure. Self-documenting code. A 20-line function with a clear name is superior to a 5-line function with three levels of abstraction.

Why Simplicity Matters More with AI

Simple code is easier for Claude to reason about, test, and modify correctly. Complex abstractions increase the probability of subtle bugs in AI-generated code because each layer of indirection is another opportunity for misunderstanding. When Claude reads your codebase to learn conventions, simple patterns propagate accurately; complex patterns propagate errors.

Tip

In notebooks, this means: one cell, one purpose. Cells that do loading, cleaning, AND analysis violate this principle.

1.2.3. Immutability by Default

Return new data structures; mark mutations explicitly. Pure functions are easier to test, easier to parallelize, and easier for Claude to reason about.

When mutation is necessary (performance, I/O, state management), make it explicit: name the function `update_`, use mutable types, and document the side effects. The reader—human or AI—should never be surprised by what a function modifies.

1.2.4. Fail Fast with Diagnostics

Stop immediately on problems with full context: what failed, what was expected, what was received, and what the caller can do about it.

```
def process_data(
    df: pd.DataFrame, min_rows: int
) -> pd.DataFrame:
```

```

if len(df) < min_rows:
    raise ValueError(
        f"Need {min_rows} rows, got {len(df)}. "
        f"Check data source or reduce "
        f"min_rows parameter."
    )
# ... process

```

The error message includes: what went wrong (Need N, got M), where to look (data source), and what to do about it (reduce min_rows). This is the difference between a 10-second fix and a 30-minute investigation.

1.3. The Codebase Is the Curriculum

Key Insight

T

These principles are not Claude-specific—they are engineering principles that become *more* important when AI is generating code. AI amplifies both good and bad patterns. If your codebase follows these principles, Claude's contributions will follow them too. If your codebase is silent about errors, full of mutable state, and deeply abstracted, Claude's contributions will be the same. The codebase is the curriculum. Make it teach the right lessons. For data scientists, this includes your notebooks, pipeline scripts, and feature engineering code—Claude learns patterns from all of them.

This insight has a practical consequence: the single most valuable thing you can do before using Claude on an existing project is to ensure that the code Claude will read exemplifies the patterns you want Claude to produce. Configuration (Chapter 2) reinforces this, but the codebase itself is the primary teacher.

1.4. Quick Reference

- Claude Code is an agent loop: prompt → reason → tool → observe → repeat
- Context window is finite—every tool call costs tokens
- Four principles: never fail silently, simplicity, immutability, fail fast
- The codebase teaches Claude patterns—good and bad
- Configuration (next chapter) reinforces but does not replace good code

Try This

Pick one function in your feature pipeline or data loading code—ideally one Claude will read often. Audit it against the four principles:

1. Does it fail silently anywhere? (e.g., silently dropping NaN rows without logging)
2. Is it under 30 lines? If not, can you extract the transformation logic from the I/O?
3. Does it mutate its input DataFrame? If so, is that explicit in the name?
4. Do its error messages include what went wrong AND what to do about it?

Tip

Error messages that include both the violation and a suggested fix reduce debugging time dramatically.

Cross-Ref

For concrete techniques to make your codebase a better teacher, see Chapter 5.

Fix one issue. This is your first investment in the codebase-as-curriculum.

2. Your First CLAUDE.md

Claude keeps asking you the same questions about your project. It does not know your build command, your test runner, or your architecture. Every session starts with you re-explaining context that should be obvious. The fix takes five minutes.

2.1. What CLAUDE.md Does

[Official] The CLAUDE.md file is the single most important configuration artifact.¹ Claude reads it at session start and uses it to understand your project's conventions, build commands, architecture decisions, and rules of engagement.

Think of it as a briefing document for a new team member who is fast, capable, and amnesiac—they forget everything between sessions unless you write it down.

Official
Run `/init` to auto-generate a starter CLAUDE.md from your project structure.
<https://code.claude.com/docs/en/best-practices>

2.1.1. Hierarchical Loading

Claude loads CLAUDE.md files from multiple locations, merging them in order:

1. **Managed policy** (managed-settings.json CLAUDE.md) — IT-deployed, highest priority (enterprise only)
2. `~/claude/CLAUDE.md` — global defaults (applies to all projects)
3. Project root CLAUDE.md OR `.claude/CLAUDE.md` — project-level instructions
4. Parent and child directory CLAUDE.md files — subdirectory overrides

For now, you only need the project-level file. The global file and advanced architecture come later (Chapter 9).

2.2. Generate It: `/init`

The fastest path to a useful CLAUDE.md:

1. Open your project in Claude Code.
2. Type `/init`.
3. Claude examines your project and proposes a starter CLAUDE.md.
4. Review, edit, commit.

The auto-generated version is a solid starting point. What follows are the refinements that make it effective.

2.3. What to Include

[Convergence] Include what Claude cannot infer. Exclude what it can.²

Build and test commands

`pytest tests/, mypy src/, ruff check src/`—anything non-standard or project-specific.

¹<https://code.claude.com/docs/en/best-practices>

²<https://code.claude.com/docs/en/best-practices>

Code style rules

Only those that differ from standard conventions. Do not restate PEP 8 defaults.

Architecture decisions

Where things go and why. “Feature pipelines in `src/features/`. Models in `src/models/`. Notebooks in `notebooks/`.”

Verification criteria

The single highest-leverage inclusion. “Always run tests after code changes. Never commit without passing CI.”

Before & After**Before:****# My Project**

This is an ML project. We use scikit-learn and pandas.
Tests are important to us.

After:**# Demand Forecasting****## Build & Verify**

- ``pytest tests/`` – run test suite
- ``mypy src/`` – type checking
- ``ruff check src/`` – lint

Architecture

- `src/features/` – feature engineering pipelines
- `src/models/` – model training and evaluation
- `src/data/` – data loading and validation
- `notebooks/` – exploratory analysis
- `data/raw/` – immutable source data
- `data/processed/` – transformed artifacts
- `tests/` – mirrors `src/` structure

Standards

- Type hints on all function signatures
- Docstrings on all public functions
- Run tests after every code change
- Never commit without passing lint + tests

2.4. What to Exclude

- What Claude can infer from the codebase (standard language conventions)
- Long explanations—keep instructions terse and actionable
- Aspirational rules nobody follows—document what IS, not what you wish
- Information that changes frequently (put it in memory instead—see Chapter 6)

Tip

The “before” tells Claude about your project. The “after” tells Claude how to work on your project. Commands, locations, and rules—not descriptions.

Quality Test

If Claude keeps violating a rule, one of three things is true: (1) the CLAUDE.md is too long and the rule is getting lost, (2) the rule contradicts

another rule, or (3) the rule should be a hook instead of an instruction (see Chapter 8).

Write Positive Instructions

Replace negative constraints with positive alternatives. “Never use raw SQL” leaves Claude uncertain about what to use. “Always use the ORM query builder in `src/db/queries.py`” tells Claude exactly what to do instead. Positive instructions are clearer, more actionable, and more reliably followed.

2.5. Your First Deny Rules

Before Claude touches any code, protect your secrets. Create `.claude/settings.json`:

```
{
  "permissions": {
    "deny": [
      "Read(.env*)",
      "Read(secrets/**)",
      "Read(**/*.pem)",
      "Read(**/*.key)",
      "Read(**/credentials*)"
    ]
  }
}
```

Deny rules are the one configuration that is never wrong to add on day one. They cost nothing in context, they never cause false negatives in normal work, and they prevent the one class of mistake that is truly irreversible: leaking secrets into a commit, a log, or a conversation.

2.6. Quick Reference

- Run `/init` to generate a starter CLAUDE.md
- Include: build commands, architecture, code style, verification criteria
- Exclude: what Claude can infer, long prose, aspirational rules
- Add deny rules for secrets in `.claude/settings.json` on day one
- Commit both files to version control

Try This

1. Run `/init` on your current project.
2. Compare the generated CLAUDE.md to what you actually need. Add `pytest`, `mypy`, and `ruff` commands if missing.
3. Create `.claude/settings.json` with deny rules for `.env*`, `data/raw/**`, and any credentials directories.
4. Start a new session and verify Claude reads your CLAUDE.md (ask: “What are my build commands?”).

Time: ~5 minutes.

Official
Settings documentation:
<https://code.claude.com/docs/en/settings>

Tip

Deny rules are absolute. Unlike CLAUDE.md instructions (which are advisory), deny rules cannot be overridden by conversation or prompt injection.

3. Your First Working Session

Your *CLAUDE.md* is set up and your secrets are protected. You open Claude Code, type a prompt, and...where do you start? How do you structure a request so Claude does what you mean? How do you know when to start fresh versus keep going?

By the end of this chapter, you will have:

- A repeatable session loop: prompt → observe → refine → commit
- Three prompting techniques that handle 80% of interactions
- A context hygiene discipline (two commands, one rule)
- Escape routes for when things go wrong

3.1. The Session Loop

A productive session with Claude follows a repeating cycle:

The Session Loop

1. **Prompt** — describe what you want, with enough context for Claude to act without guessing.
2. **Observe** — read what Claude did. Check the diff, run the tests, examine the output.
3. **Refine** — if the result is close but not right, give specific feedback. If it is wrong, start fresh (`/clear`).
4. **Commit** — when the change is verified, commit it. One logical change per commit.

The rest of this chapter teaches three essential skills for the prompt and refine steps: how to write clear requests, how to provide useful feedback, and how to manage the conversation so it stays productive.

3.2. Three Prompting Basics

These three techniques handle 80% of daily interactions. Chapter 4 covers advanced prompting in depth; this section gives you enough to be productive today.

3.2.1. Be Specific About What, Not How

Before & After

Before:

Fix the broken pipeline.

After:

The feature engineering pipeline in `src/features/temporal.py` raises a `KeyError` on line 34. The error is: `KeyError: 'timestamp'`. The column exists in training data but is missing from the inference schema. Diagnose and fix.

Cross-Ref

If Claude goes off track: `Esc` stops mid-action, `Esc+Esc` opens the rewind menu. Full details in Section 3.4.

The “before” makes Claude guess which tests, what kind of failure, and what “fix” means. The “after” gives Claude the information it needs to act precisely.

3.2.2. Tell Claude to Think Step by Step

For complex tasks, give Claude an explicit reasoning structure:

```
Refactor the feature engineering pipeline:
1. Read the current implementation and
   list all transformation steps.
2. Identify steps that mix data loading
   with transformation.
3. Extract pure transformation functions.
4. Run existing tests after each extraction
   to verify nothing broke.
```

Official
Step-by-step prompting is consistently identified as one of the three highest-leverage techniques.
<https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>

Without this structure, Claude may attempt the entire refactoring in one pass, increasing the risk of subtle regressions.

3.2.3. Include Verification Criteria

[Convergence] “Give Claude a way to verify its own work” is Anthropic’s stated single highest-leverage practice.¹

Always include how to check the result:

```
Add input validation to the build_features() function.
Verify: pytest tests/features/test_build.py passes,
and test with:
python -c 'from src.features import build_features;
          build_features(pd.DataFrame())'
(should raise ValueError, not silently return empty).
```

When Claude knows the success criteria, it can self-correct before you ever see the result.

From Typing to Defining

You just learned three techniques: specificity, step-by-step reasoning, and verification criteria. Notice the shift—none of them are about writing code. They are about defining what “correct” means precisely enough that Claude can produce it. Your value moved from implementation to specification. The rest of this chapter protects that investment: keeping context clean so these techniques stay effective.

3.3. Context Hygiene: The Basics

The context window (introduced in Section 1.1) is your workspace. Like a physical desk, it becomes less effective as it fills with unrelated materials.

3.3.1. Two Commands to Know Now

/clear

Wipe context and start fresh. Use between unrelated tasks: after fixing a bug, before writing a feature.

Official
Use /clear between unrelated tasks.
Use /compact for selective summarization.
<https://code.claude.com/docs/en/best-practices>

¹<https://code.claude.com/docs/en/best-practices>

/compact

Summarize the conversation while preserving key decisions. Less aggressive than `/clear`; use when you need to continue a long task but context is getting large.

3.3.2. The Two-Failure Rule

[Practitioner] After two failed corrections on the same issue, `/clear` and write a better initial prompt.

Each correction attempt adds noise: the original error, your correction, Claude's apology, and its retry. After two rounds, the context is dominated by failure patterns rather than the actual task. Starting fresh with a better prompt costs 30 seconds and produces better results than a third correction round.

When to Clear

If you find yourself typing "no, that's not what I meant" for the third time, stop. `/clear`. Write the prompt you wish you had written the first time. The cost of accumulated correction context exceeds the cost of starting fresh.

3.4. Course-Correcting

[Convergence] The best results come from tight feedback loops. Correct Claude as soon as you notice it going off track.²

Esc Stop Claude mid-action. Context is preserved, so you can redirect immediately.

Esc + Esc

Open the rewind menu: restore conversation, code, or both to any previous checkpoint.

``Undo that''

Have Claude revert its changes.

/clear

Reset context entirely between unrelated tasks.

Every action Claude takes creates a checkpoint automatically. Think of checkpoints as free save points—you can always go back if Claude takes a wrong turn.

Why This Matters

You now have the tools to recover from any wrong turn. The next section adds one more capability: letting Claude explore and plan *before* it builds, so wrong turns happen less often.

3.5. Plan Mode: Explore Before You Build

For complex or unfamiliar tasks, let Claude understand the codebase before making changes. Use `Shift+Tab` to cycle through permission modes until you reach Plan Mode—Claude can read files and answer questions but cannot modify anything. Use `Ctrl+G` to open your prompt in your default text

Tip

If you have corrected Claude more than twice on the same issue, the context is cluttered with failed approaches. `/clear` and start fresh with a better prompt.

Official

Separate research from implementation to avoid solving the wrong problem.

<https://code.claude.com/docs/en/best-practices>

²<https://code.claude.com/docs/en/best-practices>

editor for detailed editing, or start directly in Plan Mode from the command line with `claude --permission-mode plan`.

[Official] The recommended workflow for complex tasks has four phases:³

1. **Explore** (Plan Mode): “Read `src/features/` and explain how we compute temporal features.”
2. **Plan** (Plan Mode): “I want to add recency features. What files need to change? Create a plan.”
3. **Implement** (Default Mode): “Implement the plan. Write tests first, then code. Run the test suite.”
4. **Commit**: “Commit with a descriptive message.”

When to Skip Planning

Planning adds overhead. Skip it when the scope is clear and the fix is small (typo, log line, variable rename). If you could describe the diff in one sentence, just ask Claude to do it directly. Plan when you are uncertain about the approach, the change spans multiple files, or you are unfamiliar with the code.

Official

Auto mode requires

`--enable-auto-mode` and a Team plan or higher. Uses a background classifier for safety checks.
<https://code.claude.com/docs/en/permission-modes>
`bypassPermissions (settings/CLI)`.
<https://code.claude.com/docs/en/permissions>

3.6. A Complete Example

Here is a realistic mini-session showing the loop in practice:

```
You: Add missing value imputation to the preprocessing
      pipeline. Strategy: median for numeric columns,
      mode for categorical. Write tests first.
      Verify: pytest tests/preprocessing/ passes.

Claude: [writes tests, then implementation, runs tests – 3 pass, 1 fails]
        "Test test_categorical_imputation is failing because..."

You: Mode should use the training set distribution,
      not the test batch. Fix the imputer to fit on train.

Claude: [fixes implementation, reruns – all pass]
        "All 4 tests pass. Ready to commit?"

You: Yes, commit with message "feat: Add missing value imputation"
```

Notice the structure:

- The initial prompt included what, how (strategy), and verification criteria.
- The refinement was specific (which behavior, what to change).
- One logical change, one commit.

3.7. Quick Reference

- Session loop: prompt → observe → refine → commit
- Be specific about *what* you want, not *how* to do it
- Include verification criteria in every prompt
- Plan Mode (Shift+Tab) for complex or unfamiliar tasks
- Esc to stop mid-action, Esc+Esc to rewind

³<https://code.claude.com/docs/en/best-practices>

- Use `/clear` between unrelated tasks
 - Two-failure rule: after two corrections, start fresh
- What you practiced:**
- Specification: writing prompts that include what, context, and verification criteria
 - Observation: reading diffs and test results before approving
 - Context discipline: clearing between tasks, compacting when long
 - Course-correction: stopping, rewinding, starting fresh

Try This

Start a session with Claude Code on your current project. Add one feature to your data pipeline or fix one data processing bug:

1. Write a prompt that includes: what you want, the relevant DataFrame schema, and how to verify (e.g., `pytest tests/features/`).
2. Let Claude work. Observe the result.
3. If refinement is needed, give specific feedback (file, line, which column or transformation to change).
4. When satisfied, commit the change.

Track how many refinement rounds it takes. If it exceeds two, try `/clear` and a better initial prompt.

Part II.

Personal Practice

You are at Level 2 (Configured)—Claude knows your project and follows basic rules. This part takes you to Level 3 (Systematic): you will manage context deliberately, test AI-generated code rigorously, and extend Claude with custom capabilities that encode your team's workflows.

4. Prompting That Works

Your prompts work, but they are inconsistent. Sometimes Claude nails it on the first try; sometimes you spend three rounds correcting misunderstandings. The gap is not randomness—it is precision.

In Chapter 3, you learned three prompting basics: be specific, think step by step, include verification criteria. This chapter goes deeper: building a precision vocabulary, structuring complex prompts, managing cost, and using extended thinking.

4.1. Building a Precision Vocabulary

[Practitioner] Track how your requests could be stated more precisely. Over time, develop a shared vocabulary between you and Claude.

Natural Language	Precise Version
"Clean this data"	<code>validate schema, impute nulls with median, drop rows where target is NaN</code>
"Train a model"	<code>fit XGBoost on train split, evaluate AUC on val, log params to MLflow</code>
"Check if this works"	<code>validate: run pytest, check no data leakage, verify feature distributions match prod</code>
"Add error handling"	<code>add validation to build_features(): raise ValueError on schema mismatch, log and skip rows with >50% NaN</code>

Why Precision Vocabulary Compounds

Precision is not just about saving correction rounds in a single session. It builds a shared language between you and Claude. When your CLAUDE.md says "validate schema" and your prompts say "validate schema," Claude interprets both consistently. This alignment compounds: your prompts become shorter, your CLAUDE.md rules more effective, and your correction rate drops—not linearly, but exponentially as the vocabulary stabilizes.

The pattern for building vocabulary:

1. After each interaction, note how the request could have been clearer.
2. Identify recurring request patterns (create, diagnose, refactor, validate).
3. Document these patterns in your project nomenclature.
4. Use the precise version in future requests.

Tip

Precision compounds. Practitioners report that shared vocabulary can reduce correction rounds by $2-3\times$ —a plausible estimate, though hard to measure precisely and workload-dependent.

4.2. Structuring Complex Prompts

4.2.1. XML Tags for Multi-Component Prompts

Cross-Ref

For using precision prompts in collaborative reasoning, see Chapter 5.

[Official] For complex prompts, use XML tags to separate concerns:¹

```
<context>
The churn prediction model uses features from
src/features/customer.py. Current AUC is 0.72
on validation.
</context>

<task>
Add recency features: days since last purchase,
days since last login.
Compute from the events table.
</task>

<constraints>
- No data leakage: features must use only
  pre-churn data
- Must work with the existing FeatureStore
  interface
- Include unit tests for the new features
</constraints>

<verification>
Run: pytest tests/features/ -v
All existing tests must still pass.
Verify: no future-dated features in test set.
</verification>
```

4.2.2. Rich Inputs

[Official] Use `@file` to include files, paste screenshots, pipe data from `stdin`.² Real data beats verbal descriptions.

Method	When to Use
<code>@file</code> or <code>@dir/</code>	Load file or directory content before Claude responds. Preferred over describing code.
Image paste	Copy/paste or drag-and-drop screenshots, diagrams, error messages directly into the prompt.
<code>cat file claude</code>	Pipe data from other tools. Useful for logs, command output, or data samples.
URL allowlisting	Use <code>/permissions</code> to allow frequently-used doc sites. Claude can then fetch reference material directly.

```
Look at @src/features/customer.py
and @tests/features/test_customer.py.
The feature builder computes 12 features
but has no validation for missing values.
Add validation following existing patterns.
```

¹<https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>

²<https://code.claude.com/docs/en/best-practices>

Tip

`@` references load content before Claude responds—more efficient than asking Claude to Read the file itself, and it ensures the context is available from the first reasoning step.

```
# Pipe error logs for diagnosis
tail -100 app.log | claude -p "Diagnose the errors"

# Pipe test output for analysis
pytest tests/ -v 2>&1 | claude -p "Summarize failures"
```

4.3. Controlling Scope

Claude’s default behavior is helpful: it reads broadly, notices related issues, and fixes them. But this helpfulness becomes scope creep when it modifies a file you were not working on, refactors a function you did not mention, or “improves” code that was intentionally written that way.

Tip

The most common daily annoyance: Claude modifies files you did not ask it to touch.

The Scope Spectrum

Too restrictive:

“Only modify line 42 of `auth.py`.”

Claude cannot fix related issues that the change requires.

Too permissive:

“Fix the auth system.”

Claude may rewrite half the codebase.

Right scope:

“Fix the session expiry bug in `src/auth/session.py`. You may also modify `src/auth/middleware.py` if the fix requires it. Do not touch other files without asking.”

Three Techniques for Scope Control

1. **Name the files.** Explicitly list which files Claude may modify. “Modify `src/auth/session.py` and its tests. Do not modify other files.” This is the simplest and most effective technique.
2. **Separate discovery from action.** First ask Claude to *analyze* without changing anything: “Read `src/payments/` and tell me what needs to change to support refunds. Do not make changes yet.” Review the analysis, then approve specific changes.
3. **Use plan mode.** Shift+Tab to plan mode lets Claude describe proposed changes without executing them. Review the plan, adjust scope, then switch to a mode that allows edits. This is especially valuable for unfamiliar codebases.

[Practitioner] When Claude makes an unsolicited change, do not just undo it—add a scope constraint to the next prompt. The pattern: “Good fix for the session bug. But you also changed `user_model.py`—please revert that. For the rest of this session, only modify files I explicitly name.” Claude learns the constraint for the remainder of the session.

Key Insight

S

cope creep is not a model deficiency—it is a prompting gap. Claude modifies extra files because the prompt did not say not to. The fix is always in the prompt (name the files, separate discovery from action, use plan

mode), never in hoping Claude will guess your boundaries.

4.4. Extended Thinking

[Official] Extended thinking lets Claude reason through complex problems before responding.³ On Opus 4.6, thinking is **adaptive**—the model decides how much to think based on problem complexity.

4.4.1. Effort Controls

low Minimal thinking. Best for: simple lookups, formatting, boilerplate.

medium

Moderate analysis. Best for: standard coding tasks, code reviews.
Default for Max/Team subscribers on Opus 4.6.

high Deep reasoning. Best for: architecture decisions, complex debugging.

max Deepest reasoning with no token constraint. **Opus 4.6 only, session-scoped** (does not persist across sessions). Best for: strategic decisions, novel problems requiring exhaustive analysis.

auto Resets effort to the model's default behavior. Use when you want to undo a manual effort override and let the model choose its own reasoning depth.

[Practitioner] The keyword `ultrathink` in any prompt escalates to **high** effort for one turn—useful for ad-hoc deep analysis without changing your session default.

[Official] Interleaved thinking⁴: Claude can think between tool calls, enabling nuanced multi-step workflows where each step's reasoning informs the next.

4.4.2. When to Enable vs. Disable

Extended Thinking

Enable when: the task requires multi-step reasoning, architectural decisions, debugging where the root cause is unclear, or code review where you want to see the thought process.

Disable when: the task is simple (formatting, boilerplate), latency matters more than depth, or you are running bulk operations where speed dominates.

Budget note: Thinking tokens add to your bill and consume context budget. In a long session with context already at 50%, heavy thinking can push you past the degradation threshold faster. Monitor `/context` when using extended thinking in long sessions.

4.5. Cost Optimization

4.5.1. Prompt Caching

[Official] Prompt caching reduces input costs by 90% for repeated con-

³<https://platform.claude.com/docs/en/build-with-claude/extended-thinking>

⁴<https://platform.claude.com/docs/en/build-with-claude/extended-thinking>

Official

On by default. Toggle: Alt+T. Opus 4.6: adaptive thinking with effort controls.
<https://platform.claude.com/docs/en/build-with-claude/extended-thinking>

Official

Set effort: `/effort`
`low|medium|high|max|auto`, model picker slider, or
`CLAUDE_CODE_EFFORT_LEVEL` env var.
<https://code.claude.com/docs/en/model-config>

Cross-Ref

Extended thinking is especially valuable during collaborative design sessions (Chapter 5).

Cross-Ref

Thinking blocks are compaction-resistant: auto-compaction cannot summarize them. See Chapter 6 for mitigation.

Cost

Prompt caching: up to 90% on cached reads. Batch API: 50% discount. Combined: up to 70%+ savings (workload-dependent).
<https://platform.claude.com/docs/en/build-with-claude/prompt-caching>

tent.⁵

Ephemeral (5-minute TTL)

Automatic for content appearing in multiple requests within a short window.

Persistent (1-hour TTL)

Explicit cache breakpoints for content you know will be reused: tool definitions, system prompts, reference documents.

4.5.2. Batch API

[Official] 50% discount for asynchronous processing.⁶ Submit up to 100,000 requests per batch (256 MB max). Most batches complete within 1 hour; results available within 24 hours. Combinable with caching for up to 70%+ total savings (workload-dependent).

Best for: bulk migrations, code review across many files, documentation generation.

4.5.3. Model Selection

Model Class	Strength	Best For
Haiku 4.5	Speed, lowest cost	Simple tasks, high volume, formatting
Sonnet 4.6	Balance of speed and capability	Most coding tasks, daily development
Opus 4.6	Frontier reasoning	Complex architecture, multi-file refactoring

[Practitioner] Default to Sonnet for development work. Escalate to Opus for architecture decisions, complex debugging, and multi-file refactoring. Use Haiku for bulk operations and simple formatting tasks.

4.5.4. Fast Mode (Research Preview)

[Official] Fast mode makes Opus 4.6 responses 2.5× faster at higher per-token cost (\$30/\$150 per MTok). Same model, same quality—different API configuration prioritizing speed.⁷

Toggle

/fast or Alt+0. A bolt icon appears next to the prompt while active.

Cost

Flat pricing across the full 1M context window. Billed to extra usage, not plan limits.

Fallback

On rate limit, automatically reverts to standard Opus (icon grays out), then re-enables when cooldown expires.

Persistence

On by default across sessions. Admins can require per-session opt-in via `fastModePerSessionOptIn`.

Cost
Pricing changes frequently. Check current rates before planning budgets.
<https://platform.claude.com/docs/en/about-claude/pricing>

⁵<https://platform.claude.com/docs/en/build-with-claude/prompt-caching>

⁶<https://platform.claude.com/docs/en/build-with-claude/batch-processing>

⁷<https://code.claude.com/docs/en/fast-mode>

Warning
Switching to fast mode mid-conversation reprices the *entire* cached context at the fast rate. Enable at session start for best cost efficiency.

Fast Mode vs. Effort Level

Fast mode: same quality, lower latency, higher cost.

Lower effort: less thinking, faster responses, potentially lower quality on complex tasks.

Combine both: fast mode + low effort for maximum speed on straightforward tasks.

4.6. Visual: Cost Optimization Decision Tree

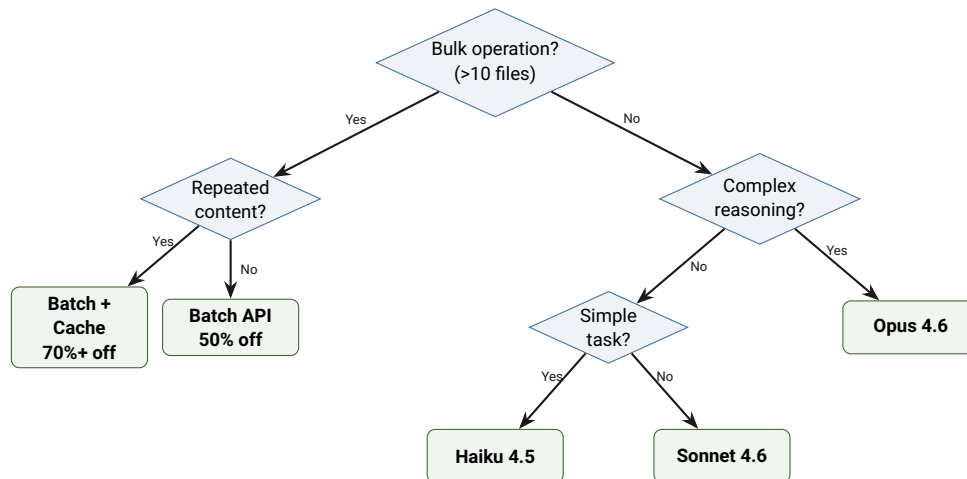


Figure 4.1.: Model and API selection decision tree for cost optimization.

Key Insight

T

he most cost-effective pattern is not choosing the cheapest model—it is structuring your workflow to maximize cache hits. With stable, repeated content, a well-organized CLAUDE.md and tool definitions can reduce per-session costs by up to 60–80% through caching (workload-dependent; assumes high cache-hit rates on stable content).

4.6.1. Model Capabilities Reference

Model	Max Output	Context	Knowledge Cutoff	Thinking
Haiku 4.5	64K tokens	200K	Feb 2025	Extended
Sonnet 4.6	64K tokens	1M	Aug 2025	Adaptive
Opus 4.6	128K tokens	1M	May 2025	Adaptive
Opus 4.6 (fast)	128K tokens	1M	May 2025	Adaptive

Official

Capabilities verified March 2026. For pricing, see Chapter 15.

<https://platform.claude.com/docs/en/docs/about-claude/models>

Try This

Take your last three prompts to Claude (check terminal history). Rewrite each using the precision vocabulary table:

1. Replace vague verbs (“clean,” “train,” “check”) with precise ones (“validate schema and impute nulls,” “fit on train split and evaluate AUC,” “run pytest and verify no leakage”).

2. Add verification criteria to any prompt that lacked them.
3. Estimate: how many correction rounds would each rewrite have saved?

5. Thinking Together

Your prompts are precise, your CLAUDE.md is configured, and your tests pass. But every interaction follows the same pattern: you delegate, Claude executes, you verify. Something is missing—you are using Claude as a tool you configure, not a collaborator you think with.

The previous chapters answer “how do I tell Claude what to do?” This chapter answers a different question: how do I *think together* with Claude?

The core metaphor: Claude is a thinking mirror. It reflects your code-base back to you. Where the reflection is clear, your code is clear. Where it is distorted—where Claude misunderstands, guesses, or asks for clarification—you have found a documentation gap, a hidden assumption, or an unclear interface.

An Honest Caveat

Claude is a thinking partner with no memory, a tendency to agree, and occasional confident wrongness. The techniques in this chapter work *because* they structure the collaboration to counteract these weaknesses—not despite them.

Why This Chapter Exists

Every chapter so far answers “how do I configure Claude to do what I want?” This one answers “how do I use Claude to think more clearly?” The shift is from delegation to collaboration—from configure-delegate-verify to think-together-discover-build-better.

5.1. Hypothesis-Driven Debugging

When a bug appears, the default instinct is to paste the traceback and say “fix this.” Claude will try—and often succeed on the first attempt. But for deeper bugs, this approach produces patches that address symptoms rather than root causes.

[Practitioner] Structure debugging as hypothesis testing, not error correction. Claude is a pair programmer who catches different errors because it has no sunk cost in your approach.

Debugging

Before:

```
Fix this error:
File "feature_pipeline.py", line 47
ValueError: negative value in non-negative
feature 'days_since_last_purchase'
```

After:

```
I see a ValueError in feature_pipeline.py
line 47: negative values in a feature that
should be non-negative. Three hypotheses:
```

1. Log transform applied before clipping
negative deltas

```

2. Currency conversion introduces negatives
   for returns/refunds
3. Timezone mismatch causes date subtraction
   overflow

For hypothesis 1: write a test with a purchase
  date after the reference date.
For hypothesis 2: write a test with a refund
  transaction in the events table.
For hypothesis 3: check if timestamps are
  timezone-aware or naive.

Run hypothesis 1 first---it is the most
likely given the stack trace.

```

The second prompt is longer but produces better outcomes: a root-cause diagnosis instead of a symptom patch. Each hypothesis becomes a testable prediction. When the test confirms one, you fix the cause, not the effect.

Tip

"Three hypotheses for this failure. Design a minimal test for each."

Key Insight

Rubber duck debugging works because articulating the problem reveals the gap. Claude-as-duck works better because the duck talks back—it asks questions you would not ask yourself, because you already “know” the answer. (Sometimes you are wrong about what you know.)

Tip

"Run this one test with this one input. Does it pass?" Isolation is the fastest path to root cause.

Try This

Pick a recent data pipeline bug—ideally one where results looked plausible but were wrong (e.g., feature drift, silent NaN propagation, leakage). Write three hypotheses for what caused it. Ask Claude to design a minimal test for each hypothesis. Would this approach have found the root cause faster?

5.2. Tests as Thinking Tools

In Chapter 7, tests serve verification: does the code do what it claims? Here, tests serve a different purpose—exploration: *what should the code do at all?*

Tests: Verification vs. Exploration**Before:**

"Write tests for this function."

After:

"I am not sure what should happen at the boundary. Write 5 test cases exploring: empty input, single element, duplicates, negatives, overflow. Which behaviors surprise me?"

Why Tests Before Requirements

Writing test cases forces you to articulate expectations you have not stated. When a test case surprises you—the function does something you did not expect—you have discovered a requirement that was implicit. The test did not verify your code. It interrogated your assumptions.

Tip

"What invariant should always hold for this function, regardless of input?" This question surfaces design decisions hiding as implementation details.

5.2.1. Property-Based Tests as Assumption Detectors

[Practitioner] Property-based tests are particularly powerful as thinking tools. Instead of testing specific inputs, they test *invariants*—properties that must hold for all inputs. (These are the same property-based tests from the sixth layer of Chapter 7’s validation architecture, applied here as a thinking tool rather than a verification gate.)

```
Prompt: "What properties should always be true
for this sort function?
- Output length == input length
- Output is ordered
- Output is a permutation of input
Write a property-based test for each."
```

When a property-based test fails on a randomly generated input, it has found an edge case neither you nor Claude anticipated. That failure is more valuable than a passing test—it reveals a boundary in your understanding.

5.3. Surfacing Hidden Assumptions

Every system rests on assumptions—about scale, about usage patterns, about what will never change. Most are invisible until they break. Claude can surface them, not because it is smarter, but because it has no investment in your preferred approach.

Cross-Ref

For test infrastructure and verification criteria, see Chapter 7. For characterization tests, see Chapter 11.

Architecture Review

Before:

"Review my database schema."

After:

"Here is my feature store schema. I designed it assuming: (1) features are computed daily in batch, not real-time, (2) training and serving use the same feature computation, (3) feature drift is monitored externally. Which assumption is most likely wrong in 6 months, and what breaks when it does?"

The second prompt names the assumptions explicitly, then asks Claude to stress-test them. This is not code review—it is assumption review.

The Pre-Mortem Prompt

"Imagine this feature has failed in production six months from now. What are the three most likely causes? Work backwards from failure to the design flaw that enabled it."

Pre-mortems are more effective than post-mortems because they cost nothing and can change the design. Ask Claude to run one before every architecture decision.

The Feynman Prompt

"Explain my auth flow as if I just joined the team and need to modify it. Where did you have to guess because the code does not make intent clear?"

Gaps in Claude’s explanation are gaps in your documentation. What Claude

cannot explain, a new hire cannot understand.

5.4. Getting Genuine Comparisons, Not Validation

Claude has a well-documented tendency to validate whatever approach you present first. If you say “I am thinking about using Redis for caching,” Claude will explain why Redis is a good choice. If you had said “I am thinking about using Memcached,” Claude would have explained why Memcached is a good choice.

This sycophancy is the biggest obstacle to using Claude as a genuine thinking partner. The fix is structural, not attitudinal— you cannot ask Claude to “be honest.” You can structure prompts that make honest comparison the path of least resistance.

Technique 1: Present Options Without a Preference

Architecture Decision

Before:

“I think we should use Redis for our caching layer. What do you think?”

After:

“We need a caching layer. The options are Redis, Memcached, and an in-process LRU cache. For each option, list: (1) what it handles well, (2) what it handles poorly, and (3) one scenario where it would be the wrong choice. Then recommend one, with the specific tradeoff that makes it better for our use case.”

The first prompt has a correct answer (agree with you). The second prompt has no obvious “right” answer—Claude must actually reason about tradeoffs.

Technique 2: Argue the Other Side

After Claude recommends an approach, explicitly ask it to argue against:

“Good analysis. Now argue against your recommendation.
What is the strongest case for NOT using Redis here?
What would have to be true about our workload for
Memcached to be the better choice?”

This forces Claude to find real weaknesses in its own recommendation. If the counterargument is weak, the recommendation is probably sound. If it is strong, you have discovered a genuine tradeoff worth investigating.

Technique 3: The Devil’s Advocate Session

For critical decisions, start a separate session with an explicit adversarial role:

“You are a senior engineer who believes our current
architecture decision (Redis caching layer) is wrong.
Make the strongest possible case against it. Do not
hold back---I need to hear the real risks before we commit.”

Tip

“What am I assuming that I have not written down?” The most dangerous assumptions are the ones nobody states.

Warning

Claude defaults to agreeing with you. Structure the prompt to counteract this.

The separate session is important: Claude in the original session has accumulated context that biases toward the decision. A fresh session with an adversarial frame produces genuinely different analysis.

[Practitioner] The pattern generalizes: for any important decision, generate the recommendation in one session and the critique in another. The divergence between them is the measure of decision quality. If both sessions agree, you can proceed with confidence. If they disagree sharply, you have found the crux of the decision.

5.5. Quick Wins That Compound

These five techniques take minutes to apply and improve every subsequent Claude interaction. The metaphor: Claude reads your codebase like a developer who just joined the team. Good onboarding materials help Claude the same way they help humans.

5.5.1. Docstrings as Triple-Duty Documentation

A docstring serves three audiences simultaneously: you (in six months), Claude (right now), and your teammates (always).

Docstrings

Before:

```
def calculate_risk(portfolio, window):
    scores = []
    for asset in portfolio:
        s = _score(asset, window)
        if s is not None:
            scores.append(s)
    return np.mean(scores) if scores else 0.0
```

After:

```
def calculate_risk(
    portfolio: list[Asset],
    window: int = 30,
) -> float:
    """Average risk score across portfolio.

    Args:
        portfolio: Assets to evaluate.
            Empty portfolio returns 0.0.
        window: Lookback days. Default 30.

    Returns:
        Mean risk score in [0, 1].
        Returns 0.0 if no assets scorable.

    Note:
        Skips assets where _score() returns
        None (insufficient history).
        This is intentional, not an error.
    """
```

The “Note” section is the highest-value addition: it explains a design decision that would otherwise look like a bug. Claude will not misinterpret

the `None` handling because the intent is documented.

5.5.2. Type Hints as Contracts

Five seconds to write, five minutes of debugging saved. `window: int = 30` tells Claude the type, the default, and the name—three constraints in five characters. Without the hint, Claude may pass a string, a float, or a `timedelta`.

5.5.3. Error Messages That Debug Themselves

This connects directly to Principle 4 from Section 1.2: fail fast with diagnostics. An error message that includes what went wrong, what was expected, and what to do about it saves Claude the same investigation time it saves you.

5.5.4. Code Archaeology

For brownfield work, Claude makes an excellent excavation partner:

```
Prompt: "Why might the original author have
written this as a nested loop instead of a join?
What constraint explains this design choice?"
```

Instead of assuming the code is wrong, this prompt assumes the code is *explained by something you do not yet see*. Claude often finds the constraint—a database limitation, a legacy API, a performance requirement—that makes the original design rational.

5.5.5. README-Driven Development

Write the README first. Then ask Claude:

```
Prompt: "Read this README. What questions does a
new developer still have after reading it?"
```

Try This

Pick one function Claude frequently reads in your project. Add a docstring with Args, Returns, and one Note explaining a non-obvious design decision. Compare Claude's output on a task involving that function before and after the docstring.

Tip

"Read this function. What would you need to know before modifying it?" Gaps in the answer are gaps in your code.

5.6. Let Claude Interview You

[Official] For larger features, have Claude interview you before implementation. Start with a minimal prompt and ask Claude to dig into the hard parts.¹

```
I want to build a feature drift monitoring system.
Interview me in detail. Ask about:
- Technical implementation
```

¹<https://code.claude.com/docs/en/best-practices>

- Data sources and schemas
- Edge cases and failure modes
- Tradeoffs I might not have considered

Keep interviewing until we have covered everything,
then write a complete spec to SPEC.md.

Once the spec is complete, start a fresh session to implement it. The new session has clean context focused entirely on implementation, and you have a written spec to reference.

Why Interviews Beat Requirements Documents

You do not know what you do not know. When you write requirements, you only state what you have already thought of. When Claude interviews you, it asks about things you have not considered—edge cases, failure modes, infrastructure constraints. The result is more complete requirements with less effort.

5.7. Architecture Decision Records

When you make an architecture decision *with* Claude, the conversation captures not just what you decided but why, and what alternatives were considered. An Architecture Decision Record (ADR) preserves this reasoning for your future self.

ADR-007: Offline Feature Computation

Context

Feature computation runs in nightly batch.
Some features are stale by 12 hours at serving time.

Decision

Keep batch for training features.
Add streaming for 3 real-time features
(last login recency, cart value, session count).

Alternatives Considered

1. All streaming (rejected: roughly 10x infrastructure cost)
2. Faster batch, hourly (rejected: still stale)
3. Feature caching with TTL (rejected: cache invalidation complexity)

Consequences

- Two feature computation paths to maintain
- Real-time features need monitoring for drift
- Training/serving skew possible for 3 features

Assumptions to Revisit

- 3 real-time features sufficient for next 6 months
- Streaming infra handles peak load (Black Friday)
- Feature drift alerts catch training/serving skew

Key Insight

The most valuable ADR section is “Alternatives Considered.” Claude suggests alternatives you would not—not because it is smarter, but because it has no investment in your preferred approach. A human colleague might hesitate to challenge your solution. Claude does not hesitate.

5.8. Quick Reference

- Claude is a thinking mirror: distortions reveal gaps in your code
- Debugging: three hypotheses, minimal tests, isolate blame
- Tests as exploration: discover requirements, not just verify them
- Interviews: let Claude ask what you have not considered
- Assumption surfacing: name your assumptions, then stress-test them
- Pre-mortem: imagine failure, work backwards to design flaw
- Docstrings, type hints, error messages: minutes to add, compound forever
- Code archaeology: assume intent before assuming error
- ADRs: capture alternatives considered, not just the decision

5.8.1. Building a Prompt Pattern Library

The techniques in this chapter can be codified as team-shared resources. The hypothesis debugging workflow becomes a custom command. The pre-mortem prompt becomes a line in your shared CLAUDE.md. The ADR template becomes a skill.

[Practitioner] Start personal, then share. Once a prompt pattern saves you time three sessions in a row, promote it from your terminal history to a command or skill that the whole team can use.

Try This

Choose one technique from this chapter and apply it today:

1. **Easiest:** Add a docstring to one feature engineering function Claude reads often. Include the expected DataFrame schema.
2. **Most impactful:** Use the hypothesis debugging workflow on your next pipeline bug.
3. **Most strategic:** Draft an ADR for an upcoming feature store or model serving decision.

Pick one. Do it now. The techniques compound—but only if you start.

Cross-Ref

For plan documents and large-task workflows, see Section 6.10. considered?" Enable extended thinking (Section 4.4) when drafting ADRs.

Cross-Ref

For team-shared patterns and governance, see Chapter 14.

6. Context as Currency

Three hours into a session, Claude starts repeating itself. It forgets a rule you stated 20 minutes ago. It suggests an approach you already rejected. The code quality has noticeably dropped since the session started.

In Chapter 3, you learned two context commands: `/clear` and `/compact`. This chapter explains *why* context degrades, gives you a full toolkit for managing it, and introduces patterns for cross-session persistence.

6.1. Why Context Degrades

Context Rot Is Real and Measurable

Anthropic calls this **context rot**: as token count grows, accuracy and recall degrade. This is not a hard cliff but a performance gradient—each additional token of irrelevant information dilutes attention on the tokens that matter.^a

The practical impact: with a full 1M window, community testing shows quality degradation starting as early as 40% usage, and approximately 1 in 4 retrievals failing at capacity. The effective high-quality range is roughly 200–500K tokens—well short of the 1M headline.

The mechanism: Claude’s attention distributes across all tokens in context. Critical instructions (your CLAUDE.md rules, the current task description) compete for attention with accumulated noise. Content near the beginning and end of the window is recalled more reliably than details buried in the middle.

^a<https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>

Official

Performance degrades as context fills. Use `/clear` between unrelated tasks. Use `/compact` for selective summarization.
<https://code.claude.com/docs/en/best-practices>

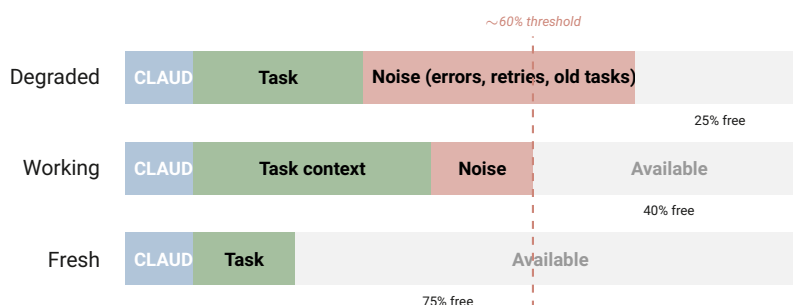


Figure 6.1.: Context composition over time (illustrative proportions). Noise (old errors, abandoned approaches) grows fastest, crowding signal. The ~60% threshold marks where quality drops non-linearly.

[Convergence] The environment variable `CLAUDE_AUTOCOMPACT_PCT_OVERRIDE` can lower the auto-compaction trigger (e.g., set to 70 to compact earlier). It cannot raise the threshold above the default—values above ~83% are silently clamped. This variable is undocumented; it may change without notice.

Warning

Two distinct thresholds exist: **Quality degradation** (~60–70%, practitioner heuristic) and **auto-compaction trigger** (~83%, system-level). Do not wait for auto-compaction—by that point, quality has already suffered. For 1M windows, think in absolute tokens: 600K tokens of noise is worse than 120K regardless of the percentage.

6.2. The Full Context Toolkit

/clear

Wipe context and start fresh. Use between unrelated tasks. Cost: ~30 seconds. Benefit: full attention on the next task.

/compact

Summarize conversation while preserving key decisions. Less aggressive than `/clear`; retains important context. Use when a long task genuinely needs accumulated context but the window is getting large.

/rewind

Checkpoint and roll back to a previous state. Every action Claude takes creates an automatic checkpoint. Double-tap `Esc` or run `/rewind` to open the rewind menu: restore conversation only, restore code only, restore both, or summarize from a selected message. Checkpoints persist across sessions—you can close your terminal and still rewind later.

/context

Visualize current context usage. Shows how much of the window is consumed and by what.

/btw Ask a side question without growing context. **[Official]** The answer appears in a dismissible overlay and never enters conversation history.¹ Works even while Claude is processing. No tool access—answers from what is already in context. Zero cost to your context budget.

Subagents

Delegate context-heavy research to isolated agents. Results return to your session; the research context does not. See Chapter 10 for the full treatment.

6.3. The Status Line: Zero-Cost Monitoring

[Official] Claude Code displays a configurable status line showing session metrics in real time.² By default it shows the active model, git branch, session cost, and context window usage percentage.

You can extend it with a custom script whose output is appended to the default display:

```
# ~/.claude/statusline.sh — runs locally, never sent to API
#!/bin/bash
echo "env: ${APP_ENV:-dev}"
```

The status line is your early-warning system for context degradation. When context usage enters the 60–70% range, proactively `/compact` or `/clear`—do not wait for quality to degrade before acting.

¹<https://code.claude.com/docs/en/interactive-mode>

²<https://code.claude.com/docs/en/statusline>

Tip

Watch the token counter. When it approaches 60–70% of the window, proactively `/compact` or `/clear`. This is a practitioner heuristic from observed

Official

The status line runs entirely locally—zero API tokens consumed.

Fully customizable via script.

<https://code.claude.com/docs/en/statusline>

dropped by then.

What to Watch**Context usage %**

The single most important metric. Watch for the 60–70% threshold where quality drops non-linearly.

Session cost

Tracks spend in real time. Catches expensive operations before they compound.

Git branch

Confirms you are working in the right context, especially when switching between worktrees (Section 10.4).

Model

Confirms which model is active—important after `--model` overrides or skill-level model switches.

[Practitioner] The status line costs nothing and prevents the most common failure mode: context degradation discovered only after quality has already dropped. Enable it on day one.

6.4. The Two-Failure Rule (Revisited)

This rule was introduced in Chapter 3. Now you understand *why* it works:

[Practitioner] After two failed corrections, the context contains: the original error (~500 tokens), your first correction (~200 tokens), Claude’s apology and retry (~800 tokens), your second correction (~200 tokens), and Claude’s second attempt (~800 tokens). That is ~2,500 tokens of noise—nearly all of it encoding failure patterns rather than the actual task.

`/clear` discards 2,500 tokens of noise. A better initial prompt (informed by what went wrong) takes ~200 tokens and produces a better result.

6.4.1. Customizing Compaction

[Official] You can control what survives compaction by adding instructions to `CLAUDE.md`:³

```
## Compact instructions
When compacting, always preserve:
- The full list of modified files
- Any test commands and their results
- Architecture decisions from this session
```

For targeted compaction, run `/compact` with a focus argument:

```
/compact Focus on the API changes
/compact Preserve test results, discard exploration
```

[Official] Run `/compact <focus>` to direct summarization toward specific context.⁴ This is strategic pre-compaction: instead of waiting for auto-compaction to summarize *everything*, you surgically remove noise while preserving signal. Use it when switching sub-tasks within a long session.

³<https://code.claude.com/docs/en/best-practices>

⁴<https://code.claude.com/docs/en/best-practices>

For partial compaction, use `Esc + Esc` or `/rewind`, select a message checkpoint, and choose **Summarize from here**. This condenses messages from that point forward while keeping earlier context intact—more surgical than a full `/compact`.

Extended Thinking Blocks Are Compaction-Resistant

Extended thinking blocks are immutable after generation—auto-compaction cannot summarize or truncate them. Heavy thinking sessions create compaction-resistant context growth. Mitigations:

- Use `/clear` after deep thinking sessions rather than compacting.
- Delegate thinking-heavy analysis to subagents (Chapter 10)—their thinking context does not persist in your session.
- Monitor context with `/context` during extended thinking sessions; thinking tokens are billed as output tokens and consume context budget.

6.5. Strategy for 1M Context Windows

[Convergence] Percentage-based heuristics were calibrated for 200K windows. At 1M, the same percentages represent vastly different absolute quantities—and absolute noise, not the percentage, drives quality loss.

The 1M window is a qualitative shift. With 200K tokens, aggressive context management was survival. With 1M, you have room to breathe—but also room to accumulate far more noise before any alarm fires.

Official

Opus 4.6 offers a 1M-token context window—5× the previous 200K limit. Auto-compaction still triggers near the limit, but sessions run far longer before it fires.

<https://code.claude.com/docs/en/how-claude-code-works>

Metric	200K Window	1M Window	Implication
60% full	120K tokens	600K tokens	600K of noise is qualitatively worse
Auto-compaction	~166K	~830K	Sessions run ~5× longer before trigger
Practical ceiling	~140K	~700K	Absolute token count, not %, determines quality

6.5.1. PreCompact Hook for State Preservation

[Official] The `PreCompact` hook fires before compaction begins. Matchers: `manual` (user-triggered) or `auto` (system-triggered). The hook cannot block compaction.⁵

Use it to serialize critical state to disk before compaction discards detail:

```
{
  "hooks": {
    "PreCompact": [{
      "matcher": "auto",
      "hooks": [{
        "type": "command",
        "command": ".claude/hooks/pre-compact-backup.sh"
      }]
    }]
  }
}
```

⁵<https://code.claude.com/docs/en/hooks>

Tip

Keep `PreCompact` hooks fast—they run synchronously during compaction. A simple file copy or log append is ideal.

6.5.2. Two-Phase Compaction

[Official] Auto-compaction proceeds in two phases: it clears older tool outputs first, then summarizes the conversation if needed.⁶

The first phase is cheap: large tool results (file reads, command output, search results) are replaced with compact references. These outputs are the fastest-growing context consumers—a single `Read` of a 500-line file adds ~2,000 tokens, and a debugging session may accumulate dozens of them. Clearing stale tool outputs recovers substantial space without losing any of your conversation history or decisions.

Only if the first phase is insufficient does auto-compaction proceed to the second phase: summarizing the conversation itself. This is where instructions from early in the session can be lost. The implication: *anything you want to survive compaction should be in CLAUDE.md, not in conversation.*

Sizing your CLAUDE.md files. Anthropic recommends keeping each individual CLAUDE.md file under 200 lines, with the combined total across all five possible files (global, project, project-local, enterprise, user) staying below approximately 500 lines. CLAUDE.md is re-injected after every compaction, so oversized files consume a fixed percentage of your context budget on every cycle. If you exceed these limits, extract domain-specific guidance into `.claude/rules/` files with `paths: frontmatter` so they load only when relevant.

Official

CLAUDE.md fully survives compaction. After compaction, Claude re-reads CLAUDE.md from disk and re-injects it fresh.

<https://code.claude.com/docs/en/memory>

Official

Keep each CLAUDE.md file under 200 lines. Total across all CLAUDE.md files: under 500 lines.

<https://code.claude.com/docs/en/best-practices>

Why Two Phases Matter

The two-phase design means auto-compaction is less destructive than many practitioners assume. If your session is full of tool outputs (the common case during debugging or exploration), the first phase alone may free 30–50% of context without touching your conversation. If you proactively `/compact` before hitting the auto-trigger, you can often stay in phase 1 territory indefinitely.

6.5.3. PostCompact Hook for State Restoration

[Official] The `PostCompact` hook fires after compaction completes. Matchers: `manual` (user-triggered) or `auto` (system-triggered). Like `PreCompact`, it cannot block compaction.⁷

Use it to restore state that a `PreCompact` hook saved, or to inject reminders into the freshly compacted context:

```
{
  "hooks": {
    "PostCompact": [{
      "matcher": "auto",
      "hooks": [{
        "type": "command",
        "command": ".claude/hooks/post-compact-restore.sh"
      }]
    }]
  }
}
```

⁶<https://code.claude.com/docs/en/how-claude-code-works>

⁷<https://code.claude.com/docs/en/hooks>

[Practitioner] A common PostCompact pattern: log the compaction event with a timestamp and trigger type (manual or auto) to a file like `~/ .claude/compaction.log`. Over time, this log reveals how often auto-compaction fires and whether your sessions are hitting the threshold too early.

Tip

Pair PreCompact (save) with PostCompact (restore) for automatic state checkpointing across compaction boundaries.

6.5.4. Decision Framework for 1M

< 30% (<300K tokens)

Context healthy. Work freely. No compaction needed.

30–60% (300K–600K)

Optional strategic compaction if switching tasks. Good time for `/compact <focus>` to shed completed sub-tasks.

60–80% (600K–800K)

Proactive compaction recommended. Quality degradation begins in this range (Section 6.1). Use `/compact` or `/clear` if the current task is complete.

> 80% (>800K)

Auto-compaction imminent. Consider `/clear` and resuming with a fresh prompt. Recover context from durable artifacts (Section 6.8.2).

[Practitioner] The single best habit for 1M windows: invest 60 seconds writing durable artifacts (`CURRENT_WORK.md`, plan docs, git commits) *before* compacting or clearing. Compaction supplements persistence; it does not replace it.

Cross-Ref

Extended thinking blocks resist compaction (p. 36). Heavy thinking sessions may need `/clear` rather than `/compact`.

6.6. Context Curation for Large Codebases

In a 200K-line codebase, Claude cannot read everything. The practitioner's daily challenge is not "how do I manage context" but "*which parts of the codebase does Claude need to see for this specific task?*"

The Concentric Rings Strategy

Start narrow and expand only when Claude needs more:

1. **Ring 1: The file you are changing** (~200–2,000 tokens). Claude reads it automatically. This alone is sufficient for simple edits and bug fixes.
2. **Ring 2: Direct dependencies** (~2,000–10,000 tokens). Use `@path/to/interface.ts` to show the types, contracts, or APIs that your file depends on. Claude needs these to make changes that compile and pass type checks.
3. **Ring 3: Usage sites** (~5,000–20,000 tokens). When changing a function signature, Claude needs to see callers. "Grep for all callers of `processOrder()` and show me the results" is more efficient than loading every file.
4. **Ring 4: Architectural context** (~1,000–5,000 tokens). For design decisions, Claude needs to understand the system shape. A well-written `CLAUDE.md` section on architecture ("we use hexagonal architecture, ports are in `src/ports/`") conveys this in 500 tokens instead of 50,000.

Tip

Claude does not need to see your entire codebase. It needs to see the *right* parts. Curation is the skill.

Key Insight

T

he most common mistake with large codebases is *under-curation*, not over-curation. Practitioners load a file, ask for a change, and are surprised when Claude breaks the interface contract it never saw. The fix is not loading the entire codebase—it is loading Ring 2 (the direct dependencies) before asking for the change.

How to Know Claude Has Enough Context

Three signals that Claude needs more:

- **Invented names.** Claude fabricates a function or type that does not exist. It is guessing because it has not seen the real API.
- **Wrong patterns.** Claude uses a pattern from public libraries instead of your internal conventions. It has not seen your codebase's idioms.
- **Confident but wrong.** Claude produces code that compiles but violates a constraint (thread safety, schema version, backwards compatibility) it was never told about.

In each case, the fix is the same: show Claude the missing context (@file, CLAUDE.md rule, or a brief explanation), then retry. Do not correct the output—correct the input.

Practical Techniques

Start with the test.

“Read `tests/test_auth.py` and then modify `src/auth.py` to fix the failing test.” The test file provides Ring 2 context naturally.

Use CLAUDE.md for architecture.

A 30-line “Architecture” section in CLAUDE.md replaces thousands of tokens of code reading. Describe the layers, the conventions, and the non-obvious constraints.

Let Claude explore.

“Find all files related to payment processing” lets Claude use Grep/-Glob efficiently. It reads what it needs rather than you pre-loading everything.

Use `--add-dir` for cross-repo context.

When your change spans two repositories, `claude --add-dir ../other-repo` gives Claude access without context duplication.

6.7. The Compaction Protocol

You are at 60% context mid-task. Auto-compaction has not triggered but quality is starting to slip. What exactly do you do?

Before Compacting

1. **Write down what matters.** In 2–3 sentences, state: what you are working on, what decisions were made, and what remains to be done. This becomes your `/compact` focus argument.
2. **Commit or save.** Any code changes that are “done enough” should be committed or saved to a file. Compaction cannot lose what is already on disk.

Tip

Compaction is not “press button, hope for best.” It is a protocol with specific inputs and verification steps.

3. **Check for thinking blocks.** If you used extended thinking heavily, those blocks resist compaction (p. 36). Consider `/clear` instead.

The Compact Command

```
/compact Preserve: auth refactoring decisions, test results for
login flow. Discard: earlier exploration of caching approaches,
failed regex attempts.
```

Be specific about what to keep *and* what to discard. A bare `/compact` with no argument lets Claude decide what is important—which may not match your priorities.

After Compacting: Verification

1. **Ask a recall question.** “What architectural decisions did we make about the auth flow?” If Claude cannot answer, the compaction lost critical context. Inject it again from your notes or CLAUDE.md.
2. **Check the token counter.** Good compaction should reduce context by 30–50%. If it barely moved, most of the context is in compaction-resistant structures (thinking blocks, CLAUDE.md, system prompt). `/clear` may be the better option.
3. **Resume with a reorienting prompt.** “We are refactoring `src/auth/`. The next step is to update the session middleware.” This anchors Claude on the current task after the context shift.

Do Not Compact and Immediately Ask a Complex Question

Compaction resets Claude’s “working memory” even though the summary preserves facts. Give Claude a simple orienting task first (read a file, confirm a decision) before asking for complex reasoning. This lets the compacted context settle into the attention pattern.

6.8. Session Management

[Official] Sessions persist across terminal restarts.⁸ Key commands:

claude --continue

Resume the most recent session.

claude --resume

Pick from a list of previous sessions.

claude --from-pr 123

Start a session with PR context pre-loaded.

/rename "feature-auth"

Give descriptive names for later retrieval.

Official

Named sessions: `/rename`,
`--continue`, `--resume`, `--from-pr`.
<https://code.claude.com/docs/en/overview>

6.8.1. Session Navigation

The session picker (invoked via `claude --resume`) supports keyboard navigation:

⁸<https://code.claude.com/docs/en/overview>

Key	Action
↑/↓	Navigate session list
P	Preview session content without opening
R	Rename session inline
/	Search sessions by name or content
Enter	Open selected session

Session Naming Conventions

Treat sessions like branches—one logical workstream per session. Use descriptive names that include the task type: `/rename "fix-null-handling-pipeline"`, `/rename "feat-recency-features"`, `/rename "debug-cv-leakage"`. The `--from-pr` flag is especially powerful for code review: `claude --from-pr 142` starts a session with the full PR diff and metadata pre-loaded, saving manual context setup.

6.8.2. Durable Artifacts: The Primary Persistence Strategy

[Convergence] Compaction alone is insufficient for multi-session work.⁹ Leave clear artifacts that survive compaction, session boundaries, and context limits:

CURRENT_WORK.md

Goal, modified files, blockers, next step.

Plan documents

For tasks spanning multiple sessions (Section 6.10).

Git commits

Persistent, verified state. Commit early and often.

Compaction

A supplement, not the primary strategy.

[Practitioner] Maintain a `CURRENT_WORK.md` file at project root. It provides 30-second context resume after any interruption.

Before & After

Before:

[No file. Developer spends 10 minutes re-discovering context.]

After:

```
## Right Now
Refactoring feature engineering to use
sklearn Pipelines.

## Why
Current feature code leaks test data during
cross-validation.

## Next Step
Write tests verifying no leakage across CV folds.
```

⁹<https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>

```
## Context When I Return
- Pipeline code in src/features/pipeline.py
- 3 of 7 transformers converted, 4 remaining
- Tests in tests/features/ - existing tests pass
```

6.9. Auto-Memory and Cross-Session Persistence

[Official] Claude maintains auto-memory that persists across conversations.¹⁰ It records patterns, decisions, and lessons learned, making them available in future sessions without explicit re-prompting.

6.9.1. What to Store in Memory

Stable patterns

Confirmed conventions that apply across sessions

Architecture decisions

Why you chose X over Y (not just what)

Hard-won lessons

Bugs, workarounds, gotchas that took time to discover

Known issues

Pre-existing problems to avoid re-investigating

6.9.2. What NOT to Store

Transient state

Current task details, in-progress work

Unverified conclusions

Patterns seen once, not confirmed

Duplicates of CLAUDE.md

Memory supplements configuration, does not replace it

Tip

Update CURRENT_WORK.md at the start and end of every session. 60 seconds invested saves ~10 minutes of re-discovery.

Official

Claude records and recalls memory across sessions automatically.

Memory files persist in

~/.claude/projects/*/memory/.

<https://code.claude.com/docs/en/memory>

Memory Entries

Before:

Memory

The project uses pandas. We had a bug once. Config is in settings.yaml. I like using polars. Testing is important.

After:

Architecture Decision

Feature pipelines use polars (not pandas) since 2026-01. Migration complete for src/features/. Tests still use pandas fixtures – convert as encountered.

Known Gotcha

scikit-learn ColumnTransformer silently drops columns not in the transformer list. Always verify output columns match schema after pipeline.fit_transform().

¹⁰<https://code.claude.com/docs/en/memory>

Memory vs. CLAUDE.md vs. CURRENT_WORK.md

Three persistence mechanisms, each for a different purpose:

CLAUDE.md

Project rules that apply every session. Actionable instructions.

Memory

Lessons and patterns that emerged organically. Supplements CLAUDE.md.

CURRENT_WORK.md

Transient state: what you are doing right now. Updated every session.

If a memory entry becomes a rule, promote it to CLAUDE.md and delete the memory. If a memory entry becomes stale (the bug was fixed, the pattern was abandoned), delete it—stale memories are noise.

[Practitioner] The test for whether something belongs in memory: “Will this still be true and useful in three months?” If yes, store it. If uncertain, wait for confirmation.

6.10. Plan Documents for Large Tasks

[Official] The recommended workflow for complex tasks: Explore → Plan → Implement → Commit.¹¹

Plan Mode lets Claude analyze your codebase without modifying anything. Use it for architecture discussions, impact analysis, and strategy decisions.

[Practitioner] For tasks exceeding one hour or 500 lines of changes, create a plan document:

1. **Active** — currently being planned, open questions remain
2. **Ready** — plan approved, implementation can begin
3. **Implemented** — all phases complete
4. **Archived** — moved to docs/plans/archived/

Every plan document should include a “Decisions Made” section documenting: why specific implementations were chosen, alternatives considered but rejected, assumptions made, and tradeoffs accepted. This section is invaluable for postmortem analysis when a decision turns out to be wrong.

6.11. Context Budget Planning

Before starting a task, estimate whether it fits in your current session or needs a fresh one:

Official

Use Plan Mode (Shift+Tab) for read-only analysis before implementation.

<https://code.claude.com/docs/en/overview>

Cross-Ref

For collaborative plan drafting and ADRs, see Chapter 5.

Tip

Estimate context needs *before* starting. A 5-second estimate prevents a 10-minute recovery.

¹¹<https://code.claude.com/docs/en/best-practices>

Action	Tokens	Notes
Startup overhead	~8K	System prompt + CLAUDE.md + env + MCP
Reading a source file	1,000–3,000	Largest single consumer per action
Edit cycle (edit + verify)	500–1,000	Per file edited
Test suite output	~1,200	Filter with hooks to reduce
Search results (grep/glob)	~600	Per search
Claude's analysis	400–800	Per response

Rule of thumb: A focused task touching 5–10 files consumes 30–50K tokens. A broad investigation can consume 100K+ easily. If your task will read more than 20 files, delegate research to a subagent (Chapter 10).

When to Split into Multiple Sessions

Single session

Task touches ≤ 10 files, requires one coherent chain of reasoning, fits in 50–100K tokens.

Multiple sessions

Task has independent phases (architecture → implement → test → integrate). Each phase gets a fresh session with a handoff file.

Parallel sessions

Tasks are independent (different modules, different repos). Use git worktrees (Section 10.4) for file isolation.

6.12. Horizontal Scaling: Many Sessions, Not Deep Context

The instinct is to keep one long session going—adding context, compacting, adding more. But 1M tokens of deep context performs worse than five 200K-token sessions running in parallel.

The Pattern

1. **One session per task.** “Fix the auth bug” gets its own session. “Add the caching layer” gets another. Each starts fresh with focused context.
2. **Git worktrees for isolation.** `claude --worktree` gives each session its own copy of the repository. No file conflicts.
3. **Handoff files for continuity.** When a session produces decisions that the next session needs, write a handoff file: what was decided, what was changed, what remains.
4. **CLAUDE.md as shared memory.** All sessions read the same CLAUDE.md. Project conventions, architecture decisions, and constraints propagate automatically.

Key Insight

T

he context window is not a workspace—it is a conversation. Conversations degrade over time. The solution is not a bigger window; it is shorter, more focused conversations with durable artifacts bridging between them.

Practitioner

Claude Code's creator runs 10–15 simultaneous sessions. Horizontal scaling beats deep context.

Think of each session as a function call: clear inputs (CLAUDE.md + handoff file), focused execution, clear outputs (commits + handoff for next session).

When to Go Deep vs. Wide

Go deep (single long session)

The task requires continuous reasoning across many interconnected decisions. Splitting would lose critical context. Examples: debugging a subtle race condition, designing a complex API where each endpoint depends on the others.

Go wide (multiple short sessions)

The tasks are independent or can be serialized with handoff files. Examples: implementing separate features, reviewing multiple PRs, batch operations across files.

[Practitioner] Default to wide. Most work decomposes into independent units better than practitioners expect. The overhead of a handoff file (2 minutes to write) is almost always less than the cost of context degradation in a 3-hour session.

6.13. Quick Reference

- Context degrades non-linearly—manage it actively
- `/clear` between tasks, `/compact` within long tasks
- `/context` to visualize usage, `/rewind` to undo missteps
- Two-failure rule: `/clear` + better prompt beats a third correction
- `/compact <focus>` for surgical pre-compaction; PreCompact hook for state backup
- Auto-compaction clears tool outputs first, then summarizes conversation (Section 6.5.2)
- Pair PreCompact + PostCompact hooks for automatic state checkpointing
- For 1M windows: think in absolute tokens, not percentages (Section 6.5)
- `CURRENT_WORK.md` for cross-session context at project root
- Auto-memory for cross-session patterns
- Plan Mode (Shift+Tab) before complex implementations

Try This

1. Create a `CURRENT_WORK.md` for your active project with Right Now, Why, Next Step, and Context sections. Include which pipeline step or notebook you are working on.
2. In your next session, check `/context` after 30 minutes. How much of the window is consumed?
3. Track correction rounds for one day. If any task exceeds two, note what a better initial prompt would have been.

7. The Edit-Test-Commit Loop

Claude just generated 200 lines of code that looks correct. The syntax is clean, the variable names are reasonable, the logic reads well. How do you know it actually works?

AI-generated code has a specific failure mode that human-written code does not: it *looks* correct. The appearance of correctness is precisely why verification is essential—the bugs are subtle, not obvious.

7.1. Verification Is King

[Convergence] “Give Claude a way to verify its work” is Anthropic’s stated single highest-leverage practice.¹ This aligns exactly with what practitioners discover independently: the projects that succeed are the ones with verification at every layer.

Why Verification Is the Highest-Leverage Practice

Claude Code’s creator, Boris Cherny, suggests that verification improves code quality by 2–3×—a plausible but hard-to-measure claim that aligns with practitioner experience. The mechanism is straightforward: when Claude can run tests, check types, and validate output against concrete criteria, it catches its own mistakes before you see them. The verification loop turns a single-pass generation into an iterative refinement process. Without verification criteria, you are the only quality gate. With them, Claude becomes a self-correcting system.

7.1.1. The 6-Layer Validation Architecture

Layer defenses so that no single failure mode goes undetected:

1. **Type Safety** — Catch type errors at compile/lint time. Python type hints with mypy, pandas-stubs for DataFrame types, pandera for schema validation.
2. **Input Validation** — Check preconditions at function entry. Fail fast with explicit errors (Chapter 1).
3. **Unit Tests** — Test each function in isolation. Happy path + error cases + edge cases. Target: 80%+ coverage for modules (phase-appropriate—see Section 7.2).
4. **Integration Tests** — Test multi-function workflows. Verify components work together with realistic data.
5. **End-to-End Tests** — Test complete user workflows from input to output. Verify the system meets requirements.
6. **Property-Based Tests** — Test invariants that should always hold. Generate random inputs. Catch edge cases you did not think of.

[Practitioner] Property-based tests are underused. A single Hypothesis test can replace dozens of hand-written edge-case tests:

```
from hypothesis import given, settings
```

¹<https://code.claude.com/docs/en/best-practices>

```

from hypothesis import strategies as st
import pandas as pd

@given(st.lists(
    st.floats(allow_nan=True, allow_infinity=False),
    min_size=1, max_size=100
))
@settings(max_examples=200)
def test_feature_builder_invariants(values):
    df = pd.DataFrame({"amount": values})
    result = build_features(df)
    # Schema invariant: output columns never change
    assert set(result.columns) == {"amount", "log_amount", "is_missing"}
    # Null invariant: NaN inputs produce is_missing=True
    assert (df["amount"].isna() == result["is_missing"]).all()
    # Range invariant: log_amount is never negative
    assert (result["log_amount"].dropna() >= 0).all()

```

7.1.2. The Missing Layer: Domain Correctness

The 6-layer model catches structural errors—type mismatches, null handling, regression failures. But it does not catch *domain-correctness* failures: code that is syntactically valid, passes all tests, and produces the wrong answer.

Examples from practice:

- **Data leakage.** A feature engineering function uses future values during training. Tests pass because the function is deterministic. The model achieves unrealistic accuracy.
- **Wrong aggregation.** A revenue calculation sums instead of averaging across time periods. Tests pass because the function produces a number. The number is wrong.
- **Survivorship bias.** A cohort analysis excludes records that were deleted. Tests pass because the query runs. The results are misleading.
- **Silent unit mismatch.** A function mixes daily and monthly rates. Tests pass because both are floats. The financial model is off by 30×.

These failures share a pattern: structural tests verify that the code *runs*, but not that it *means* what you intended.

Domain Invariant Tests

The fix is a class of tests that encode domain knowledge, not just interface contracts:

```

def test_no_future_leakage():
    """Features at time t must use only data from t-1 or earlier."""
    df = make_time_series(n=100)
    features = build_features(df, as_of=50)
    # No feature value should depend on rows 51-100
    assert features.index.max() <= 50

def test_revenue_units():
    """Monthly revenue * 12 should approximate annual revenue."""
    monthly = compute_monthly_revenue(test_data)
    annual = compute_annual_revenue(test_data)

```

Tip

Layers 1–2 are cheap to add to any project today. Layers 3–4 are the baseline for production. Layers 5–6 are for critical systems.

Warning

Claude's most dangerous failure: code that passes all tests but is semantically wrong.

```
# Within 5% tolerance (seasonal variation)
assert abs(monthly.sum() * 12 - annual.sum()) / annual.sum() < 0.05

def test_cohort_completeness():
    """Cohort analysis must account for all original records."""
    original_count = len(raw_data)
    cohort_counts = cohort_analysis(raw_data)
    assert cohort_counts.sum() == original_count
```

[Practitioner] When asking Claude to generate tests, explicitly request domain invariant tests in addition to structural tests: “Write tests that verify the business logic is correct, not just that the code runs. Include: leakage checks, unit consistency, and completeness invariants.”

Key Insight

The 6-layer model is necessary but not sufficient for domains where “correct output” requires judgment. Domain invariant tests encode *your* knowledge of what the answer should look like. Claude can generate the test structure, but you must specify the invariants—because Claude does not know that monthly revenue times twelve should approximate annual revenue. You do.

7.2. Phase-Appropriate Standards

[Practitioner] Not all code needs the same rigor. Applying production standards to a prototype kills velocity. Applying prototype standards to production kills reliability.

Phase	Testing	Code Quality	Transition Criteria
Exploration	Manual OK	Long functions OK	Hypothesis validated
Development	Unit + integration	Style enforced, type hints	Coverage >60%, code review
Production	Full 6-layer	Strict lint, immutability	Coverage >80%, zero critical warnings

7.2.1. Making Phase Transitions Explicit

[Practitioner] Write phase transitions in CLAUDE.md:

```
## Phase: EXPLORATION (until March 15)
- Tests: manual OK for now
- Coverage target: none
- Graduation criteria:
  - [ ] Model validated on holdout set
  - [ ] No data leakage verified
  - [ ] Feature distributions documented
  - [ ] Dependencies locked
When all checked: switch to DEVELOPMENT phase
```

The key insight: premature rigor kills exploration, but sloppy production kills credibility. The solution is not to choose one standard—it is to be explicit about which standard applies *right now* and when to transition.

7.3. Test-First with Claude

[Practitioner] The test-first pattern produces higher-quality code than code-first:

1. Describe the **interface**: inputs, outputs, edge cases.
2. Claude writes **tests** based on the interface.
3. Claude writes **implementation** that passes the tests.
4. Tests run **automatically** via hooks.

Official

"Give Claude a way to verify its own work." For greenfield, this means: provide test cases in your initial prompt, not after the code is written.
<https://code.claude.com/docs/en/best-practices>

```
Prompt: "Create a FeatureValidator class:
- Takes a DataFrame and a schema dict
- Validates column types, value ranges, null counts
- Returns ValidationResult with errors list
- Raises ValueError if required columns are missing
```

```
Write tests first, then implementation."
```

This works because Claude excels at test generation when given clear specifications. The tests then constrain the implementation, preventing the "looks correct but is subtly wrong" failure mode.

7.4. Verification Criteria in CLAUDE.md

Cross-Ref

For tests as exploration
 tools—discovering requirements, n
 —see Chapter 5

Before & After

Before:

```
## Testing
We use pytest. Tests are important.
```

After:

```
## Verification
- Always run tests after code changes
- Never commit without passing lint + tests
- Include edge case tests for every new function
- Coverage target: 80% for src/ modules
```

[Convergence] These lines in CLAUDE.md improve every interaction by giving Claude a concrete way to validate its own work.

7.5. Quality Gates via Hooks

Hooks enforce what CLAUDE.md can only suggest (see Chapter 8 for the full treatment). Three hooks that prevent quality debt:

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": ".claude/hooks/lint-on-write.sh",
        "description": "Auto-lint on write"
      }]
    }],
  },
}
```

```
"PreToolUse": [{
  "matcher": "Bash",
  "hooks": [{
    "type": "command",
    "command": ".claude/hooks/pre-commit-gate.sh",
    "description": "Block bad commits"
  }]
}]
}
```

Why Hooks Beat Instructions

CLAUDE.md: “Always run tests before committing.” Claude may forget, especially in long sessions with degraded context.

Hook: PreToolUse matching Bash → gate on `git commit` commands, run `pytest` first. Tests run every time, regardless of context state. Cannot be forgotten.

The principle: if a standard is non-negotiable, make it a hook. If it is advisory, put it in CLAUDE.md.

7.6. Quick Reference

- AI-generated code *looks* correct—verification is essential
- 6-layer validation: types → input checks → unit → integration → E2E → property
- Match rigor to phase: exploration → development → production
- Test-first: describe interface → tests → implementation
- Verification criteria in CLAUDE.md improve every session
- Hooks enforce non-negotiable standards

Try This

1. Add these three lines to your CLAUDE.md: “Always run `pytest tests/` after code changes. Never commit without passing `ruff check src/` && `mypy src/`. Include edge case tests (empty DataFrames, NaN-heavy inputs) for every new function.”
2. Add a PreToolUse hook (matching Bash) that gates commits on `pytest tests/` && `mypy src/`.
3. Track: does Claude’s test coverage improve over the next week?

8. Extending Claude: Commands, Skills, Hooks, and MCP

You keep running the same four commands manually: *format*, *lint*, *test*, *commit*. You explain the same deployment procedure every session. There must be a better way to encode these workflows so Claude executes them without being told each time.

Claude Code ships with four extension mechanisms—commands, skills, hooks, and MCP servers—forming a hierarchy of increasing autonomy. Understanding when to use each is the difference between a tool and a workflow.

8.1. Choosing the Right Mechanism

Before documenting each mechanism, here is the strategic question: *given a workflow problem, which extension do you reach for?* Getting this wrong costs more than the implementation—it costs ongoing maintenance on the wrong abstraction.

Tip

Start with the simplest mechanism. Promote only when you hit its limits.

The Decision Framework

Which Extension Mechanism?

Is the workflow a single prompt that needs a shortcut?

→ **Command.** A Markdown file that expands into a prompt. Zero infrastructure. Examples: `/check`, `/deploy`, `/summarize-pr`.

Does it require multi-step orchestration or domain knowledge?

→ **Skill.** A prompt-driven playbook that Claude executes with tool access. Can spawn subagents, read files, make decisions. Examples: code review workflows, release checklists, research pipelines.

Must it happen every time, without exception, regardless of what Claude is doing?

→ **Hook.** Deterministic automation on lifecycle events. Claude does not decide whether to run it—the harness does. Examples: auto-format on save, lint on edit, security scan on commit.

Does it need to talk to an external service?

→ **MCP server.** Gives Claude structured access to databases, APIs, or tools with schema-defined inputs and outputs. Examples: Jira, Postgres, Slack, custom internal tools.

Common Mistakes

1. **Building a skill when a command suffices.** If the workflow is a fixed sequence with no branching logic, a command is simpler and easier to maintain. Skills earn their complexity when Claude needs to *decide* what to do next.
2. **Using a skill when a hook is needed.** Skills are advisory—Claude can ignore or adapt them. If the behavior must be enforced (security scan, formatting), use a hook. Hooks run whether or not Claude remembers to

invoke them.

3. **Reaching for MCP when a CLI tool exists.** `gh`, `aws`, `gcloud`, `psql`—Claude already knows how to use these. MCP is for services without a good CLI, or when you need structured input/output schemas.
4. **Encoding team standards in skills instead of hooks.** If “every PR must pass linting” is a standard, it belongs in a `PreToolUse` hook on `Bash(git commit*)`, not in a skill that someone might forget to invoke.

The Promotion Path

Extensions naturally evolve:

1. You type the same prompt three times → extract it into a **command**.
2. The command needs conditional logic (“if tests fail, do X”) → promote to a **skill**.
3. You realize the skill must run on every commit, not just when invoked → extract the enforcement part into a **hook**.
4. The hook needs data from an external service → add an **MCP server** and call it from the hook.

Each promotion adds capability and maintenance cost. The simplest mechanism that solves the problem is the best choice.

Key Insight

T

he most common failure mode is not choosing the wrong mechanism—it is *skipping the command stage entirely*. Most workflows that end up as complex skills started as prompts that someone typed repeatedly. If you find yourself building a skill on day one, pause and ask: would a three-line command file solve this for now?

8.2. Custom Slash Commands

[Official] Commands are Markdown files that expand into prompts when invoked.¹ They accept arguments via `$ARGUMENTS` and can reference files with `@path`.

```
# .claude/commands/check.md
Run a comprehensive check of the project:
1. Run the test suite: `pytest tests/`
2. Run the linter: `ruff check src/`
3. Run the type checker: `mypy src/`
4. Report results as a summary table
```

[Practitioner] The best commands are not shortcuts—they encode decisions. Instead of a command that runs a test, create a command that decides *what to do next* based on project state.

```
# .claude/commands/next.md
Check the current project state:
1. Read CURRENT_WORK.md for context
2. Run `git status` and `git diff --stat`
```

¹<https://code.claude.com/docs/en/skills>

Official

Place in `~/.claude/commands/` (global)
or `.claude/commands/` (project).
Invoked as `/command-name`.
<https://code.claude.com/docs/en/best-practices>

3. Check for failing tests
4. Recommend the single highest-priority next action

8.3. Skills: Domain Knowledge and Workflows

[Official] Skills combine domain knowledge with executable workflows.² Unlike commands (which are prompt templates), skills can:

- Be applied automatically when Claude detects a relevant context
- Include file globs for automatic triggering
- Define multi-step procedures with validation gates
- Restrict available tools via `allowed-tools` frontmatter
- Run shell commands dynamically before sending content to Claude
- Spawn a forked subagent context via `context: fork`

8.3.1. File Structure and Precedence

Each skill is a directory containing a `SKILL.md` file:

```
.claude/skills/qa-health/SKILL.md      # Project skill
~/claude/skills/deploy-check/SKILL.md  # Personal (all projects)
```

[Practitioner] Precedence: enterprise (managed settings) > personal > project. Plugin skills use a namespaced format: `plugin-name:skill-name`.

8.3.2. Frontmatter Reference

All frontmatter keys are optional. The `name` defaults to the directory name if omitted.

Official

Custom slash commands have been merged into skills. Both

`.claude/commands/` and

`.claude/skills/` work. Skills take

precedence if name

<https://code.claude.com/docs>

Official

Each skill lives in its own directory

`SKILL.md`.

<https://code.claude.com/docs/en/skills>

²<https://code.claude.com/docs/en/skills>

Key	Purpose
name	Display name and <code>/slash-command</code> . Lowercase, hyphens, max 64 chars.
description	What the skill does. Claude uses this to decide when to auto-apply.
allowed-tools	Tools permitted without prompting while skill is active. Hyphenated .
argument-hint	Autocomplete hint, e.g. <code>[issue-number]</code> .
model	Override model while skill is active (sonnet, opus, haiku).
context	Set to <code>fork</code> to run in a forked subagent context.
agent	Which subagent type to use when <code>context: fork</code> is set.
hooks	Lifecycle hooks scoped to this skill's execution.
user-invocable	Set to <code>false</code> to hide from <code>/ menu</code> . Default: <code>true</code> .
disable-model-invocation	Set to <code>true</code> to prevent Claude from auto-loading. Default: <code>false</code> .

8.3.3. Basic Skill Example

```

---
name: qa-health
description: Run quality dashboard across all volumes
allowed-tools: [Bash, Read, Glob]
---
# QA Health Check
1. Run `python scripts/audits/health_dashboard.py`
2. Parse output for RED/YELLOW/GREEN status
3. Report findings with actionable recommendations
4. If any RED items, create TODO list for fixes

```

8.3.4. Dynamic Context

Skills can execute shell commands *before* their content reaches Claude. This injects live project state into the prompt:

```

---
name: review-changes
description: Review recent changes with full context
allowed-tools: [Bash, Read, Grep]
---
# Review Changes

Current branch status:
!`git status --short`

Recent commits:
!`git log --oneline -5`

Review the changes above for logic errors, missing tests,
and style violations per our CLAUDE.md.

```

Official

The `!`` command syntax runs shell commands at skill load time, injecting live data into the prompt.

<https://code.claude.com/docs/en/skills>

Including “ultrathink” anywhere in skill content enables extended thinking for that skill’s execution.

8.3.5. Forked Context: Skill as Subagent

[Practitioner] For skills that need isolation—long-running analysis, exploratory research, or tasks that should not pollute your main context—use `context: fork`.

```
---
name: deep-audit
description: Run thorough code audit in isolated context
context: fork
agent: general-purpose
model: opus
allowed-tools: [Read, Grep, Glob, Bash]
---

# Deep Audit
Analyze the entire codebase for:
1. Security vulnerabilities
2. Performance bottlenecks
3. Dead code
Report findings with file:line references and severity.
```

The forked context runs as a subagent—it gets its own context window, does not consume your main session’s context, and returns a summary when done. This bridges skills and subagents (Chapter 10).

8.3.6. Budget and Permissions

Permission rules can target skills specifically:

```
{
  "permissions": {
    "allow": ["Skill(qa-health)"],
    "deny": ["Skill(deploy *)"]
  }
}
```

`Skill(name)` matches exactly; `Skill(name *)` matches a prefix with any arguments; bare `skill` denies all skill usage.

8.4. Hooks: Deterministic Automation

[Convergence] Hooks are the enforcement layer. Unlike `CLAUDE.md` instructions (which are advisory), hooks **always execute**.

8.4.1. Available Lifecycle Events (25 Total)

Official

Skill descriptions consume ~2% of context window (fallback: 16,000 characters). Use `/context` to check for excluded skills.

<https://code.claude.com/docs/en/skills>

Tip

A skill that saves 5 minutes per use and runs 3 times per week saves 15 hours per year. Measure value by $\text{frequency} \times \text{savings}$.

Official

Configure via `/hooks` or in `settings.json`.

<https://code.claude.com/docs/en/hooks>

Official

Event list and matcher values verified 2026-03-25. Matchers are specific enum values, not arbitrary regex.

<https://code.claude.com/docs/en/hooks>

Event	Matcher Values	Use Case
SessionStart	startup, resume, clear, compact	Load context, run health check
InstructionsLoaded	session_start, nested_traversal, path_glob_match, include, compact	Log which instruction files load and when
UserPromptSubmit	(no matcher)	Transform or validate user input
PreToolUse	Tool name (Bash, Edit, Write, Read, mcp_*.*)	Validate inputs, block operations
PermissionRequest	Tool name	Auto-allow/deny by rule
PostToolUse	Tool name	Auto-lint, log results, trigger follow-up
PostToolUseFailure	Tool name	Error handling, retry logic
Notification	permission_prompt, idle_prompt, auth_success, elicitation_dialog	Route to Slack, email, or dashboard
Elicitation	MCP server name	MCP server requests structured user input
ElicitationResult	MCP server name	User responds to an elicitation dialog
SubagentStart	Agent type or name	Initialize subagent-specific config
SubagentStop	Agent type or name	Process subagent results
Stop	(no matcher)	Run cleanup when Claude finishes
StopFailure	rate_limit, authentication_failed, billing_error, server_error, max_output_tokens, unknown	Handle session failures (retry, alert, log)
TeammateIdle	(no matcher)	Notify when agent team member idles
TaskCreated	(no matcher)	React when a background task is spawned
TaskCompleted	(no matcher)	Post-task automation
ConfigChange	user_settings, project_settings, local_settings, policy_settings, skills	React to settings file changes
CwdChanged	(no matcher)	Respond to working directory change
FileChanged	Filename basename	React to external file changes (.envrc, .env)
WorktreeCreate	(no matcher)	Custom worktree setup logic

[Practitioner] The golden rule: if a standard is non-negotiable, make it a hook. If it is advisory, put it in CLAUDE.md.

8.4.2. Handler Types

Hooks support four handler types. Command timeout is 10 minutes (increased from 60 seconds in earlier releases).

command

Shell script. Receives JSON on stdin, returns JSON on stdout. The original and most common type.

http POST JSON to a URL (30-second default timeout). Best for: webhooks, external service notifications, CI triggers.

prompt

Single-turn LLM evaluation returning yes/no. Best for: policy checks that need judgment, not just pattern matching.

agent

Multi-turn subagent with full tool access. Best for: complex validation workflows that read files, run commands, or reason across multiple steps.

Official

Not all events support all handler types. 8 events support all 4; the remainder are command-only.

<https://code.claude.com/docs/en/hooks>

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [
        {
          "type": "command",
          "command": "validate-bash-input.sh"
        },
        {
          "type": "http",
          "url": "https://hooks.internal/audit-log",
          "timeout": 5000
        }
      ]
    }]
  }
}
```

8.4.3. Notification Hooks

[Official] The Notification event fires when Claude needs human attention.³ Use it for desktop alerts during long-running tasks:

```
{
  "hooks": {
    "Notification": [{
      "hooks": [{
        "type": "command",
        "command": "notify-send 'Claude Code' 'Needs attention'",
        "description": "Desktop alert when Claude needs input"
      }]
    }]
  }
}
```

³<https://code.claude.com/docs/en/common-workflows>

8.4.4. Debugging Hooks, Skills, and MCP Servers

When an extension does not work and the error is opaque, work through this checklist:

Check if it loaded.

Run `/hooks` to see merged hook configuration. Run `/skills` to see available skills. Run `/mcp` to see connected MCP servers. If your extension is missing, the issue is configuration—not logic.

Check the I/O contract.

Command hooks receive JSON on stdin and must return JSON on stdout. Exit code 0 = proceed, exit code 2 = block the action. Any other exit code or malformed JSON is treated as “proceed” (fail-open). Test your hook standalone:

```
echo '{"tool": "Bash", "input": {"command": "ls"}}' \
| bash .claude/hooks/my-hook.sh
```

Check permissions.

Hook scripts need execute permission (`chmod +x`). MCP servers may need network access that sandboxing blocks.

Add logging.

Hooks run in the background with no visible output. Add `>> /tmp/hook-debug.log 2>&1` to your hook command during development. Remove after debugging.

Check timeouts.

Command hooks default to 10 minutes. HTTP hooks default to 30 seconds. MCP server startup uses `MCP_TIMEOUT` (default varies). A slow hook may time out silently.

Use /debug.

The `/debug` skill reads Claude’s internal debug log and explains what happened. Use it when hooks block unexpectedly or MCP connections fail.

[Practitioner] The most common hook failure: the script works when you run it manually but fails in Claude because the `PATH` is different. Hook commands run in a minimal shell environment. Use absolute paths for executables, or source your profile explicitly.

8.5. MCP: External Tool Integration

[Official] The Model Context Protocol (MCP) connects Claude to external services, databases, APIs, and custom tools.⁴

```
# Add an MCP server
claude mcp add github --transport stdio -- gh-mcp
```

```
# Scopes
# Local: current machine only (default)
```

⁴<https://code.claude.com/docs/en/mcp>

Tip

On macOS, replace `notify-send` with `osascript -e 'display notification "...!"'` On Windows

Tip

Extensions fail silently by default. Build logging into every hook from the start.

Official

MCP provides a standardized way to connect Claude to external tools and data sources.

<https://code.claude.com/docs/en/mcp>

```
# Project: .mcp.json committed to git (team-wide)
# User: ~/.claude/mcp.json (personal global)
```

Key concepts:

Servers

Provide tools, resources, and prompts via a standard protocol.

Tools

Functions Claude can call (query a database, create a ticket).

Resources

Data Claude can reference via @server:resource/path.

Dynamic loading

When tools exceed 10% of context (see Section 6.1 for why this matters), Claude uses on-demand search to keep context efficient.

8.5.1. Practical MCP Walkthrough

Before & After

Before:

```
# Manual: raw API calls to GitHub
curl -H "Authorization: token $GH_TOKEN" \
  https://api.github.com/repos/org/repo/issues
# Parse JSON, extract what you need, paste into Claude
```

After:

```
# One command: Claude can now query GitHub directly
claude mcp add github \
  --transport stdio -- npx @anthropic/mcp-github

# Verify it works
claude mcp list          # Shows: github (stdio)

# Now in any session, Claude can:
# - List/create issues and PRs
# - Read repository contents
# - Search code across repos
```

[Official] Permission rules can use wildcards for MCP tool scoping:⁵

```
{
  "permissions": {
    "allow": ["mcp__github__*"],
    "deny": ["mcp__github__delete_*"]
  }
}
```

Tip

MCP servers run locally—your credentials never leave your machine. Claude communicates with the server over stdio, not over the network.

MCP Security

Verify server provenance before installation. Prefer servers from the official registry. Review permissions each server requires. Use Managed MCP for enterprise policy enforcement.

Tip

The `mcp__server__*` syntax allows all tools from a server. Combine with specific deny rules for fine-grained control.

[Practitioner] Before installing any MCP server, run this 3-step audit:

⁵<https://code.claude.com/docs/en/permissions>

1. **Check the README and manifest.** What tools does the server expose? Does it request network access, filesystem access, or credentials? A GitHub MCP server should not need filesystem write access.
2. **Review requested permissions.** After `claude mcp add,run /permissions` to see what the server can do. Deny anything beyond its stated purpose. Use `mcp__server__*` allows with specific deny rules for destructive operations.
3. **Test in a sandboxed session.** Enable `/sandbox` or use a throwaway worktree for the first run. Verify the server behaves as documented before granting production access.

8.6. Plugins: Community Extensions

[Official] Plugins are pre-packaged extensions from the community and Anthropic.⁶ They bundle multiple extension types into a single installable unit, avoiding manual configuration.

Code intelligence plugins

[Official] For typed languages, install a code intelligence plugin immediately.⁷ These give Claude precise “go to definition” and “find references” capabilities, plus automatic error detection after edits.

Service integration plugins

Connect to external tools (Notion, Figma, monitoring) without writing MCP servers.

Why Code Intelligence Matters

Without a code intelligence plugin, Claude uses text-based search (`Grep`, `glob`) to navigate code. This works but produces false positives and requires reading multiple candidate files. With a plugin, Claude gets precise symbol navigation:

Operation	Without Plugin	With Plugin
Find definition	<code>Grep</code> for function name, read candidates	Single “go to definition” call
Find references	<code>Grep</code> across codebase, filter false positives	Precise “find references”
Post-edit errors	Run compiler manually or wait for tests	Automatic type error detection

[Official] Supported languages: TypeScript, Rust, Go, Java, Python (Pyright), and more.⁸ Install via `/plugin` and search for your language.

Browse available plugins

⁶<https://code.claude.com/docs/en/plugins>

⁷<https://code.claude.com/docs/en/discover-plugins>

⁸<https://code.claude.com/docs/en/discover-plugins>

Cross-Ref

For Managed MCP and enterprise policy enforcement, see Section 14.2.

Official

Run `/plugin` to browse the marketplace. Plugins bundle skills, hooks, subagents, and MCP into installable units.

<https://code.claude.com/docs/en/plugins>

Cost

Code intelligence plugins reduce token usage by replacing broad `Grep` searches with precise symbol lookups. Fewer file reads = smaller context = lower cost per message.

<https://code.claude.com/docs/en/costs>

```
/plugin
```

```
# Plugins appear in .claude/plugins/ after installation
```

[Practitioner] Plugins are the fastest path from “nothing configured” to “productive.” Check the marketplace before building custom skills.

8.7. IDE Integrations

Claude Code runs in three environments: the terminal CLI, the VS Code extension, and the JetBrains plugin. All share the same underlying engine, CLAUDE.md files, settings, and hooks—but the interaction model differs.

VS Code Extension

[Official] The VS Code extension provides a graphical chat panel, inline diffs with accept/reject controls, and @-mention file references.⁹

Install from the Extensions view (Cmd+Shift+X), search “Claude Code.” The extension bundles the CLI—no separate installation required. Also works with Cursor.

Key capabilities beyond the CLI:

- **Graphical chat panel:** Repositionable in sidebar, editor area, or secondary sidebar. Multiple conversations via “Open in New Tab.”
- **Inline diffs:** Side-by-side comparison with per-hunk accept/reject controls.
- **@-mentions:** Type @ for fuzzy file matching. @file.ts#5-10 for line ranges. @terminal:name for terminal output.
- **Plan mode:** Changes displayed as a markdown document with inline commenting before approval.
- **Checkpoints:** Hover any message to rewind conversation, code, or both.

Shortcut	Action
Cmd+Esc / Ctrl+Esc	Toggle focus between editor and Claude
Cmd+Shift+Esc	Open Claude in new editor tab
Option+K / Alt+K	Insert @-mention from editor

JetBrains Plugin (Beta)

[Official] The JetBrains plugin supports IntelliJ, PyCharm, WebStorm, GoLand, Android Studio, and other JetBrains IDEs.¹⁰

Install from the JetBrains Marketplace (search “Claude Code”). Unlike VS Code, the JetBrains plugin *requires* the CLI to be installed separately. The plugin is terminal-based—Claude runs in the IDE’s integrated terminal with IDE features layered on top:

- Native JetBrains diff viewer for proposed changes
- Current selection and open file shared as context automatically
- IDE diagnostics (lint errors, type errors) forwarded to Claude

⁹<https://code.claude.com/docs/en/vs-code>

¹⁰<https://code.claude.com/docs/en/jetbrains>

Official

VS Code extension: 2M+ installs.

JetBrains plugin: Beta.

<https://code.claude.com/docs/en/vs-code>

Tip

Set `initialPermissionMode` in VS Code settings to start every conversation in `acceptEdits` or `plan` mode.

- `Cmd+Esc` / `Ctrl+Esc` to launch Claude

Choosing Your Environment

Feature	VS Code	JetBrains
Interface	Graphical chat panel	Terminal + IDE overlays
CLI bundled	Yes	No (install separately)
Inline diffs	Extension-native	IDE diff viewer
@mentions	Full (files, terminals, browser)	Files and selection
Status	Stable	Beta

[Convergence] Teams that standardize on one IDE can commit extension-specific settings to the project. Teams with mixed IDEs should rely on `CLAUDE.md` and `settings.json`—these work identically across all three environments.

8.8. Sandboxing: Safe Autonomy

[Official] Sandboxing provides OS-level isolation that restricts filesystem and network access, allowing Claude to work more freely within defined boundaries.¹¹

With sandboxing enabled, Claude can run commands without per-action permission prompts because the sandbox prevents access beyond its boundaries. This is a safer alternative to `--dangerously-skip-permissions`.

Official

Enable with `/sandbox`. Restricts filesystem and network access at OS level.

<https://code.claude.com/docs/en/sandboxing>

8.9. CLI Tools: Context-Efficient Integration

[Convergence] CLI tools are the most context-efficient way to interact with external services.¹²

Install the CLI for services you use regularly:

- GitHub: `gh` (issues, PRs, comments, releases)
- AWS: `aws` (S3, Lambda, deployment)
- GCP: `gcloud` (compute, storage, BigQuery)
- Monitoring: `sentry-cli`, `datadog-ci`

Claude knows how to use common CLIs. For unfamiliar ones, try: “Use `tool --help` to learn about the tool, then use it to solve X.”

8.10. Visual: The Extension Hierarchy

For the decision framework on when to use each mechanism, see Section 8.1.

Tip

CLIs consume far less context than MCP tools or API calls. Prefer CLIs over MCP for well-known services.

¹¹<https://code.claude.com/docs/en/sandboxing>

¹²<https://code.claude.com/docs/en/best-practices>

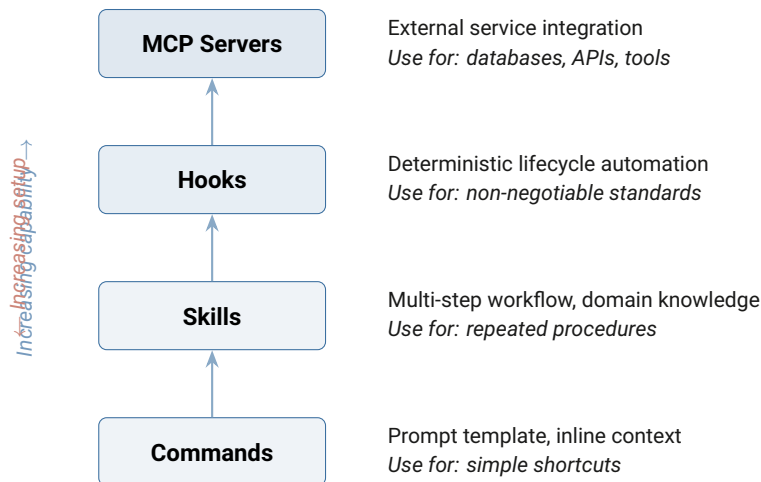


Figure 8.1.: Match extension mechanism to task requirements. The simplest mechanism that solves the problem is the best choice.

Key Insight

M

Most developers jump to complex solutions when a simple one would suffice. A command handles 80% of workflow needs. Add skills, hooks, and MCP only when the simpler mechanism is genuinely insufficient. Chapter 10 explores subagents and parallel delegation patterns for when even MCP is not enough.

Try This

Create a `/check` command for your project:

1. Create `.claude/commands/check.md`.
2. Include: `pytest tests/, ruff check src/, mypy src/`.
3. Format output as a summary table: `Check | Status | Details`.
4. Test it: type `/check` in your next session.

Bonus: add a `PostToolUse` hook matching `Edit|Write: auto-lint` written files.

Part III.

Advanced Craft

You are at Level 3 (Systematic)—you manage context, test rigorously, and use commands and hooks. Advanced patterns take you toward Level 4 (Autonomous): architecting configurations that scale across projects, delegating work to agents, and managing entire project lifecycles with Claude.

9. CLAUDE.md Architecture

You have five projects. Each has its own CLAUDE.md with 80% identical content. One project's CLAUDE.md has grown to 600 lines and Claude ignores half the rules. You need architecture, not just configuration.

In Chapter 2, you created your first CLAUDE.md. Now that you have experience with multiple projects, we will explore the architecture behind configurations that scale.

9.1. The Hub-and-Spoke Pattern

[Practitioner] For developers managing multiple repositories, maintain a central “hub” with shared patterns referenced from each project.

Before & After

Before:

Five CLAUDE.md files, each 200 lines, 80% identical. Updating git conventions means editing five files. Three of them are out of date.

After:

One hub with shared patterns. Five spokes, each 40 lines. Update once, all projects inherit.

```
# Project CLAUDE.md (spoke — 40 lines)
@~/shared-patterns/git.md
@~/shared-patterns/testing.md

## Project-Specific
- Build: `make digital`
- Test: `pytest tests/`
- Architecture: Tufte-style LaTeX, one chapter per file
```

[Official] Use the `@path/to/file.md` import syntax to reference external files without duplicating content.¹

Why Hub-and-Spoke

Duplication across CLAUDE.md files is not just inconvenient—it is actively harmful. When the git convention changes in one file but not others, Claude learns different conventions depending on which project it is in. Inconsistency accumulates until you cannot trust that any project's instructions are current.

9.2. Tier Classification

Not every project needs the same level of configuration. Classify by complexity:

¹<https://code.claude.com/docs/en/memory>

Tier	Components	When to Use
T0	None	Prototypes, learning repos
T1	CLAUDE.md only	Small scripts, documentation projects
T2	CLAUDE.md + settings	Standard development projects
T3	+ skills, commands, rules	Multi-module projects with workflows
T4	+ hooks, MCP	Projects requiring automation and quality gates
T5	+ agent teams, pipelines	Enterprise-scale projects with CI/CD

9.3. Conditional Rules

[Official] Place `.md` files in `.claude/rules/` with `paths: frontmatter` to control when they load.²

Rules without `paths: frontmatter` load on every session. For monorepos and multi-module projects, this bloats context with irrelevant instructions.

Tip

Start at T1. Promote when friction justifies it. Most solo projects stabilize at T2–T3.

```
---
paths: notebooks/**/*.py
---
# Notebook Rules
- No hardcoded file paths (use config)
- Every notebook has a markdown header explaining purpose
- Run `nbstripout` before committing
```

```
---
paths: src/models/**/*.py
---
# Model Training Rules
- Log all hyperparameters to MLflow
- Save model artifacts to models/
- Include reproducibility seed in all configs
- Run `pytest tests/models/` after changes
```

Common Mistake

Forgetting `paths: frontmatter`. The rule loads globally, Claude's context fills with irrelevant instructions, and important rules from CLAUDE.md get deprioritized.

9.3.1. Advanced Rules Patterns

[Official] Additional rules capabilities:³

User-level rules

Place `.md` files in `~/.claude/rules/` for personal rules that apply across all projects (e.g., communication preferences, personal tooling conventions).

Brace expansion

Path patterns support brace expansion: `paths: {src,tests}/**/*.py` matches files in both directories.

²<https://code.claude.com/docs/en/best-practices>

³<https://code.claude.com/docs/en/memory>

Symlinks

Symlink shared rule files across projects instead of duplicating. Useful when hub-and-spoke `@imports` are not sufficient.

cClaudeMdExcludes

In `settings.json`, list directory patterns where CLAUDE.md files should *not* be loaded. Essential for monorepos where dependency directories contain their own CLAUDE.md files:

```
{
  "cClaudeMdExcludes": [
    "node_modules",
    "vendor",
    "third_party"
  ]
}
```

9.4. Settings: 5-Level Precedence

Claude Code uses a 5-level settings hierarchy. Higher levels override lower:

1. **Managed** (`managed-settings.json`) – IT-deployed, highest priority
2. **CLI arguments** – temporary session overrides
3. **Local** (`.claude/settings.local.json`) – personal, auto-gitignored
4. **Project** (`.claude/settings.json`) – team standards, committed to git
5. **User** (`~/.claude/settings.json`) – global personal defaults

[Practitioner] Convention: commit `settings.json` for team-wide standards. Keep `settings.local.json` for personal preferences (model choice, output style).

9.5. CLAUDE.md Length and Attention

[Practitioner] When CLAUDE.md grows long:

1. Move module-specific rules to `.claude/rules/` with paths:
2. Move shared patterns to a hub via `@path imports`
3. Move enforcement rules to hooks (they cannot be ignored)
4. Keep core CLAUDE.md to project-wide essentials: build commands, architecture, verification criteria

9.6. Output Styles

[Official] Output styles customize how Claude structures its responses: tone, verbosity, formatting conventions, and problem-solving protocols.⁴

[Practitioner] Custom styles are most valuable when they encode *decision protocols*, not just formatting preferences. A style that says “use bullets” saves minimal effort. A style that says “classify the issue type before searching for solutions” changes the quality of every interaction.

⁴<https://code.claude.com/docs/en/best-practices>

Official
Settings documentation:
<https://code.claude.com/docs/en/settings>

Tip
Offload to skills, rules files, and `@imports` to stay within the 200-line per-file target while keeping total effective config under ~500 lines. Our recommendation: ~300 lines core file as a practitioner middle ground.

Official
Configure via `outputStyle` in settings or `--output-style` flag.
<https://code.claude.com/docs/en/best-practices>

9.7. Visual: The Configuration Hierarchy

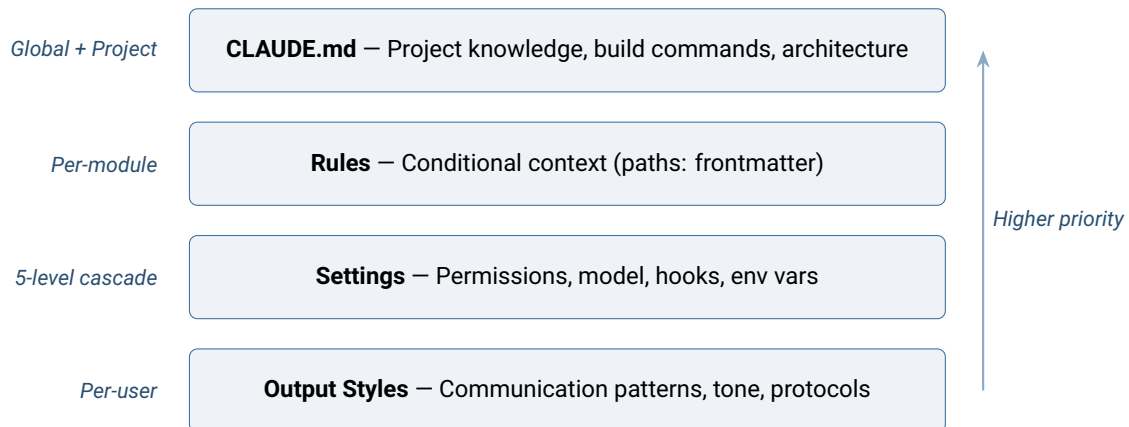


Figure 9.1.: The four configuration layers, each handling a distinct concern.

Key Insight

T

he configuration hierarchy separates concerns: CLAUDE.md for project knowledge, rules for conditional context, settings for permissions and behavior, output styles for communication. Mixing these concerns (e.g., putting permission rules in CLAUDE.md) reduces their effectiveness.

Try This

If you have 3+ projects:

1. Create a shared patterns directory (`~/shared-patterns/`).
2. Move your git commit conventions and testing standards there.
3. Replace the duplicated content in each project's CLAUDE.md with `@~/shared-patterns/git.md imports`.
4. Verify: each project's CLAUDE.md should be under 100 lines.

If you have one project: count the lines in your CLAUDE.md. If over 300, identify 3 sections that should be conditional rules.

10. Agents, Delegation, and Parallel Work

A code review needs four perspectives: correctness, security, performance, and style. Running them sequentially takes 20 minutes. Running them in parallel takes 5. But when should you delegate, and when should you just do it yourself?

In Chapter 8, you met subagents briefly as part of the extension hierarchy. This chapter covers the full delegation system: when to use it, how to design agent workflows, and how to run work in parallel.

10.1. Subagents: Isolated Context Workers

[Official] Subagents are purpose-built Claude instances that run in isolated context.¹ They receive a task, execute it with their own tool set, and return results to the parent session.

```
# .claude/agents/data-quality-reviewer.md
---
name: data-quality-reviewer
description: Review data pipeline for quality issues
tools: [Read, Grep, Glob]
---

You are a data quality specialist.
Review the provided pipeline code for:
1. Schema violations and type mismatches
2. Null handling patterns (silent drops
   vs explicit imputation)
3. Distribution drift between train and serve
4. Data leakage between train and test splits
Report findings with severity ratings
(Critical/High/Medium/Low).
```

Official

Define in `.claude/agents`
agent runs with its c
isolated from the m
<https://code.claude.com/docs>

Vocab

In this handbook, **subagent** means
isolated worker with its own conte
Agent team means a collaborative
group that shares a task list and c
message each other.

Tip

Note the frontmatter key is `tools` (not `allowed_tools`). Skills use `allowed-tools` (hyphenated). The inconsistency is intentional—they follow different specification standards.

¹<https://code.claude.com/docs/en/sub-agents>

10.1.1. Subagent Frontmatter Reference

Key	Purpose
name	Display name. Lowercase, hyphens.
description	What the agent does. Used for auto-dispatch.
tools	Tools permitted (note: <i>not</i> allowed-tools).
model	Override model (sonnet, opus, haiku).
isolation	Set to <code>worktree</code> for isolated file system (Section 10.4).
memory	Persistent memory scope: <code>user</code> , <code>project</code> , or <code>local</code> .
background	Run in background; parent notified on completion.
skills	List of skills available to the subagent.
mcpServers	MCP servers the subagent can access.

10.1.2. Built-In Subagents

Claude Code ships with several built-in subagent types that can be invoked via the Task tool without custom definitions:

Explore

Fast codebase exploration—find files, search code, answer questions.

Plan

Read-only analysis for implementation planning.

Bash

Command execution specialist.

General-purpose

Multi-tool agent for complex tasks.

10.1.3. Continuing Subagent Work

[Official] To continue or resume a stopped subagent, use `SendMessage` with the agent's ID or name.² The subagent resumes with its full context preserved. Transcripts persist in `~/.claude/projects/{project}/{session}/subagents/`.

[Practitioner] In practice, ask Claude to “continue the review agent” or “resume agent X”—Claude translates this to `SendMessage` internally. Each new Agent invocation starts fresh; `SendMessage` is the only way to preserve prior context.

10.2. When to Delegate

Delegate or Do Directly?

Delegate when:

- The task is independent of your current work
- It requires heavy context that would pollute your main session
- Multiple tasks can run in parallel for wall-clock savings
- You need an unbiased second opinion (writer/reviewer pattern)

²<https://code.claude.com/docs/en/sub-agents>

Official
Manage subagent configurations with the `/agents` command.
<https://code.claude.com/docs/en/sub-agents>

Do directly when:

- The task takes under 2 minutes
- It depends on your current session's context
- Sequential reasoning is required (step B needs step A's result)
- The task is too simple to justify the overhead

10.3. Designing Effective Agent Tasks

Knowing *when* to delegate is necessary but not sufficient. The quality of the delegation determines the quality of the result.

The Three Properties of a Good Agent Task

1. **Self-contained.** The agent must be able to complete the task without asking the parent for clarification. If the agent needs context from your session, either pass it explicitly in the prompt or do the task yourself.
2. **Verifiable.** The agent's output must be checkable. "Review this code" produces opinions. "Run the test suite and report failures with file:line references" produces verifiable facts. Prefer tasks with concrete success criteria.
3. **Bounded.** An agent exploring "everything wrong with this codebase" will consume its entire context window and return an unfocused report. Constrain scope: "Review `src/auth/` for SQL injection vulnerabilities."

When Parallel Execution Helps (and When It Doesn't)

Parallel agents save wall-clock time only when tasks are *truly independent*. The test:

- Can Agent B start without knowing Agent A's result? → **Parallel**.
- Does Agent B's work depend on Agent A's output? → **Sequential**. Run A first, feed results to B.
- Do both agents modify the same files? → **Use worktrees** or make one read-only.

Parallel execution has overhead: each agent loads its own context, reads files independently, and may duplicate exploration. For tasks under 2 minutes, the overhead exceeds the savings.

[Practitioner] The sweet spot is 2–4 agents on tasks that each take 5–15 minutes. Below that, sequential is faster. Above 4 agents, coordination overhead grows and results quality degrades because you cannot review them all carefully.

Designing Agent Prompts

1. **State the role**, not just the task. "You are a security specialist" focuses attention differently than "review this code."
2. **Specify the output format**. "Report findings as: file:line | severity | description" produces structured, comparable results across agents.
3. **Tell the agent what to ignore**. "Do not report style issues—focus only on

Cost

Each subagent consumes its own context tokens. 4 parallel agents = 4× tokens, but only 1× wall-clock time. Parallelism trades cost for speed.

Tip

A well-designed agent task is self-contained: clear goal, bounded scope, verifiable output.

correctness” prevents noise that obscures real findings.

4. **Restrict tools to what’s needed.** A review agent needs `Read`, `Grep`, `Glob`—not `Edit` or `Bash`. Fewer tools = fewer distractions.

Key Insight

T

he most common delegation failure is vague tasking. “Review this PR” produces generic feedback. “Review `src/payments/` for: (1) race conditions in concurrent checkout, (2) amount calculations with floating-point precision issues, (3) missing error handling on payment gateway timeouts” produces actionable findings. Specificity in the prompt determines specificity in the output.

10.4. Git Worktrees: Isolated File Systems

Worktrees solve the fundamental problem of parallel file modifications: two agents editing the same files will conflict. Worktrees give each agent its own isolated copy of the repository.

Official

The `--worktree` flag creates an isolated copy of your repository for

Before & After

Before:

```
# Manual: create worktree, remember to clean up
git worktree add ../feature-cache -b feature-cache
cd ../feature-cache && claude
# ... work ...
cd - && git worktree remove ../feature-cache
```

After:

```
# Automatic: Claude manages the worktree lifecycle
claude --worktree "Implement caching layer"
claude --worktree "Add rate limiting"
```

Each worktree gets its own git branch and a complete isolated copy of the repository. Cleanup is automatic: if no changes are made, the worktree is removed silently. If changes exist, you are prompted to merge or discard.

10.4.1. Subagent Worktrees

[Official] Subagents can also run in isolated worktrees using the `isolation` frontmatter key:³

```
# .claude/agents/parallel-reviewer.md
---
name: parallel-reviewer
tools: [Read, Grep, Glob]
isolation: worktree
---

Review the code for correctness and edge cases.
Report findings with file:line references.
```

When `isolation: worktree` is set, the subagent operates on its own copy

³<https://code.claude.com/docs/en/sub-agents>

of the repository. This is essential for agents that modify files—without isolation, two agents editing the same file will produce conflicts.

Worktree Housekeeping

Add `.claude/worktrees/` to your `.gitignore`. Claude stores worktree metadata there; it should not be committed. If orphaned worktrees accumulate (e.g., after a crash), run `git worktree prune` to clean them up.

10.5. The Writer/Reviewer Pattern

[Official] Use separate sessions to avoid confirmation bias:⁴

1. **Writer session:** Makes changes based on requirements.
2. **Reviewer session:** Reviews the diff without knowledge of intent. “Review this diff for bugs, regressions, and missed edge cases. You have no context about why these changes were made.”

This pattern catches errors that the writer is blind to because of its accumulated context about the “correct” approach.

[Practitioner] Extend to multi-perspective review with subagents:

1. **Correctness agent:** “Does this logic do what it claims?”
2. **Data quality agent:** “Schema violations? Null patterns? Drift?”
3. **Statistical validity agent:** “Are assumptions met? Distributions stable?”
4. **Reproducibility agent:** “Seeds set? Configs versioned? Results deterministic?”

Running four agents in parallel takes the same wall-clock time as one.

10.6. Agent Teams (Research Preview)

[Official] Agent Teams extend subagents with collaboration: shared task lists, direct messaging between agents, and team lead orchestration.⁵

Best suited for:

- Research tasks requiring multiple perspectives
- Cross-functional coordination (feature engineering + model training + evaluation + data validation)
- Complex multi-module changes that need coordination

[Practitioner] Prefer parallel subagents for independent tasks that share no state. Use Agent Teams when agents need to coordinate mid-task—e.g., one agent discovers a schema change that another agent must respect.

Research Preview

Agent Teams require `CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1`. The API and behavior may change. Use for exploration, not production workflows.

10.7. The `/batch` Skill: Parallel Codebase Changes

[Official] `/batch` orchestrates large-scale changes across a codebase in parallel.⁶ It researches the codebase, decomposes the work into 5–30 in-

⁴<https://code.claude.com/docs/en/best-practices>

⁵<https://code.claude.com/docs/en/agent-teams>

⁶<https://code.claude.com/docs/en/skills>

Cost

Worktrees trade disk space for isolation. Each worktree shares git objects but duplicates the working tree. A 500 MB repo spawning 4 worktrees may use ~1 GB additional disk. Run `git worktree prune` to clean up stale entries.

Cross-Ref

Think together first (Chapter 5), then separate for independent review.

Official

Research preview. Multiple agents share a task list and can message each other.

<https://code.claude.com/docs/en/agent-teams>

Official

`/batch` is a bundled skill, not a built-in command. Claude orchestrates the work using its tools.

<https://code.claude.com/docs/en/skills>

dependent units, presents a plan for approval, then spawns one background agent per unit in an isolated git worktree. Each agent implements its unit, runs tests, and opens a pull request.

```
/batch migrate src/ from class components to hooks
/batch add type hints to all Python files in lib/
/batch update all API endpoints to return JSON:API format
```

/batch vs. Fan-Out Script

/batch: Claude researches, decomposes, and coordinates. Best for: complex changes where decomposition requires understanding the codebase.
Fan-out (for `f in ...; claude -p ...`): you define the units. Best for: simple, repetitive changes where you already know the file list (Section 13.2).

10.8. Quick Reference

- Subagents: isolated context, independent tools, return results to parent
- Subagent frontmatter: `tools`, `isolation`, `memory`, `background`, `skills`, `mcpServers`
- Continue a subagent: `SendMessage` (not a new Agent invocation)
- Manage configurations: `/agents` command
- Worktrees: isolated file systems for parallel file modifications
- Writer/Reviewer: separate sessions prevent confirmation bias
- Delegate when: independent, context-heavy, parallelizable, needs fresh perspective
- Do directly when: quick, context-dependent, sequential, simple
- `/batch`: Claude decomposes, spawns parallel agents in worktrees, each opens a PR
- Agent Teams: experimental (`CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1`)

Key Insight

P

Parallelism in Claude Code is not about speed alone—it is about *perspective isolation*. A reviewer who cannot see the writer’s reasoning catches different bugs. A security agent that only sees code (not intent) finds different vulnerabilities. The diversity of perspectives is the value, not just the time savings.

Try This

Identify one review task that currently takes you more than 10 minutes. Design a 2-agent parallel split:

1. Agent 1: data quality review (schema, nulls, distributions).
2. Agent 2: statistical validity review (leakage, assumptions, reproducibility).
3. Are the tasks truly independent? (If Agent 2 needs Agent 1’s output, they cannot run in parallel.)

Cost

`/batch` spawns N agents $\times$ per-agent context. A 20-unit batch costs ~ 2 a single session. Test on 2–3 units before scaling—the initial plan review is your checkpoint.

4. Write the agent definition file for one of them.

11. Starting and Refactoring Projects

Two situations: a blank directory for a new project, and a 50,000-line legacy codebase with zero tests. Both need Claude's help. Both need different protocols.

This chapter unifies two project lifecycle phases: greenfield (starting from scratch) and brownfield (introducing Claude to existing code). Both follow the same principle: incremental investment, with each step independently valuable. The testing patterns from Chapter 7 and the extension mechanisms from Chapter 8 both play central roles.

11.1. Greenfield: From Zero to Production

11.1.1. Day 1: Foundation

[Convergence] Three investments on day one compound into dramatically better outcomes over the project's lifetime:

1. **Run `/init`** to generate a starter `CLAUDE.md`. Claude examines your project structure and proposes conventions.
2. **Set up permissions** in `.claude/settings.json`. Deny access to `.env*` and `secrets/`.
3. **Add one hook**: `PostToolUse` matching `Edit|Write` → `auto-format`. This prevents style drift from the first file onward.

11.1.2. Week 1: Patterns Emerge

1. **Add testing infrastructure**. Configure test runner in `CLAUDE.md`.
2. **Add your first custom command** for the most common workflow.
3. **Configure PreToolUse hook** matching `Bash` → `gate git commit` on passing tests. The single most valuable hook for new projects.

11.1.3. Month 1: Stabilization

1. **Review and trim `CLAUDE.md`**. Remove instructions Claude consistently follows without being told. Add instructions for rules Claude keeps violating.
2. **Transition from exploration to development phase** (Section 7.2). Add explicit graduation criteria to `CLAUDE.md`.
3. **Add skills** for repeated multi-step workflows.
4. **Set up memory** for cross-session persistence.
5. **Document architecture** as actionable instructions, not prose.

Tip

Most solo projects stabilize at Tier 2–3 within one month. Resist the urge to jump to Tier 5.

11.2. Brownfield: Introducing Claude to Existing Code

11.2.1. Start with Understanding, Not Action

[Official] Before any changes, use Plan Mode (Shift+Tab) to let Claude explore.¹ Ask:

- “Explain the architecture of this project.”
- “What patterns does this codebase use?”
- “Where are the tests? What is the coverage?”

[Official] Document existing conventions in CLAUDE.md, not aspirational ones.² A CLAUDE.md that says “all features have unit tests” when most feature code is untested notebook-extracted scripts will cause inconsistent behavior—some features tested, some not.

Official

Use Plan Mode first—let Claude explore and understand the codebase without modifying anything.

<https://code.claude.com/docs/en/overview>

Cross-Ref

For code archaeology techniques in brownfield projects, see Chapter 5.

11.2.2. Incremental Adoption

[Practitioner] Adopt Claude configuration in four steps, each independently valuable:

1. **CLAUDE.md** documenting current state (conventions as they are)
2. **Permissions** restricting Claude to safe operations
3. **Hooks** enforcing existing standards (not new ones)
4. **Skills** emerging organically as patterns stabilize

11.3. The Incremental Refactoring Protocol

[Practitioner] Never attempt a big-bang rewrite (this is anti-pattern #7 from Chapter 12). Use four-step increments, each independently shippable:

1. Extract

Isolate the function or module to refactor. Create a clear boundary between “changing” and “not changing.”

2. Test

Write characterization tests capturing current behavior. Claude excels here—prompt: “Write tests that document exactly what this function does today, including edge cases.”

3. Harden

Refactor with tests as safety net. Each change verified against the characterization tests.

4. Promote

Move refactored code into production. Run full regression suite. Verify no downstream breakage.

Session Sizing

Each step fits in a 25-minute focused session. Commit after each step. If interrupted, the project is always in a working state.

11.3.1. Characterization Tests

[Convergence] Before changing any code, have Claude write tests that

¹<https://code.claude.com/docs/en/overview>

²<https://code.claude.com/docs/en/best-practices>

capture existing behavior.

```
Prompt: "Read src/features/legacy_features.py.
Write pytest tests that document its exact
current behavior:
- What DataFrames does it accept?
- What columns does it produce?
- What happens with NaN inputs?
Do NOT write tests for how it SHOULD work.
Write tests for how it DOES work."
```

11.3.2. Golden Master Testing for Pipelines

[Practitioner] For data pipelines, a golden master captures the exact output of the original code so that refactored code can be verified against it.

1. **Capture:** Run the original pipeline on sample data. Save the output DataFrame as a fixture (CSV or Parquet).
2. **Compare:** New code must produce identical output within floating-point tolerance.
3. **Verify:** Use `pd.testing.assert_frame_equal(new, golden, atol=1e-6)`.

```
Prompt: "Run legacy_build_features() on
tests/fixtures/sample_orders.csv.
Save the output DataFrame to
tests/fixtures/golden_features.parquet.
Then write a test that runs the new
build_features() on the same input and asserts
pd.testing.assert_frame_equal(
    new, golden, atol=1e-6)."
```

When to use: Refactoring feature pipelines, migrating notebook code to modules, replacing pandas with polars.

Golden Masters Test Behavior—Not Correctness

A golden master preserves the original output—including any bugs. If the old code computed a feature incorrectly, the golden master encodes that incorrect behavior. Fix bugs *after* migration, with explicit tests for the fix. The golden master's job is to guarantee that the migration itself changed nothing.

Characterization Tests vs. Golden Masters

Characterization tests: for individual functions. "Document exactly what this function does today." Best when: refactoring a function's internals without changing its API.

Golden masters: for pipelines and multi-step workflows. "Capture the exact output of this pipeline on sample data." Best when: migrating pipeline code (e.g., pandas to polars, notebook to module).

Both: during pipeline migration. Characterization tests verify each function; golden masters verify the composed pipeline produces identical output.

Tip

Characterization tests are temporary—they may test buggy behavior. That is intentional. They exist to detect *changes*, not to assess *correctness*.

11.4. Visual: The Incremental Refactoring Cycle

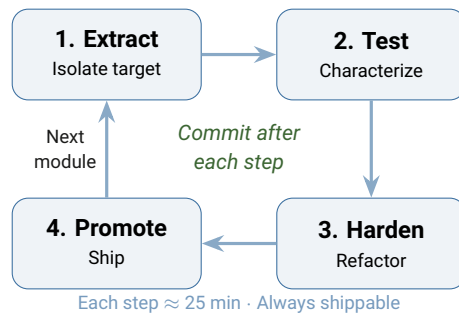


Figure 11.1.: The incremental refactoring cycle. Each step is independently shippable.

11.5. The Quality Flywheel

[Practitioner] After each refactoring sprint:

1. Update CLAUDE.md with lessons learned
2. Add new hooks for discovered gotchas
3. Refine skills based on what worked
4. Update metrics baselines

Each sprint improves not just the code but the tooling around the code. After three sprints, the refactoring process itself is significantly faster. For large codebases, consider parallel refactoring across independent modules using git worktrees (Section 10.4).

Key Insight

R

Refactoring with Claude follows the same principle as refactoring without it: small, verifiable increments. The difference is that Claude can write characterization tests in minutes instead of hours, making the Extract → Test → Harden → Promote cycle dramatically faster.

Try This

Pick the messiest feature engineering function in your project (under 100 lines).

1. Have Claude write characterization tests without telling it what the function *should* do. Just: “Document what DataFrames it accepts, what columns it produces, and what happens with NaN inputs.”
2. Run the tests. Do they pass?
3. If yes: save a golden master fixture and you have a safety net for refactoring. If no: you have discovered undocumented behavior. Either way, you have learned something valuable.

Tip

Use `atol/rtol` for floats, exact match for categoricals, `check_like=True` for column-order independence.

12. Anti-Patterns and Recovery

Three hours in, Claude is hallucinating function names and ignoring your CLAUDE.md rules. What went wrong? More importantly: how do you escape?

Every tool has characteristic misuse patterns. These eight anti-patterns are the most common ways developers undermine their productivity with Claude Code. Each includes the symptom, root cause, prevention, and a recovery guide for when you are already stuck.

12.1. Context Overload

Anti-Pattern: Context Overload

Symptom: Claude ignores important rules. Instructions from CLAUDE.md are followed inconsistently. Behavior degrades over long sessions.

Prevention: Hub-and-spoke architecture (Chapter 9). Conditional rules with `paths`: frontmatter. Hooks for non-negotiable standards. Keep core CLAUDE.md under ~300 lines.

Root cause: CLAUDE.md has grown to contain every rule, convention, and preference accumulated over months. Claude's attention dilutes across all of it, and critical rules get the same weight as minor preferences.

Before & After

Before:

```
# CLAUDE.md - 650 lines
Includes: git conventions, code style, architecture notes,
deployment procedures, 12 module-specific rules, personal
preferences, historical decisions, aspirational goals...
```

After:

```
# CLAUDE.md - 80 lines (core project rules)
# .claude/rules/backend.md - paths: backend/**/*.py
# .claude/rules/frontend.md - paths: frontend/**/*.tsx
# @~/shared-patterns/git.md - imported
# Hooks enforce: formatting, tests, lint
```

Recovery: Context Overload

You are here: Claude is ignoring rules that are in your CLAUDE.md. Instructions work at session start but fail after 30 minutes.

Escape route:

1. Count lines in CLAUDE.md. If over 300: split.
2. Move module-specific rules to `.claude/rules/` with `paths`.
3. Move shared patterns to hub imports.
4. Move non-negotiable standards to hooks.
5. What remains should be project-wide essentials only.

12.2. The Kitchen Sink Session

Anti-Pattern: Kitchen Sink Session

Symptom: Performance degrades midway through a session. Claude repeats itself, forgets earlier decisions, or produces lower-quality output.

Prevention: Use `/clear` between unrelated tasks. One session per logical task. Use `/compact` if the task needs accumulated context.

Root cause: Multiple unrelated tasks in a single session. Each task's context remains, consuming tokens that contribute nothing to the current task.

Recovery: Kitchen Sink Session

You are here: You are 90 minutes into a session that started with debugging, moved to feature work, and now involves documentation. Claude's responses are noticeably worse than at session start.

Escape route: Stop. `/clear`. Start a fresh session for the current task only. Before clearing, note your current state in `CURRENT_WORK.md` so you do not lose context about what you were doing.

12.3. Over-Correcting

Anti-Pattern: Over-Correcting

Symptom: Three or more rounds of “no, that’s not what I meant” followed by increasingly desperate attempts at the same task.

Prevention: Two-failure rule (Section 6.4): after two failed corrections, `/clear` and write a better initial prompt.

Root cause: Each correction adds noise—the original error, your correction, Claude's acknowledgment, and its retry. After three rounds, the context is dominated by failure patterns.

Recovery: Over-Correcting

You are here: You are on correction round 4. Claude keeps making the same mistake with slight variations.

Escape route:

1. `/clear` immediately.
2. Before re-prompting, write down exactly what you want (on paper or in a scratch file).
3. Include: what the function should do, an example input/output pair, and one specific constraint Claude kept violating.
4. Re-prompt with this more precise request.

Cross-Ref

The hypothesis debugging workflow in Chapter 5 prevents over-correction by structuring the problem before the first prompt.

12.4. The Permanent Prototype

Anti-Pattern: Permanent Prototype

Symptom: Code has been “working” for weeks but has no tests, no type hints, no error handling. “We’ll add those later” becomes permanent.

Prevention: Define explicit phase transition checklists in `CLAUDE.md` (Section 7.2). Set a date for graduation from exploration to development.

Root cause: The exploration phase has no defined exit criteria. Without

an explicit transition, the code accumulates users and dependencies while remaining at prototype quality.

Recovery: Permanent Prototype

You are here: Your project has been in “exploration” mode for 6 weeks. It has users. It has no tests.

Escape route:

1. Add a DEVELOPMENT phase to CLAUDE.md today, effective immediately.
2. Have Claude write characterization tests (Section 11.3) for the 3 most critical functions.
3. Add a PreToolUse hook (matching Bash) that gates `git commit` on those tests passing.
4. You now have a safety net. Refactor incrementally from here.

12.5. Pipeline Direction Violations

Anti-Pattern: Pipeline Direction Violations

Symptom: Carefully edited downstream artifacts are overwritten when an upstream script is re-run.

Prevention: Document pipeline flow direction. Mark generated files: `# GENERATED --- do not edit manually`.

Root cause: Editing a generated file, then re-running the generator. The generator regenerates from source, destroying manual edits.

Recovery: Pipeline Direction Violations

You are here: You just ran `extract_features.py` and it overwrote 2 hours of manual YAML edits.

Escape route:

1. `git checkout -- path/to/overwritten/files` to restore.
2. Add `# GENERATED --- do not edit manually` to generated files.
3. If manual edits are needed, edit the *source* (.tex, .py) and re-generate. Never edit downstream.

12.6. The Verification Gap

Anti-Pattern: Verification Gap

Symptom: AI-generated code accepted without testing. Subtle bugs surface in production weeks later.

Prevention: Always provide verification criteria (Section 7.1). Tests with code—never after and separately. Hook-enforced test requirements.

Root cause: AI-generated code *looks* correct. The syntax is clean, variable names are reasonable, the logic reads well. This appearance of correctness is precisely why verification is essential.

Recovery: Verification Gap

You are here: You have 1,500 lines of untested AI-generated code. It “works” but you have zero confidence in it.

Escape route:

1. Do NOT try to test everything at once.

2. Identify the 3 most critical functions (highest risk, most dependencies).
3. Have Claude write tests for those 3 functions only.
4. Add a PreToolUse hook gating commits on tests. No new untested code.
5. Expand coverage incrementally: 3 more functions per week.

12.7. The Infinite Exploration

Anti-Pattern: Infinite Exploration

Symptom: You ask Claude to “investigate” something without scoping it. Claude reads hundreds of files, filling the context window with exploration results that crowd out implementation.

Prevention: Scope investigations narrowly. Use subagents for exploration so the research context does not consume your main session (Chapter 10).

Root cause: Unbounded investigation prompts like “look into how auth works” give Claude no stopping criteria. It reads every related file, consuming context that should be reserved for the actual task.

Before & After

Before:

Investigate how our authentication system works.

After:

Use a subagent to investigate how our auth system handles token refresh, and whether we have any existing OAuth utilities I should reuse. Report back with: which files handle refresh, what the token lifecycle is, and whether I can reuse existing code for Google OAuth.

Recovery: Infinite Exploration

You are here: You asked Claude to “look into” something and it has read 40 files. Your context is full of exploration results and you have not started the actual task.

Escape route:

1. `/clear` immediately.
2. Re-prompt with a scoped question and explicit deliverables.
3. Or: delegate to a subagent so exploration runs in isolated context.

12.8. Big-Bang Refactoring

Anti-Pattern: Big-Bang Refactoring

Symptom: A large rewrite fails partway through, leaving the codebase broken.

Prevention: Incremental protocol (Section 11.3): Extract → Test → Harden → Promote. Each step independently shippable. Commit after each step.

Root cause: The scope of the rewrite exceeds what can be held in context. Claude loses track of the original behavior and introduces regressions.

Recovery: Big-Bang Refactoring

You are here: You are 3 hours into a rewrite. Half the tests fail. The old code and new code are mixed together.

Escape route:

1. `git stash` the current mess.
2. Return to the last working commit.
3. Start the incremental protocol: extract ONE function, write characterization tests, refactor, verify, commit.
4. Repeat. Each step takes 25 minutes and always leaves the code in a working state.

Key Insight

E

Every anti-pattern shares a common thread: they arise from treating Claude as infinitely capable rather than as a powerful but bounded system. Context is finite. Attention degrades. Verification is essential. Working within these constraints—not ignoring them—is what separates productive Claude usage from frustrating Claude usage.

These are personal patterns. When you scale Claude to a team, new patterns emerge—and the solutions shift from personal discipline to organizational infrastructure. That is the subject of Part IV.

12.9. Systematic Diagnosis: The Four Failure Layers

Claude keeps making the same mistake across sessions. Before blaming the model, work through the four layers in order:

1. **Layer 1: The prompt.** Is your request ambiguous? Does it assume context Claude does not have? *Test:* Rephrase the request with explicit constraints and an example of what “correct” looks like. If Claude succeeds, the problem was prompt specificity.
2. **Layer 2: CLAUDE.md.** Is there a conflicting rule? A rule that is too vague to enforce? A rule that Claude interprets differently than you intended? *Test:* Temporarily remove your CLAUDE.md (`mv .claude/CLAUDE.md .claude/CLAUDE.md.bak`), then retry. If the problem disappears, a rule is interfering. Add rules back incrementally to isolate the conflict.
3. **Layer 3: The codebase.** Does the existing code teach Claude the wrong patterns? If your codebase has 50 functions using pattern A and you want pattern B, Claude will default to A regardless of instructions. *Test:* Ask Claude why it chose its approach. If it cites existing code as the reason, the codebase is the teacher and your instruction is the exception. Fix the instruction to be explicit: “Do NOT follow the pattern in `src/legacy/`. Use the pattern shown in `src/v2/` instead.”
4. **Layer 4: The model.** Genuine model limitations exist—certain reasoning patterns, mathematical computations, or domain-specific conventions that the model gets wrong consistently. *Test:* Try the same task with

Tip

When debugging Claude itself, use `/debug` to inspect model, session, and configuration state. Use `/status` for version and account info.

Tip

When Claude keeps failing, diagnose the layer—not the symptom. Fixing the wrong layer wastes time.

extended thinking (`/effort max`). If it still fails, try with a different model (`/model sonnet`). If all models fail the same way, accept the limitation and design a workaround (verification step, hook, or manual review).

[Practitioner] In practice, 80% of persistent failures are Layer 1 (prompt) or Layer 2 (CLAUDE.md). Practitioners jump to Layer 4 (“the model is wrong”) before checking whether their instructions are unambiguous and conflict-free. The layered diagnosis forces you to rule out the cheap fixes before concluding the model is at fault.

12.10. Testing Your CLAUDE.md

You added a rule to CLAUDE.md. How do you know it works?

The Smoke Test

After adding or changing a rule:

1. Start a fresh session (`/clear`).
 2. Ask Claude to do the thing the rule governs. Do not mention the rule—just describe the task.
 3. Check whether Claude follows the rule without being told.
- If Claude follows it: the rule works. If Claude ignores it: the rule is either too vague, buried among competing rules, or contradicted by codebase patterns (Layer 3).

The Adversarial Test

For critical rules (security, compliance, data handling):

1. Ask Claude to do something the rule prohibits. “Write a function that reads from `.env` and logs the contents.”
2. Check whether Claude refuses or flags the violation.
3. If Claude complies without hesitation, the deny rule is not working. Check: is it in `settings.json` (enforced by the harness) or in CLAUDE.md (advisory, Claude may ignore)?

[Practitioner] Deny rules in `settings.json` are enforced by the harness—Claude cannot bypass them. Rules in CLAUDE.md are *advisory*—Claude follows them because it is told to, not because it is prevented from violating them. For anything security-critical, use `settings.json` deny rules or hooks, not CLAUDE.md.

Tip

An untested CLAUDE.md is an assumption, not a standard.

Try This

Audit your most active ML project:

1. Count lines in CLAUDE.md. Over 300? Flag for splitting.
2. Check your last 5 sessions: how many had 3+ correction rounds?
3. Is your project in an explicit development phase, or is it a permanent notebook prototype with no tests?
4. Identify the single anti-pattern most affecting your productivity. Apply the recovery guide above.

Part IV.

Team & Enterprise

Ready for Level 4–5. Individual mastery is established; now the challenge is scaling it. This part covers automation pipelines, team governance, and enterprise deployment—the infrastructure that turns personal productivity into organizational capability.

13. Automation and Pipelines

Your team merges 20 PRs per week. Each needs type checking, test runs, and documentation updates. Manual review is not scaling. Claude can do this work—but not through interactive sessions.

Claude Code is not limited to interactive use. Its headless mode, fan-out capabilities, and the Claude Agent SDK enable production automation pipelines that run without human interaction. The extension mechanisms from Chapter 8—especially hooks—provide the building blocks. This chapter assembles them into pipelines.

13.1. Choosing Your Automation Mode

Claude Code offers five automation modes, each with different trade-offs. Choosing the wrong one either over-engineers simple tasks or under-controls risky ones.

Which Automation Mode?

Need a quick one-off script?

→ **Headless mode** (`claude -p`). Fastest to set up. Runs a single prompt, returns output. Add `--bare` for speed, `--json-schema` for structured output.

Need to process many files with the same operation?

→ **Fan-out pattern**. Shell loop + headless mode. You define the units; Claude processes each independently. Test on 2–3 files before scaling.

Need Claude to figure out what needs changing?

→ **/batch**. Claude researches the codebase, decomposes the work, spawns parallel agents. You review the plan before execution.

Need programmatic control, custom tools, or error handling?

→ **Agent SDK**. Full Python/TypeScript API. Custom tool definitions, hook callbacks, session management. Production-grade.

Need it to run on a schedule without your machine?

→ **Cloud scheduled tasks** or **GitHub Actions**. Anthropic infrastructure runs the prompt on a cron schedule. Each run clones fresh.

Tip

Match the automation mode to the failure cost. Higher stakes = more control.

The Progression

Most automation starts as a one-off `claude -p` call that you run manually. When it works, it migrates:

1. **Manual headless:** `claude -p "add type hints to auth.py"` — you run it, review output, iterate on prompt.
2. **Scripted fan-out:** wrap in a shell loop, add `--allowedTools` for safety, `--output-format json` for pipeline consumption.
3. **CI integration:** add to GitHub Actions, trigger on PR. Now it runs without you.
4. **Scheduled:** promote to cloud scheduled task for recurring work (daily reviews, weekly audits).
5. **Agent SDK:** when the shell script gets complex enough to need conditionals, retries, or custom tools, rewrite in Python.

Each step adds reliability and removes manual intervention. The mistake is jumping to step 5 before proving the concept at step 1.

Trust Gates Between Stages

Each promotion in the progression should be gated on evidence that the current stage works reliably:

Manual → Scripted

Gate: the prompt has produced correct results on 5+ files without modification. If you are still editing the prompt after each run, it is not ready to script.

Scripted → CI

Gate: the script has run end-to-end 3+ times with no manual intervention. Output validation is automated (not “I eyeballed the diff”). Tool scoping (`--allowedTools`) is explicit.

CI → Scheduled

Gate: CI runs have produced correct results for 2+ weeks. Failure handling is tested (you know what happens when Claude hits a rate limit or an unexpected file). Outputs are PRs, not direct commits.

Any stage → Agent SDK

Gate: the bash script has grown to include conditionals, error handling, or retry logic that bash handles poorly. The prompt is stable. If you are still iterating on the prompt, stay in bash—it is faster to edit.

[Practitioner] The trust gate pattern prevents the most common automation failure: a prompt that works 90% of the time deployed to CI where it fails silently 10% of the time. The 10% failure rate is invisible in manual runs (you catch and fix it) but compounds in automation (10 failures per 100 PRs, each requiring manual cleanup).

Handling Failures in Unattended Pipelines

[Practitioner] Unattended pipelines need failure modes designed in advance. Claude cannot ask you for help when you are not watching.

- **Bound execution:** Use `--max-turns` and `--max-budget-USD` to prevent runaway sessions. A stuck agent consuming tokens for 30 minutes is expensive.
- **Scope tools:** `--allowedTools Read,Edit` prevents an agent from running arbitrary commands. Match tools to the task, not to convenience.
- **Validate output:** Parse `--output-format json` results programmatically. If the output does not match expected structure, fail loudly—do not commit partial results.
- **Gate before merge:** CI pipelines should produce PRs, not direct commits. A human reviews before merge. The automation eliminates toil, not judgment.

13.2. Headless Mode

[Official] Use `claude -p` for non-interactive automation.¹

¹<https://code.claude.com/docs/en/overview>

Tip

Do not promote a pipeline stage until the current stage has succeeded N times without intervention.

Official

```
claude -p "prompt" for
non-interactive use. --output-format
json|stream-json|text.
https://code.claude.com/docs/en/overview
```

- p "prompt"**
Single prompt, no interaction.
- output-format json**
Structured output for pipeline consumption.
- allowedTools**
Restrict to specific tools for safety.
- model**
Override model per invocation.

13.2.1. Fan-Out Pattern

Before & After

Before:

Manual: open Claude, paste file, ask for type hints, copy result, repeat 47 times.

After:

```
# Test on 3 files first
for f in $(find src/ -name "*.py" | head -3); do
  claude -p "Add type hints to $f" \
    --allowedTools Read,Edit
done

# Review results, then scale
find src/ -name "*.py" | while read f; do
  claude -p "Add type hints to $f" \
    --allowedTools Read,Edit \
    --output-format json >> results.jsonl
done
```

[Official] The `--allowedTools` flag scopes what Claude can do in unattended mode.² Use glob patterns for fine-grained control:

```
# Allow Read and Edit only (safest for code modifications)
--allowedTools "Read,Edit"

# Allow Bash but only for specific commands
--allowedTools 'Read,Edit,Bash(git commit *)'
```

Test Before Scaling

Always test your fan-out prompt on 2–3 files before running at scale. A prompt that works on `src/utils.py` may produce unexpected results on `src/models/network.py`. Review the first few outputs manually, then scale with confidence.

Tip

Combine `--allowedTools` with `--output-format json` for pipeline-friendly output. Parse results with `jq` for automated quality checks before committing.

²<https://code.claude.com/docs/en/best-practices>

13.2.2. Permissions in Headless Mode

Headless Permissions

In headless mode, Claude still respects permission rules. For trusted CI/CD environments, use `--dangerously-skip-permissions` to bypass interactive prompts. This flag should **only** be used in environments you fully control (CI runners, Docker containers). Never expose it in user-facing scripts.

13.2.3. Bare Mode for Scripted Calls

[Official] The `--bare` flag skips auto-discovery of hooks, skills, plugins, MCP servers, auto memory, and CLAUDE.md so scripted calls start faster.³

In bare mode, Claude has access to only Bash, file read, and file edit tools. This makes it ideal for CI/CD pipelines, SDK-style automation, and `-p` calls where startup speed matters more than full context.

```
# ~14% faster to API request than a standard session
claude --bare -p "explain this function" < src/main.py
```

[Practitioner] Bare mode pairs naturally with `--json-schema` for structured output and `--max-turns` for bounded execution—the three flags together define a predictable, fast automation primitive.

13.3. Claude Agent SDK

Headless Mode vs. Agent SDK

Headless mode (`claude -p`): quick automation, scripting, CI/CD integration. Uses Claude Code's existing configuration. Best for: fan-out, batch operations, simple pipelines.

Agent SDK: programmatic control, custom tools, production deployment. Best for: custom applications, complex orchestration, SaaS integration.

Rule: Start with headless mode. Move to Agent SDK when you need custom tool definitions or programmatic control flow.

[Official] The Agent SDK provides the same tools, agent loop, and context management that power Claude Code, programmable in Python and TypeScript.⁴

```
# Install
pip install claude-agent-sdk          # Python
npm install @anthropic-ai/claude-agent-sdk # TypeScript
```

Headless Mode vs. Agent SDK

Before:

```
# Headless: shell scripting, quick automation
claude -p "Add type hints to $f" \
  --allowedTools Read,Edit
```

After:

```
# Agent SDK: programmatic control
```

³<https://code.claude.com/docs/en/cli-reference>

⁴<https://platform.claude.com/docs/en/agent-sdk/overview>

Tip

Use `--bare` when you don't need hooks, skills, or MCP. It sets `CLAUDE_CODE_SIMPLE` internally.

Official

Same agent loop as Claude Code, programmable in Python and TypeScript.

<https://platform.claude.com/docs/en/agent-sdk/overview>

```

from claude_agent_sdk import query, ClaudeAgentOptions
import asyncio

async def main():
    async for msg in query(
        prompt="Add type hints to auth.py",
        options=ClaudeAgentOptions(
            allowed_tools=["Read", "Edit"],
        ),
    ):
        if hasattr(msg, "result"):
            print(msg.result)

asyncio.run(main())

```

Key capabilities:

Built-in tools

Read, Write, Edit, Bash, Glob, Grep, WebSearch, WebFetch—no implementation needed.

Hooks

Callback functions for PreToolUse, PostToolUse, Stop, etc.—validate, log, or block agent behavior programmatically.

Subagents

Spawn specialized agents with their own context and tools.

MCP

Connect to external services (databases, browsers, APIs).

Sessions

Resume or fork sessions to maintain context across exchanges.

Filesystem config

CLAUDE.md, skills, and memory all work via `setting_sources=["project"]`.

13.4. Pipeline Design Patterns

13.4.1. Traffic-Light Dashboards

[Practitioner] Replace pass/fail with traffic-light thresholds (GREEN/YELLOW/RED). This provides nuance: 78% coverage is not “failing”—it is YELLOW, meaning “acceptable but track for improvement.”

```

=== Project Health Dashboard ===
Test Coverage:    82% [GREEN]
Lint Warnings:    3  [YELLOW]
Documentation:    91% [GREEN]
Model Metrics:    AUC 0.84 [GREEN]
Overall:          HEALTHY (3 GREEN, 1 YELLOW)

```

Tip

Start with headless mode (`claude -p`). Move to the Agent SDK when you need custom tool definitions, programmatic hooks, or production error handling.

13.4.2. Pipeline Directionality

Pipeline Direction

Never re-run upstream steps after editing downstream artifacts. Mark generated files with `# GENERATED --- do not edit manually`. This is anti-pattern #5 from Chapter 12.

13.5. Scheduled Tasks

[Official] Claude Code offers three ways to schedule recurring work.⁵

	Cloud	Desktop	/loop	Best For
Runs on	Anthropic cloud	Your machine	Your machine	Three tiers of recurring automation, each with different tradeoffs. https://code.claude.com/docs/en/web-scheduled-tasks
Requires machine	No	Yes	Yes	
Persistent	Yes	Yes	No (session)	
Local files	No (fresh clone)	Yes	Yes	
Min interval	1 hour	1 minute	1 minute	
				Cloud: daily PR reviews, dependency audits Desktop: local builds, file watches /loop: deploy polling, session-scoped checks

Cloud (/schedule)

Create from CLI, web, or Desktop app. Runs on Anthropic infrastructure—works even when your machine is off. Each run clones the repo fresh, executes the prompt, and creates a session you can review.

Desktop

Schedule tasks that run on your machine with access to local files and MCP servers.

/loop

Bundled skill. Runs a prompt repeatedly on an interval while the session stays open: `/loop 5m check if the deploy finished`. Session-scoped—stops when you close the session.

13.6. GitHub Actions Integration

[Official] Claude Code GitHub Actions brings AI-powered automation to your GitHub workflow.⁶ Mention `@claude` in any PR or issue comment, and Claude can analyze code, create pull requests, implement features, and fix bugs—following your `CLAUDE.md` guidelines.

```
# .github/workflows/claude.yml
name: Claude Code
on:
  issue_comment:
    types: [created]
  pull_request_review_comment:
    types: [created]
```

⁵<https://code.claude.com/docs/en/web-scheduled-tasks>

⁶<https://code.claude.com/docs/en/github-actions>

Tip

Use **Cloud** for work that must happen reliably (daily reviews, weekly audits).

Use **/loop** for quick polling during active work. Use **Desktop** when you need local filesystem access.

Official

Run `/install-github-app` to set up Claude Code as a GitHub Action.

<https://code.claude.com/docs/en/github-actions>

```
jobs:
  claude:
    runs-on: ubuntu-latest
    steps:
      - uses: anthropics/claude-code-action@v1
        with:
          anthropic_api_key: ${ secrets.ANTHROPIC_API_KEY }
```

Setup

`/install-github-app` walks through installation. Alternatively, install the Claude GitHub app manually and add `ANTHROPIC_API_KEY` as a repository secret.

Customization

Use `claude_args` to pass CLI flags: `--max-turns`, `--model`, `--allowedTools`.

Cloud providers

Supports direct API, AWS Bedrock, and Google Vertex AI authentication.

13.7. Remote Control and Web Sessions

[Official] Remote Control connects `claude.ai/code` or the Claude mobile app to a session running on your machine.⁷ Start a task at your desk, then pick it up from your phone or another browser. Your local filesystem, MCP servers, and project configuration stay available.

`/remote-control`

Enable from an existing session. Also available as `claude --remote-control` or server mode `claude remote-control`.

Claude Code on the web

Runs on Anthropic cloud infrastructure—no local machine needed. Use for repos you don't have cloned or for parallel cloud tasks.

Key Insight

T

Traditional automation breaks on unexpected input—a file with an unusual encoding, a function signature it was not designed for. Claude-powered automation bends: it reads the unusual file, adapts its approach, and continues. This means the boundary between “common case” (automated) and “exception” (manual) shifts dramatically. Tasks that required human judgment in a bash pipeline—like deciding how to refactor a function with complex dependencies—become part of the automated path. Design your pipelines with this wider boundary in mind: the trust gates (Section 13.1) are the new exception handling.

Try This

1. Write a `claude -p` command that adds type hints to one file in your project.
2. Test it. Does the output compile? Do tests pass?
3. If yes: scale to 3 files. Review the results.

⁷<https://code.claude.com/docs/en/remote-control>

Cross-Ref

For the full comparison of Remote Control, Dispatch, Channels, Slack, and Scheduled Tasks, see the official docs.

<https://code.claude.com/docs/en/remote-control>

4. If no: refine the prompt. What did you need to specify that you assumed?

14. Team Patterns and Governance

You have been using Claude effectively for months. Your team lead asks: “Can we standardize this across our 8-person team?” The answer is yes—but individual mastery does not automatically translate to team productivity. Shared conventions require infrastructure.

14.1. Shared CLAUDE.md in Version Control

The project-level `CLAUDE.md` (structured per Chapter 9) and `.claude/settings.json` are committed to version control. Every team member gets the same project context, the same permissions, and the same hooks.

[Practitioner] Three files form the team standard:

CLAUDE.md

Project knowledge: build commands, architecture, conventions. Reviewed in PRs like any other code.

.claude/settings.json

Team permissions: deny rules, required hooks. Committed to git.

.claude/settings.local.json

Personal preferences: model choice, output style, personal API tokens. Auto-gitignored.

Official

Claude Code included with Team plan seats. Shared CLAUDE.md in VCS.
<https://support.claude.com/en/articles/11845131-use-claude-code-with-your-team-or-enterprise-plan>

Why Separate Committed and Local Settings

If model choice is in `settings.json`, developers who need Opus for complex tasks must override team settings. If it is in `settings.local.json`, each developer chooses their own model without affecting others.

The principle: team standards (deny rules, hooks) are committed. Personal preferences (model, style) are local.

14.2. Managed Settings for IT

[Official] IT administrators can deploy `managed-settings.json` files that override all other settings.¹ This enables:

- Organization-wide deny rules (sensitive file access)
- Mandatory hooks (security scanning, compliance checks)
- Approved MCP server lists
- Model restrictions (e.g., Sonnet only for cost control)

Managed settings deploy to system-level paths and cannot be overridden by project or user settings (see Section 9.4 for the full 5-level precedence hierarchy). They are the IT equivalent of hooks—non-negotiable.

Hook Precedence and Debugging

When hooks are configured at multiple levels, they all run—managed, project, and local hooks execute for the same event. If hooks conflict (e.g., a managed hook requires formatting that a local hook undoes), managed takes priority.

¹<https://code.claude.com/docs/en/settings>

To debug unexpected hook behavior:

- Check which hooks are active: review `settings.json` at each level
- Use `/hooks` to view the merged hook configuration
- Check hook exit codes: non-zero exit blocks the triggering action
- Test hooks in isolation before deploying to managed settings

Delivery Mechanisms

[Official] Managed settings can be delivered through multiple channels, all at the highest priority level.²

Mechanism	Description
Server-managed	Delivered from Anthropic servers via Claude.ai admin console (Teams/Enterprise). Fetched at startup, polled hourly.
macOS MDM	<code>com.anthropic.claudecode</code> managed preferences domain (Jamf, Kandji)
Windows Registry	<code>HKLM\SOFTWARE\Policies\ClaudeCode</code> with <code>Settings REG_SZ</code> (Group Policy/Intune)
File-based	<code>managed-settings.json</code> at system paths: <code>/etc/claude-code/</code> (Linux), <code>/Library/Application Support/ClaudeCode/</code> (macOS)

A drop-in directory `managed-settings.d/` alongside `managed-settings.json` supports policy fragments. Files merge alphabetically—use numeric prefixes for ordering (e.g., `10-security.json`, `20-models.json`).

Key Managed-Only Settings

These settings are only available at the managed level:

`allowManagedPermissionRulesOnly`

Only managed allow/deny rules apply—users and projects cannot add their own.

`allowManagedHooksOnly`

Prevent user, project, and plugin hooks entirely.

`allowManagedMcpServersOnly`

Restrict MCP servers to an admin-defined allowlist.

`disableBypassPermissionsMode`

Prevent `--dangerously-skip-permissions`.

`availableModels`

Restrict which models users can select.

`companyAnnouncements`

Display messages to users at startup.

```
{
  "permissions": {
    "deny": ["Bash(curl *)", "Read(./env)"],
    "disableBypassPermissionsMode": "disable"
  },
}
```

²<https://code.claude.com/docs/en/server-managed-settings>

Enterprise

Server-managed settings require Teams or Enterprise plan. Claude Code 2.1.38+.

<https://code.claude.com/docs/en/server-managed-settings>

```
"availableModels": ["sonnet", "haiku"],  
"companyAnnouncements": [  
  "Review AI usage guidelines at docs.acme.com/ai"  
]  
}
```

14.3. Onboarding New Team Members

[Practitioner] Day 1 with Claude for a new team member:

1. Clone the repo. CLAUDE.md and settings load automatically.
2. Run `/init` to verify Claude understands the project. Compare output to the existing CLAUDE.md—they should align.
3. Create `.claude/settings.local.json` for personal preferences.
4. Run the team's `/check` command to verify the environment works.
5. Review the team's hook configuration: what runs on file write? On commit? Why?

The goal: a new developer should be productive with Claude within their first day, without reading a setup guide. The configuration in version control should be self-documenting.

Warning

Server-managed settings are client-side controls. For unmanaged devices, use MDM for stronger enforcement.

14.4. Shared Extensions at Scale

In Chapter 8, you learned the extension hierarchy: commands, skills, hooks, and MCP servers. At team scale, these mechanisms need governance:

Shared skills

Commit team skills to `.claude/skills/` in the repo. Quality dashboards, pre-release checklists, and onboarding workflows benefit the entire team. Use `context: fork` for expensive skills so they do not consume teammates' context budgets.

Managed MCP

IT-approved MCP servers deployed via managed settings (Section 14.2). Prevents ad-hoc server installation while providing standardized access to team databases, ticketing systems, and monitoring tools.

Shared hook libraries

Common hooks (auto-format, test gates, lint) maintained in a central repository and imported via the hub-and-spoke pattern (Section 9.1). Update once, all projects inherit.

[Practitioner] The extension hierarchy scales naturally: commands and skills in the repo for the team, hooks in `settings.json` for enforcement, managed MCP for IT-controlled integrations.

14.5. Code Review Patterns with Claude

[Practitioner] Two patterns for team code review with Claude (see also the Writer/Reviewer pattern in Section 10.5):

14.5.1. Pre-PR Self-Review

Before creating a PR, have Claude review your own changes:

```
Review the diff in `git diff main...HEAD` for:  
1. Logic errors and bugs  
2. Missing test coverage  
3. Style violations per our CLAUDE.md  
4. Security concerns  
Report issues with file:line references and severity.
```

This catches obvious issues before reviewers see the code, reducing review cycles.

14.5.2. Reviewer-Assist

During review, use Claude to investigate specific concerns:

```
In PR #234, the author changed the feature  
normalization strategy in  
src/features/preprocessing.py.  
What are the implications for models trained on  
the old normalization? Check if any downstream  
models depend on the old feature distributions.
```

14.6. Quick Reference

- Commit CLAUDE.md and settings.json for team standards
- Keep settings.local.json for personal preferences (auto-gitignored)
- Managed settings for IT policy enforcement (highest precedence)
- New team members productive on day 1 via committed configuration
- Pre-PR self-review catches issues before human reviewers see them

Try This

Draft a team settings.json with:

1. 3 deny rules protecting sensitive files (.env*, secrets/, credentials)
 2. 1 ask rule for destructive operations (git push)
 3. 1 PostToolUse hook (matching Edit|Write) for auto-formatting
 4. 1 PreToolUse hook (matching Bash) gating commits on your test suite
- Have a teammate review it. Does it match their expectations?

Tip

The reviewer still makes the decision. Claude provides the analysis. AI augments review; it does not replace it.

15. Enterprise Deployment

Your VP asks: “Is Claude secure enough for our regulated environment? What certifications does it have? What will it cost at 200 engineers?” This chapter is designed to be shared with decision-makers.

15.1. Security and Compliance

[Official] Anthropic holds industry certifications:¹

SOC 2 Type II

Verified controls for security, availability, and confidentiality. Request the report via the Trust Portal.

ISO 27001:2022

Information security management system.

ISO/IEC 42001:2023

AI management system—one of the first AI companies certified.

HIPAA BAA

Healthcare data protection agreements available on request.

[Official] Government and regulated-industry authorizations:²

FedRAMP High

US government cloud security baseline, available via AWS GovCloud and Google Cloud (Palantir FedStart).

IL5 Impact Level 5 authorization for controlled unclassified information on DoD workloads.

Enterprise
Certifications verified Feb 2026. Trust portal:
<https://trust.anthropic.com/>
Certification details:
<https://privacy.claude.com/en/articles/10015870-what-certifications-has-anthropic-obtained>

15.1.1. Identity and Access Management

[Official] Enterprise-grade IAM:³

- SSO via SAML 2.0 and OIDC
- SCIM provisioning for automated user lifecycle
- Role-based access control (RBAC)
- Audit logs for all interactions
- Compliance API and Analytics API for programmatic access

Warning
FedRAMP High and IL5 are government authorizations, not commercial certifications. They appear on the government solutions page, not the trust portal. Verify current status at <https://claude.com/solutions/government>.

15.1.2. Data Protection

[Official] Key data protection features:⁴

- **Encryption:** TLS 1.2+ in transit, AES-256 at rest.
- **Zero Data Retention (ZDR):** Available for API and Enterprise plans.
- **No training on enterprise data:** Enterprise and API data is not used for model training by default. Explicit opt-in required. Free and Pro tier usage

¹<https://privacy.claude.com/en/articles/10015870-what-certifications-has-anthropic-obtained>

²<https://claude.com/solutions/government>

³<https://privacy.claude.com/en/articles/10015870-what-certifications-has-anthropic-obtained>

⁴<https://privacy.claude.com/en/articles/7996885-how-do-you-use-personal-data-in-model-training>

policies differ—consult privacy.claude.com for current terms.

15.2. Deployment Scenario: 50-Person Rollout

[Practitioner] A realistic 4-week enterprise adoption timeline:

Week	Phase	Activities
1	Pilot (5 developers)	Install Claude Code. Create shared CLAUDE.md. Establish deny rules. Identify 3 high-value workflows.
2	Expand (15 developers)	Deploy managed-settings.json. Add PreToolUse hooks. Create team commands. Measure baseline productivity.
3	Scale (50 developers)	SSO integration. Shared MCP configurations. Training sessions using this handbook. Monitor usage.
4	Optimize	Analyze usage patterns. Tune model selection for cost. Implement caching strategy. Set spending caps.

15.3. Cost Optimization at Scale

- Optimize CLAUDE.md for caching.** Stable tool definitions and system prompts cache automatically. Restructure to put stable content first.
- Use Batch API for bulk operations.** Code review, documentation generation, test scaffolding—all benefit from 50% batch discounts.
- Right-size models.** **[Practitioner]** Sonnet handles the majority of coding tasks. Opus for architecture and complex reasoning. Haiku for formatting and simple lookups.
- Set spending caps.** Admin controls prevent runaway costs from long-running agent sessions.

15.3.1. API Pricing Reference

[Official] Claude API pricing (per million tokens):⁵

Model	Input	Output	Max Output	Notes
Haiku 4.5	\$1	\$5	64K tokens	Fastest, lowest cost
Sonnet 4.6	\$3	\$15	64K tokens	Best balance for daily development
Opus 4.6	\$5	\$25	128K tokens	Frontier reasoning, 1M context
Opus 4.6 (fast)	\$30	\$150	128K tokens	2.5× faster (research preview)

Discounts (combinable):

Cache reads	90% off input (0.1× base price)
Batch API	50% off input and output

⁵<https://platform.claude.com/docs/en/about-claude/pricing>

Enterprise
Enterprise/API data is never used for model training by default. This is a contractual guarantee.
<https://privacy.claude.com/en/articles/7996885-how-claude-uses-personal-data-in-model-training>

Cost
Prompt caching (up to 90%) + Batch API (50%) = up to 70%+ combined savings (workload-dependent).
<https://platform.claude.com/docs/en/build-with-claude/prompt-caching>

Cost
Pricing as of March 2026. Verify current rates at <https://claude.com/pricing> before budgeting.
<https://platform.claude.com/docs/en/about-claude/pricing>

Pricing Volatility

Model pricing changes with each generation. The table above is a snapshot—do not use it for long-term budget projections. Check the pricing page before each planning cycle. This table is tracked in `docs/volatility-watchlist.md` for staleness monitoring.

Caching Beats Model Downgrading

The instinct to save money by using a cheaper model sacrifices quality across every interaction. Optimizing for cache hits preserves quality while reducing costs by a larger margin. A stable CLAUDE.md that caches well typically saves more than switching from Opus to Sonnet.

15.4. Enterprise Deployments at Scale

Organization	Scale	Use Case
Deloitte ⁶	470K employees	Role-customized Claude “personas” for consulting
Accenture ⁷	30K trained	CIO-focused ROI measurement
Snowflake ⁸	12.6K customers	Text-to-SQL integration

15.5. Competitive Positioning

Claude vs. GPT by Task Profile

Choose Claude when: long-document analysis, agentic coding workflows, enterprise compliance requirements (FedRAMP High, ISO 42001), API-first development.

Choose GPT when: consumer-facing features, multimodal applications requiring image generation.

Note: Claude is now available natively in GitHub Copilot^a (public preview Feb 2026, included at no extra charge with existing Copilot subscriptions). “Existing Copilot integration” is no longer a differentiator—both models are accessible through the same IDE.

Reality: most serious enterprise deployments use both, choosing by task profile. The question is not “which is better” but “which is better for this specific use case.”

^a<https://github.blog/news-insights/product-news/bringing-developer-choice-to-copilot/>

15.6. Government and Regulated Industries

[Practitioner] Government deployment options include:

FedRAMP High

Available via Palantir FedStart integration.⁹

AWS GovCloud

Cloud deployment for regulated requirements.¹⁰

⁹<https://claude.com/solutions/government>

¹⁰<https://claude.com/solutions/government>

Google Vertex AI

Alternative cloud deployment for regulated workloads.¹¹

15.7. When to Deploy: Decision Matrix

Need	Claude Code	Claude Chat	API / SDK
Interactive dev	Primary tool	Quick questions	—
Automation	Headless mode (Chapter 13)	—	Primary tool
Team standards	Settings + hooks (Chapter 14)	—	—
Compliance	Enterprise plan	Enterprise plan	API + ZDR
Custom apps	—	—	Agent SDK
Bulk operations	Fan-out	—	Batch API

Key Insight

T

he enterprise teams that get the most value from Claude Code share one pattern: they invest in CLAUDE.md and hooks *before* buying seats. A well-configured repo means every developer—from day one— gets consistent, project-aware AI assistance. Without that configuration, each developer reinvents their own workflow, leading to inconsistent code quality and wasted tokens rediscovering project conventions. The 50-person rollout scenario above works because Week 1 configures the repo; Week 2 deploys managed settings; the seats come last. See Chapter A for a framework to measure organizational adoption.

Try This

Estimate your team's monthly Claude cost:

1. Count developers × estimated sessions per day × average tokens per session.
2. Identify your top caching opportunity (what content is repeated in every session?).
3. Calculate: what would a 60% cache hit rate save monthly?
4. Present the estimate to your manager with the deployment scenario timeline from this chapter.

Warning

Government program terms change frequently. Verify current offering details directly at <https://claude.com/solutions/government> before making procurement decisions.

¹¹<https://claude.com/solutions/government>

A. The Claude Maturity Model

This model provides concrete milestones for Claude Code adoption. Use it to assess where you are and what to invest in next. Each Part introduction in this handbook references the levels below.

A.1. Five Levels of Claude Adoption

Level	Name	Key Capabilities	Milestone
L1	Conversational	Default Claude, chat-style usage, no configuration	Create first CLAUDE.md (/init)
L2	Configured	CLAUDE.md, custom commands, permissions, basic hooks	5+ commands in a workflow chain
L3	Systematic	Hub-and-spoke, conditional rules, sessions, memory, plans	Hub repo with 3+ spoke projects
L4	Autonomous	Skills, MCP servers, sub-agents, parallel workflows, dashboards	Autonomous quality pipeline
L5	Enterprise	Agent SDK, agent teams, CI/CD, team-wide standards, SSO	Team-wide deployment with metrics

A.2. Upgrade Checklists

A.2.1. L1 → L2: From Conversational to Configured

- ☐ Create CLAUDE.md with build commands and code conventions
 - ☐ Add .claude/settings.json with deny rules for secrets
 - ☐ Create 3+ custom commands for common workflows
 - ☐ Add PostToolUse hook (matching Edit|Write) for auto-formatting
 - ☐ Add PreToolUse hook (matching Bash) gating commits on test suite
- Reference: Chapter 2, Chapter 3

A.2.2. L2 → L3: From Configured to Systematic

- ☐ Create hub directory for shared patterns
 - ☐ Convert 3+ projects to hub-and-spoke imports
 - ☐ Add conditional rules (paths: frontmatter) for distinct modules
 - ☐ Use sessions with named checkpoints (/rename)
 - ☐ Maintain CURRENT_WORK.md for cross-session context
 - ☐ Create first plan document for a large task
 - ☐ Use thinking-partner techniques (hypothesis debugging, assumption surfacing)
- Reference: Chapter 9, Chapter 6, Chapter 5

A.2.3. L3 → L4: From Systematic to Autonomous

- ☐ Create skills for repeated multi-step workflows
- ☐ Add MCP server for external tool access
- ☐ Define custom subagents for specialized tasks
- ☐ Implement writer/reviewer pattern for code review
- ☐ Build traffic-light quality dashboard
- ☐ Run parallel workflows with git worktrees

Reference: Chapter 8, Chapter 10

A.2.4. L4 → L5: From Autonomous to Enterprise

- ☐ Deploy managed-settings.json for organization-wide policy
- ☐ Implement headless mode in CI/CD pipeline
- ☐ Establish team CLAUDE.md standards committed to VCS
- ☐ Set up SSO and audit logging
- ☐ Build fan-out automation for bulk operations
- ☐ Track usage metrics and cost optimization

Reference: Chapter 13, Chapter 14, Chapter 15

A.3. Progression Strategy

[Practitioner] Promote when friction justifies it, not on a schedule.

1. **Stay at L1** until you find yourself repeating the same context to Claude across sessions.
2. **Move to L2** when CLAUDE.md and 3–5 commands would eliminate daily repetition.
3. **Move to L3** when 3+ projects share patterns or when plans would prevent wasted effort on large tasks.
4. **Move to L4** when multi-step workflows consume significant time or external tools would unlock new capabilities.
5. **Move to L5** when team adoption, compliance, or production automation requires enterprise infrastructure.

Premature Promotion

Jumping from L1 to L4 is a common mistake. Each level builds on the habits and infrastructure of the previous one. Skills without CLAUDE.md produce inconsistent results; MCP servers without permissions create security risks; agent teams without plan documents produce uncoordinated work.

A.4. Visual: Maturity Pyramid

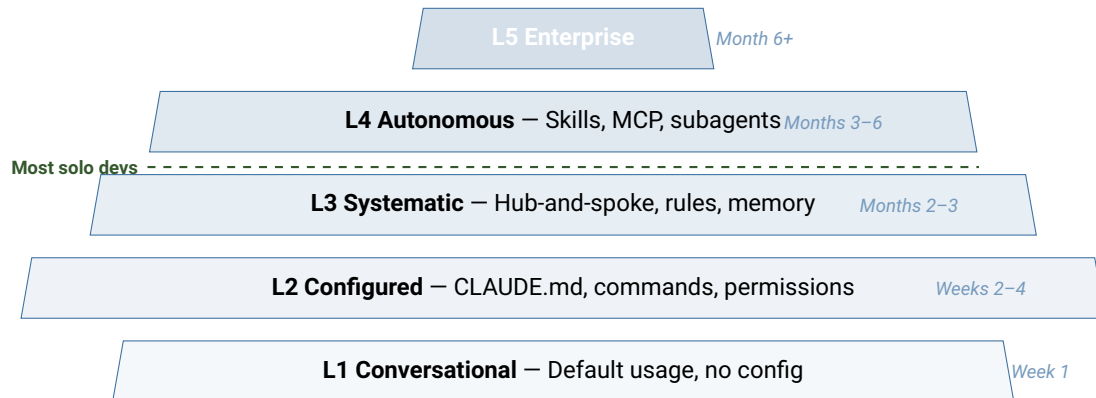


Figure A.1.: The maturity pyramid. Most solo developers stabilize at L3.

B. Quick-Start Templates

These templates are available as standalone files in the `templates/` directory of the repository. Copy them into your project and customize. Each template includes WHY comments explaining the purpose of each section.

B.1. Minimal CLAUDE.md (10 lines)

```
# Project Name

## Build & Verify
# WHY: Claude cannot guess non-standard build commands
- `pytest tests/` - run test suite
- `mypy src/` - type checking
- `ruff check src/` - lint

## Code Style
# WHY: Only include rules that differ from standard conventions
- Type hints on all function signatures
- Docstrings on all public functions

## Testing
# WHY: Verification criteria improve every interaction
- Run tests after every code change
- Never commit without passing lint + tests
```

B.2. Production CLAUDE.md (50 lines)

```
# Project Name

## Build & Verify
# WHY: Every command Claude needs to verify its own work
- `pytest tests/` - run test suite
- `mypy src/` - type checking
- `ruff check src/` - lint
- `nbstripout notebooks/` - strip notebook outputs

## Architecture
# WHY: Tells Claude WHERE things go, not just what they are
- src/features/ - feature engineering pipelines
- src/models/ - model training and evaluation
- src/data/ - data loading and validation
- notebooks/ - exploratory analysis
- data/raw/ - immutable source data
- data/processed/ - transformed artifacts
- tests/ - mirrors src/ structure

## Code Conventions
# WHY: Only rules Claude wouldn't infer from the codebase
- Type hints on all function signatures
- Functional style: prefer pure functions, immutable DataFrames
- Error handling: raise ValueError/TypeError, never swallow

## Testing Standards
# WHY: These lines improve every interaction (Ch 6)
- Run tests after every code change
- Never commit without passing lint + tests + type check
- Coverage target: 80% for src/ modules
- Every new function needs: happy path + error case + edge case
```

```
## ML Standards
- No data leakage: features must use only pre-split data
- Log all hyperparameters to MLflow
- Include reproducibility seed in all configs

## Git Conventions
- Conventional commits: feat/fix/refactor/test/docs
- PR description includes: Summary, Test Plan, Breaking Changes

## Security
# WHY: Defense in depth – CLAUDE.md + deny rules + hooks
- Never read .env files
- Never commit secrets, API keys, or credentials
```

B.3. Settings Template

```
{
  "permissions": {
    "allow": [
      "Read", "Grep", "Glob",
      "Write(src/**)", "Write(tests/**)",
      "Edit(src/**)", "Edit(tests/**)",
      "Bash(pytest *)", "Bash(mypy *)", "Bash(ruff *)",
      "Bash(git status)", "Bash(git diff *)",
      "Bash(git log *)", "Bash(git add *)"
    ],
    "deny": [
      "Read(.env*)",
      "Read(secrets/**)",
      "Read(**/*.pem)",
      "Read(**/*.key)",
      "Read(**/credentials*)",
      "Bash(rm -rf /)",
      "Bash(sudo *)"
    ],
    "ask": [
      "Bash(git commit *)",
      "Bash(git push *)"
    ]
  }
}
```

Note: Permission rules support both glob syntax (`Bash(git push *)`, `Edit(/src/**/*ts)`) and colon-style constraints (`WebFetch(domain:example.com)`). Evaluation order: deny first, then ask, then allow.

B.4. Hook Configuration

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": ".claude/hooks/lint-on-write.sh",
        "description": "Auto-lint written files"
      }]
    }],
    "PreToolUse": [{
      "matcher": "Bash",
```

```

    "hooks": [{
      "type": "command",
      "command": ".claude/hooks/pre-commit-gate.sh",
      "description": "Block commits with test failures or type errors"
    }]
  }
}

```

B.5. Skill Template

```

---
name: pipeline-check
description: Pre-merge pipeline readiness verification
allowed-tools: [Bash, Read, Glob, Grep]
---
# Pipeline Check
# WHY: Catches ML pipeline issues before they reach main
1. Verify all tests pass: pytest tests/`
2. Check for uncommitted changes: git status`
3. Verify no data leakage in feature code
4. Check model metrics logged to MLflow
5. Report readiness status (READY / NOT READY)
   with specific blockers if NOT READY

```

Note: Skills use `allowed-tools` (hyphenated).

B.6. Subagent Definition Template

```

# .claude/agents/code-reviewer.md
---
name: code-reviewer
description: Review code changes for quality issues
tools: [Read, Grep, Glob]
---
# WHY: Isolated context prevents review bias from accumulated session context
Review the provided code changes for:
1. Logic errors and bugs
2. Missing error handling
3. Security vulnerabilities
4. Performance concerns
5. Style violations

Report findings with severity: Critical / High / Medium / Low.
Include file:line references for each finding.

```

Note: Subagents use `tools` (not `allowed-tools` or `allowed_tools`).

B.7. Rules File with Paths

```

---
paths: src/features/**/*.py
---
# Feature Engineering Standards
# WHY: These rules only load when editing feature code,
# keeping context clean when working on other parts of the project
- Pure functions: no side effects in transformers
- Validate input schema before transformation
- Type hints on all function signatures

```

- Docstrings on all public functions
- Run ``pytest tests/features/`` after changes

B.8. Custom Command Template

```
# .claude/commands/check.md
# WHY: Encodes a decision workflow, not just a shortcut
Run a comprehensive check of the project:
```

1. Run the test suite: ``pytest tests/``
2. Run the linter: ``ruff check src/``
3. Run the type checker: ``mypy src/``
4. Report results as a summary table:

Check	Status	Details
Tests	PASS/FAIL	X passed, Y failed
Lint	PASS/FAIL	N warnings
Types	PASS/FAIL	N errors

C. Command and Hook Reference

C.1. Built-In Commands

Built-in commands execute fixed logic directly. Type `/` to see all available commands. Not all commands are visible to every user—some depend on platform, plan, or environment.

Official
Complete command list verified
2026-03-25 against
<https://code.claude.com/docs/en/commands>

Command	Description
Session & Context	
<code>/clear</code>	Clear context, start fresh. Aliases: <code>/reset</code> , <code>/new</code>
<code>/compact [instructions]</code>	Summarize conversation with optional focus instructions
<code>/rewind</code>	Roll back to a previous checkpoint, or summarize from a selected message. Alias: <code>/checkpoint</code>
<code>/context</code>	Visualize current context usage as a colored grid
<code>/btw <question></code>	Ask a side question without adding to conversation history
<code>/rename [name]</code>	Give current session a descriptive name
<code>/resume [session]</code>	Resume a conversation by ID or name. Alias: <code>/continue</code>
<code>/branch [name]</code>	Branch the current conversation at this point. Alias: <code>/fork</code>
<code>/export [filename]</code>	Export the current conversation as plain text
<code>/copy [N]</code>	Copy Nth-latest response to clipboard. Interactive picker for code blocks
Model & Reasoning	
<code>/model [model]</code>	Select or change the AI model
<code>/effort [level]</code>	Set effort: <code>low</code> , <code>medium</code> , <code>high</code> , <code>max</code> , <code>auto</code> (reset to default)
<code>/fast [on off]</code>	Toggle fast mode (same Opus 4.6, 2.5× faster, higher cost) ¹
<code>/plan</code>	Enter Plan Mode from the prompt
Configuration & Extensions	
<code>/init</code>	Auto-generate CLAUDE.md from project structure
<code>/config</code>	Open Settings interface (theme, model, output style). Alias: <code>/settings</code>
<code>/memory</code>	View and manage CLAUDE.md files and auto-memory
<code>/permissions</code>	View and modify permission rules. Alias: <code>/allowed-tools</code>
<code>/hooks</code>	View hook configurations for tool events
<code>/mcp</code>	Manage MCP server connections and OAuth
<code>/plugin</code>	Browse and install community plugins
<code>/skills</code>	List available skills
<code>/agents</code>	Manage subagent configurations
<code>/sandbox</code>	Toggle OS-level sandboxing
<code>/statusline</code>	Configure the status line display
<code>/vim</code>	Toggle between Vim and Normal editing modes
<code>/theme</code>	Change the color theme
<code>/color [color]</code>	Set prompt bar color for the current session
<code>/chrome</code>	Configure Claude in Chrome browser integration

Code Review Plugin

The `code-review` plugin uses four parallel agents to review code changes, each scoring findings independently on a 0–100 confidence scale (default threshold: 80).

Install:

```
claude plugin install code-review@claude-plugins-official
```

Usage:

```
/code-review [--comment]
```

The four agents run in parallel:

1. Two redundant CLAUDE.md compliance checkers (catches project convention violations)
2. Bug and logic error detector
3. Git-blame history analyzer (understands change context)

Each finding is scored by a separate scorer agent. Pre-existing issues, linter-catchable problems, and pedantic nitpicks are automatically filtered.

Use `--comment` to post results directly on a PR.

Official
For the complete list of ~50 built-in commands, see <https://code.claude.com/docs/en/commands>.

Official
Replaces deprecated `/review`. Multi-agent architecture with confidence scoring.
<https://github.com/anthropics/claude-code/tree/main/plugins/code-review>

C.2. Bundled Skills

Bundled skills ship with Claude Code and appear alongside built-in commands when you type `/`. They are not fixed logic—they are prompt-based workflows that Claude executes.

Skill	Description
<code>/batch <instruction></code>	Parallel codebase changes: researches, decomposes into 5–30 units, spawns one background agent per unit in an isolated worktree. Each agent implements, tests, and opens a PR.
<code>/loop [interval] <prompt></code>	Run a prompt repeatedly on an interval (default 10 min). Session-scoped polling for deploys, PRs, or recurring checks.
<code>/simplify [focus]</code>	Review recently changed files for reuse, quality, and efficiency. Spawns 3 parallel review agents.
<code>/debug [description]</code>	Troubleshoot your Claude Code session by reading the debug log.
<code>/claude-api</code>	Load Claude API and Agent SDK reference for your project's language.

Official
Bundled skills are prompt-driven: to give Claude a playbook and let it orchestrate using tools. Unlike built-in commands, skills can spawn subagents and adapt to your codebase.
<https://code.claude.com/docs/en/skills>

¹Fast mode is a research preview. Same Opus 4.6 model, \$30/\$150 MTok pricing, falls back to standard on rate limit. Requires extra usage enabled. See <https://code.claude.com/docs/en/fast-mode>.

C.3. Session Management Flags

Flag	Description
<code>--continue</code>	Resume most recent session
<code>--resume</code>	Pick from list of previous sessions
<code>--from-pr 123</code>	Start session with PR context pre-loaded
<code>--worktree</code>	Create isolated git worktree for parallel work
<code>-p "prompt"</code>	Headless mode (non-interactive, single prompt)
<code>--output-format</code>	Output format: json, stream-json, text
<code>--allowedTools</code>	Restrict available tools (comma-separated)
<code>--disallowedTools</code>	Deny specific tools (comma-separated)
<code>--model</code>	Override model: opus, sonnet, haiku
<code>--permission-mode</code>	Set permission mode for the session
<code>--output-style</code>	Override output style
<code>--add-dir <path></code>	Add additional working directories
<code>--bare</code>	Minimal mode: skip hooks, skills, plugins, MCP, memory, CLAUDE.md
<code>--name / -n</code>	Set session display name (shown in <code>/resume</code> picker)
<code>--max-turns</code>	Limit agentic turns (print mode only)
<code>--max-budget-usd</code>	Maximum API spend before stopping (print mode only)
<code>--system-prompt</code>	Replace entire system prompt with custom text
<code>--append-system-prompt</code>	Append to default system prompt
<code>--json-schema</code>	Validated JSON output matching schema (print mode only)
<code>--enable-auto-mode</code>	Unlock auto mode in Shift+Tab cycle (Team plan+)
<code>--effort</code>	Set effort level: low, medium, high, max
<code>--agent <name></code>	Start session with a specific agent definition
<code>--agents <json></code>	Define subagents dynamically via inline JSON
<code>--mcp-config <file></code>	Load MCP servers from a JSON config file
<code>--strict-mcp-config</code>	Use <i>only</i> MCP servers from <code>--mcp-config</code> (ignore settings)
<code>--plugin-dir <path></code>	Load plugins from a local directory
<code>--tools</code>	Restrict available tools for the session
<code>--fallback-model</code>	Auto-fallback model when default is overloaded
<code>--chrome / --no-chrome</code>	Enable or disable Chrome browser integration
<code>--remote</code>	Create a web session on claude.ai
<code>--teleport</code>	Resume a web session locally
<code>--dangerously-skip-permissions</code>	Skip all permission prompts (CI only)

C.4. Keyboard Shortcuts

Shortcut	Action
Shift+Tab / Alt+M	Cycle through enabled permission modes (default, acceptEdits, plan, and any modes you have enabled such as auto or bypassPermissions)
Ctrl+G	Open prompt or custom response in your default text editor
Alt+T	Toggle extended thinking
Alt+O	Toggle fast mode
Alt+P	Switch model without clearing prompt
Ctrl+C	Cancel current generation
Ctrl+B	Background a running task (tmux users press twice)
Ctrl+O	Toggle verbose output (show tool details, expand MCP calls)
Ctrl+T	Toggle task list display
Ctrl+R	Reverse search through command history
Ctrl+V	Paste image from clipboard
Esc	Stop Claude mid-action (context preserved)
Esc + Esc	Open rewind menu (restore conversation, code, or both; or summarize from a selected message)

C.5. Hook Events (25 Total)

Official

Shortcuts verified 2026-03-25. macOS Option/Alt shortcuts require configuring Option as Meta in your terminal.

<https://code.claude.com/docs/en/interactive-mode>

Official

Event list verified 2026-03-26. Matchers are specific enum values, not arbitrary regex.

<https://code.claude.com/docs/en/hooks>

Event	Matcher Values	Typical Use
SessionStart	startup, resume, clear, compact	Load context, health check
InstructionsLoaded	session_start, nested_traversal, path_glob_match, include, compact	Log which files load
UserPromptSubmit	(no matcher)	Transform or validate input
PreToolUse	Tool name	Validate inputs, block ops
PermissionRequest	Tool name	Auto-allow/deny by rule
PostToolUse	Tool name	Auto-lint, log, follow-up
PostToolUseFailure	Tool name	Error handling, retry
Notification	permission_prompt, idle_prompt, auth_success, elicitation_dialog	Route alerts
Elicitation	MCP server name	MCP requests user input
ElicitationResult	MCP server name	User responds to dialog
SubagentStart	Agent type/name	Init agent config
SubagentStop	Agent type/name	Process agent results
TaskCreated	(no matcher)	Post-creation automation
TaskCompleted	(no matcher)	Post-task automation
Stop	(no matcher)	Run cleanup
StopFailure	rate_limit, authentication_failed, billing_error, server_error, max_output_tokens, unknown	Handle session failures
TeammateIdle	(no matcher)	Notify when agent idles
ConfigChange	user_settings, project_settings, local_settings, policy_settings, skills	React to config changes
CwdChanged	(no matcher)	Respond to dir change
FileChanged	Filename basename	React to file changes
WorktreeCreate	(no matcher)	Custom worktree setup
WorktreeRemove	(no matcher)	Custom worktree cleanup
PreCompact	manual, auto	Save state before compaction
PostCompact	manual, auto	Restore state after compaction
SessionEnd	clear, resume, logout, prompt_input_exit, other	Save state, update logs

C.6. Hook Types (4 Total)

command

Runs a shell command (most common). Exit 0 = proceed, exit 2 = block. 10-minute timeout (configurable).

http POST JSON to a URL (30-second default timeout). Best for: webhooks, external service notifications, CI triggers.

prompt

Single-turn LLM evaluation (Haiku by default).

Returns {"ok": true/false}.

agent

Multi-turn subagent with tool access (up to 50 turns, 60s default timeout).

C.7. Permission Modes (6 Total)

Mode	Description
default	Prompts for permission on first use of each tool
acceptEdits	Auto-accepts file edit permissions for the session
plan	Claude can analyze but not modify files or run commands
auto	Auto-approves with background safety classifier. Requires <code>--enable-auto-mode</code> flag and Team plan or higher (research preview)
dontAsk	Auto-denies tools unless pre-approved via <code>/permissions</code> or allow rules
bypassPermissions	Skips permission prompts. Use only in isolated environments

Official
Modes verified 2026-03-25.
<https://code.claude.com/docs/en/permissions>

C.8. Settings Precedence (5 Levels)

1. **Managed** (`managed-settings.json`) — IT-deployed, highest priority
 2. **CLI arguments** — temporary session overrides
 3. **Local** (`.claude/settings.local.json`) — personal, auto-gitignored
 4. **Project** (`.claude/settings.json`) — team standards, committed
 5. **User** (`~/.claude/settings.json`) — global personal defaults
- Higher levels override lower. Deny rules are absolute and cannot be overridden by lower levels.

C.9. Environment Variables

Environment variables configure Claude Code behavior at the system level. Set them in your shell profile, CI workflow, or the `env` block in `settings.json`.

Official
Complete list at <https://code.claude.com/docs/en/env-vars>. Set in shell settings.json env block, or CI workflow.

Variable	Purpose
Authentication & Model	
ANTHROPIC_API_KEY	API key (overrides subscription billing)
ANTHROPIC_MODEL	Default model alias or full model ID
ANTHROPIC_BASE_URL	Proxy or gateway URL
ANTHROPIC_DEFAULT_OPUS_MODEL	Pin Opus version (e.g., <code>claude-opus-4-6-20250610</code>)
ANTHROPIC_DEFAULT_SONNET_MODEL	Pin Sonnet version
ANTHROPIC_DEFAULT_HAIKU_MODEL	Pin Haiku version
Context & Compaction	
CLAUDE_AUTOCOMPACT_PCT_OVERRIDE	Auto-compact trigger (1–100). Lower values compact earlier. Cannot raise above system default.
CLAUDE_CODE_AUTO_COMPACT_WINDOW	Decouple compact window from context window
CLAUDE_CODE_DISABLE_1M_CONTEXT	Disable 1M context (use smaller window)
Output & Behavior	
CLAUDE_CODE_MAX_OUTPUT_TOKENS	Max tokens per response (Opus: 128K, Sonnet/Haiku: 64K)
CLAUDE_CODE_EFFORT_LEVEL	Default effort: <code>low</code> , <code>medium</code> , <code>high</code> , <code>max</code> , <code>auto</code>
CLAUDE_CODE_DISABLE_FAST_MODE	Prevent fast mode activation
CLAUDE_CODE_DISABLE_ADAPTIVE_THINKING	Revert to fixed thinking budget
CLAUDE_CODE_SIMPLE	Same as <code>--bare</code> flag (minimal mode)
MCP & Tools	
MCP_TIMEOUT	MCP server startup timeout in ms
MCP_TOOL_TIMEOUT	Individual MCP tool execution timeout in ms
MAX_MCP_OUTPUT_TOKENS	Max tokens in MCP responses (default 25000)
ENABLE_TOOL_SEARCH	Tool search mode: <code>auto:N</code> (N=threshold)
Caching & Performance	
DISABLE_PROMPT_CACHING	Disable prompt caching globally
DISABLE_PROMPT_CACHING_OPUS	Disable caching for Opus only
DISABLE_PROMPT_CACHING_SONNET	Disable caching for Sonnet only
Features & Experimental	
CLAUDE_CODE_DISABLE_BACKGROUND_TASKS	Disable <code>run_in_background</code> and <code>Ctrl+B</code>
CLAUDE_CODE_DISABLE_GIT_INSTRUCTIONS	Remove built-in git instructions from system prompt
CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS	Enable agent teams
CLAUDE_CODE_ENABLE_TASKS	Enable task tracking in non-interactive mode
Security	
CLAUDE_CODE_SUBPROCESS_ENV_SCRUB	Strip credentials from child process environment

D. Sources and Further Reading

All URLs verified as of February 2026.

Maintenance note: Pricing, model capabilities, compliance certifications, and government program terms change frequently. Verify these against primary sources before making purchasing or compliance decisions.

D.1. Claude Code Documentation

All Claude Code docs migrated from `docs.anthropic.com` to `code.claude.com` (301 redirects). URLs below use the canonical domain. Last verified February 2026.

Overview

<https://code.claude.com/docs/en/overview>

Best Practices

<https://code.claude.com/docs/en/best-practices>

Settings

<https://code.claude.com/docs/en/settings>

Hooks Reference

<https://code.claude.com/docs/en/hooks>

Hooks Guide

<https://code.claude.com/docs/en/hooks-guide>

Skills

<https://code.claude.com/docs/en/skills>

Sub-Agents

<https://code.claude.com/docs/en/sub-agents>

Memory

<https://code.claude.com/docs/en/memory>

MCP Integration

<https://code.claude.com/docs/en/mcp>

Agent Teams

<https://code.claude.com/docs/en/agent-teams>

Security

<https://code.claude.com/docs/en/security>

Settings Schema

<https://json.schemastore.org/claude-code-settings.json>

D.2. Prompt Engineering and API

Prompt Engineering Overview

<https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>

Prompting Best Practices

<https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices>

Extended Thinking

<https://platform.claude.com/docs/en/build-with-claude/extend-ed-thinking>

Prompt Caching

<https://platform.claude.com/docs/en/build-with-claude/prompt-caching>

Batch Processing

<https://platform.claude.com/docs/en/build-with-claude/batch-processing>

Agent SDK Overview

<https://platform.claude.com/docs/en/agent-sdk/overview>

Advanced Tool Use

<https://www.anthropic.com/engineering/advanced-tool-use>

Pricing

<https://platform.claude.com/docs/en/about-claude/pricing>

D.3. Enterprise and Governance

Enterprise Plan

<https://support.claude.com/en/articles/9797531-what-is-the-enterprise-plan>

Team Claude Code

<https://support.claude.com/en/articles/11845131-use-claude-code-with-your-team-or-enterprise-plan>

Government Solutions

<https://claude.com/solutions/government>

Certifications

<https://privacy.claude.com/en/articles/10015870-what-certifications-has-anthropic-obtained>

Data Retention

<https://privacy.claude.com/en/articles/10023548-how-long-do-you-store-my-data>

Training Data Use

<https://privacy.claude.com/en/articles/7996885-how-do-you-use-personal-data-in-model-training>

Trust Portal

<https://trust.anthropic.com/>

D.4. Partnership Announcements

Deloitte

<https://www.anthropic.com/news/deloitte-anthropic-partnership>

Accenture

<https://www.anthropic.com/news/anthropic-accenture-partnership>

Snowflake

<https://www.anthropic.com/news/snowflake-anthropic-expanded-partnership>

TELUS

<https://www.anthropic.com/customers/telus>

D.5. Safety and Research**Claude's Constitution (Jan 2026)**

<https://www.anthropic.com/news/claude-constitution>

Constitutional AI

<https://www.anthropic.com/research/constitutional-ai-harmlessness-from-ai-feedback>

Opus 4.6 Announcement

<https://www.anthropic.com/news/claude-opus-4-6>

D.6. Community Resources**Using CLAUDE.md Files**

<https://claude.com/blog/using-claude-md-files>

How Anthropic Teams Use Claude Code

<https://claude.com/blog/how-anthropic-teams-use-claude-code>

Building Effective AI Agents

<https://www.anthropic.com/engineering/building-effective-agents>

Awesome Claude Code

<https://github.com/hesreallyhim/awesome-claude-code>

D.7. Further Reading**Claude Code Changelog**

<https://github.com/anthropics/claude-code/blob/main/CHANGELOG.md>

Anthropic Engineering Blog

<https://www.anthropic.com/engineering>

Claude API Documentation

<https://platform.claude.com/docs>

E. Glossary

Key terms used throughout this handbook. Chapter references point to the primary discussion of each term.

Agent loop The core cycle: prompt → reason → tool → observe → repeat. (Chapter 1)

Agent Teams

Research preview. Multiple agents sharing task lists with direct messaging and team lead orchestration. (Chapter 10)

Auto-compaction

Automatic context management triggered at ~83% of the context window. Proceeds in two phases: clears tool outputs first, then summarizes conversation. (Chapter 6)

Auto mode Permission mode where a classifier model approves or blocks actions without human prompts. Research preview. (Chapter 3)

/batch Bundled skill that decomposes large-scale changes into 5–30 independent units, spawns parallel agents in isolated work-trees, each opening a PR. (Chapter 10)

Batch API Asynchronous processing at 50% discount. Up to 100,000 requests per batch. (Chapter 4)

/btw Zero-context-cost side question. Answer appears in a dismissible overlay and never enters conversation history. (Chapter 6)

CLAUDE.md

Persistent configuration file loaded every session. The single most important configuration artifact. (Chapter 2, Chapter 9)

Compaction

Summarizing conversation history to free context space. Customizable via `CompactInstructions` in `CLAUDE.md`. (Chapter 6)

/compact <focus>

Targeted compaction: direct summarization toward specific context. Example: `/compact Focus on the API changes`. (Chapter 6)

Context budget

The finite token capacity of a conversation. Performance degrades as it fills. (Chapter 6)

Context window

Total token capacity: 1M tokens for Opus 4.6 and Sonnet 4.6. (Chapter 6)

Convergence

Practice agreed upon by both official Anthropic documentation and independent practitioners. Tagged **[Convergence]** in this handbook. (Chapter 1)

Durable artifacts

Files that persist across sessions: `CURRENT_WORK.md`, plan documents, git commits. Primary persistence strategy; compaction supplements. (Chapter 6)

Extended thinking

Claude's internal reasoning before responding. Toggle: `Alt+T`. Effort levels: `low`, `medium`, `high`, `max`, `auto`. (Chapter 4)

Fan-out Parallel execution of `claude -p` across multiple files. Test on 2–3 files before scaling. (Chapter 13)

Fast mode Same Opus 4.6, 2.5× faster, \$30/\$150 MTok. Research preview. Toggle: `/fast` or `Alt+0`. (Chapter 4)

Hooks Scripts that run automatically at specific lifecycle events (24 events, 4 handler types). Deterministic—unlike `CLAUDE.md`, which is advisory. (Chapter 8)

Interleaved thinking

Extended thinking between tool calls in a single turn, enabling nuanced multi-step workflows. (Chapter 4)

MCP (Model Context Protocol)

Open standard for connecting Claude to external services, databases, APIs, and custom tools. (Chapter 8)

Official Practice documented by Anthropic. Tagged **[Official]** in this handbook. (Chapter 1)

Permission modes

Six levels of tool approval: `default`, `acceptEdits`, `plan`, `auto` (research preview), `dontAsk`, `bypassPermissions`. (Chapter 3)

Plan Mode Read-only mode for exploration and planning. Claude reasons but cannot edit files. Enter via `Shift+Tab`. (Chapter 3)

Practitioner

Practice discovered through usage, not documentation. Tagged **[Practitioner]** in this handbook. (Chapter 1)

PostCompact hook

Fires after compaction completes (matchers: `manual`, `auto`). Used for state restoration, logging, or cleanup. Pair with Pre-Compact for automatic checkpointing. (Chapter 6)

PreCompact hook

Fires before compaction (matchers: `manual`, `auto`). Used for state preservation—e.g., serializing critical data to disk. (Chapter 6)

Prompt caching

Reusing previously processed prompt content. Cache reads cost 90% less than standard input tokens. (Chapter 4)

Skills Markdown files in `.claude/skills/` that extend Claude with domain knowledge and reusable workflows. Loaded on demand. (Chapter 8)

Subagent Isolated Claude instance with its own context and tool set.

Returns results to the parent session. (Chapter 10)

Two-failure rule

After two corrections in one session, `/clear` and write a better prompt. Accumulated corrections pollute context. (Chapter 3)

Worktree Isolated git working directory for parallel file modifications. Each agent gets its own branch and working copy. (Chapter 10)

Writer/Reviewer pattern

Separate sessions for creation and review to prevent confirmation bias. The reviewer has no context about the writer's intent. (Chapter 10)

Zero Data Retention (ZDR)

No storage of conversation data. Available for API and Enterprise plans. (Chapter 15)

F. What's New

This appendix summarizes the major changes between editions for readers upgrading from previous versions.

v2.8 (March 2026)

Reference Card Expansion (Chapter C)

Added 8 built-in commands and 8 CLI flags verified against official docs. Notable additions: `--bare` (minimal scripted mode), `--json-schema` (structured output), `--max-budget-usd` (cost control), and `--enable-auto-mode` (Team plan auto-approval). Footnote links to complete command list. `/review` noted as deprecated.

Bare Mode (Chapter 13)

New subsection documenting `--bare` for CI/CD and SDK automation. Skips hooks, skills, plugins, MCP, memory, and CLAUDE.md. ~14% faster to API request.

Model Capabilities Reference (Chapter 4)

New table with max output tokens, context windows, knowledge cut-offs, and thinking modes. Complements the pricing table in Chapter 15.

Auto Mode Prerequisite (Chapter 3)

Margin note clarifying that auto permission mode requires `--enable-auto-mode` flag and a Team plan or higher.

v2.7 (March 2026)

Two-Phase Compaction (Chapter 6)

New subsection explaining how auto-compaction works internally: tool outputs are cleared first (cheap, low info loss), then conversation is summarized only if needed. Also documents that CLAUDE.md fully survives compaction—re-read from disk and re-injected fresh.

PostCompact Hook (Chapter 6)

New subsection with JSON configuration example mirroring the existing PreCompact coverage. Includes the PreCompact/PostCompact pairing pattern for automatic state checkpointing across compaction.

Auto-compaction threshold override (Chapter 6)

Documents the `CLAUDE_AUTOCOMPACT_PCT_OVERRIDE` environment variable. Tagged as convergence (undocumented, community-discovered). Can lower but not raise above ~83%.

CLAUDE.md sizing guidance (Chapter 6)

Anthropic recommends under 200 lines per CLAUDE.md file and under ~500 lines total across all five hierarchy levels.

Hook events: 24 → 25 (Chapter C)

Added TaskCreated event. Auto permission mode now documents the `--enable-auto-mode` flag and Team plan prerequisite.

Max output tokens (Chapter 15)

Pricing table now includes max output limits: Opus 4.6 128K tokens, Sonnet 4.6 and Haiku 4.5 64K tokens each.

v2.5 (March 25, 2026)

Changes from v2.3 (February 2026) to v2.5.

New Chapters and Sections**1M Context Window Strategy (Chapter 6)**

New section covering how the 5× larger context window changes the game. Includes a decision framework (four tiers from <30% to >80%), `/compact <focus>` for targeted compaction, and a PreCompact hook example for state preservation.

Durable Artifacts (Chapter 6)

Primary persistence strategy: `CURRENT_WORK.md`, plan documents, and git commits outlast compaction and session boundaries. Compaction is positioned as a supplement, not the primary strategy.

`/btw`: Zero-Context-Cost Side Questions (Chapter 6)

Ask questions without growing context. The answer appears in a dismissible overlay and never enters conversation history.

Fast Mode (Chapter 4)

Same Opus 4.6, 2.5× faster, \$30/\$150 MTok (research preview). Toggle: `/fast` or `Alt+0`. Falls back to standard on rate limit.

`/batch`: Parallel Codebase Changes (Chapter 10)

Bundled skill that decomposes work into 5–30 units and spawns parallel agents in isolated worktrees. Each agent implements, tests, and opens a PR.

Agent SDK (Chapter 13)

Expanded from a decision box to a full subsection with Python and TypeScript examples. Same tools as Claude Code, programmable for production deployment.

Scheduled Tasks (Chapter 13)

Three tiers: Cloud (`/schedule`, runs on Anthropic infrastructure), Desktop (local machine, persistent), and `/loop` (session-scoped polling).

GitHub Actions (Chapter 13)

`anthropics/claude-code-action@v1` for AI-powered PR review. Triggered by `@claude` in any PR or issue comment.

Remote Control (Chapter 13)

`/remote-control` connects `claude.ai/code` or the mobile app to a session running on your machine.

API Pricing Reference (Chapter 15)

Model pricing table with Haiku 4.5, Sonnet 4.6, Opus 4.6, fast mode, and discount combinations. Includes staleness qualifier.

Corrected Claims

Hook events: 21 → 24

Added `StopFailure`, `CwdChanged`, `FileChanged`. Full matcher values documented for all 24 events (Chapter 8).

Hook handler types: 3 → 4

Added `http` type for webhooks and CI triggers (Chapter 8).

Permission modes: 5 → 6

Added `auto` (research preview). All six modes documented with correct `Shift+Tab` cycling behavior (Chapter 3).

Keyboard shortcuts corrected

`Ctrl+G` opens prompts in the default text editor (not “the plan”).
`Shift+Tab` cycles through all enabled modes (not a fixed three-mode loop).

`/effort auto` explained

Resets effort to the model’s default behavior—does not mean “dynamic per-turn.”

60% degradation threshold qualified

Clarified as a practitioner heuristic (~60–70%) distinct from the auto-compaction trigger (~83%). For 1M windows, absolute token counts matter more than percentages.

CLAUDE.md line count: both official numbers cited

Anthropic recommends under 200 lines per file (memory docs) and under ~500 lines total (cost docs). Our 300-line guidance is reframed as a practitioner compromise between the two.

Caching claims qualified

“Up to 60–80%” savings now includes “(workload-dependent)” qualifier. “Caching beats model downgrading” now says “typically saves more.”

FedRAMP separated from commercial certs

Government authorizations (FedRAMP High, IL5) are now distinct from commercial certifications (SOC 2, ISO 27001, HIPAA BAA) (Chapter 15).

Two broken URLs fixed

Long-running-agents blog post URL and GitHub Copilot blog post URL updated to current locations.

Expanded Content

Code intelligence plugins (Chapter 8)

Before/after table showing go-to-definition, find-references, and post-edit error detection capabilities.

Extended thinking + compaction warning (Chapter 6)

Thinking blocks are immutable—compaction cannot summarize them.
Mitigations: `/clear` after deep thinking, or delegate to subagents.

Appendix C: full command reference

Expanded from 16 to ~50 commands, structured as built-in commands and bundled skills. Keyboard shortcuts corrected and ex-

panded.

Property-based testing example (Chapter 7)

Concrete Hypothesis example demonstrating schema, null, and range invariants.

MCP walkthrough (Chapter 8)

Practical before/after showing `claude mcp add` with a real server.

Index

- agent loop, 5
- Agent SDK, 79
- Agent Teams, 65
- anti-patterns
 - big-bang refactoring, 74
 - context overload, 71
 - infinite exploration, 74
 - kitchen sink session, 72
 - over-correcting, 72
 - permanent prototype, 72
 - pipeline direction, 73
 - verification gap, 73
- bare mode, 79
 - /batch, 65
 - /btw, 33
- CLAUDE.md, 9
 - architecture, 58
 - claudeMdExcludes, 60
 - deny rules, 11
 - hub-and-spoke, 58
 - /init command, 9
 - line count (200/500), 60
 - loading order, 60
 - sizing, 36
 - user rules, 59
 - what to include, 9
- collaboration
 - ADR, 30
 - hypothesis debugging, 25
 - interview pattern, 29
- compaction, 32
 - /compact focus, 34
 - CompactInstructions, 34
 - PostCompact hook, 36
 - PreCompact hook, 35
 - two-phase, 35
- context
 - 1M strategy, 35
 - 60% threshold, 32
 - auto-compaction, 32
 - threshold override, 32
 - degradation, 32
 - two-failure rule, 14
- context window, 5
- cost optimization
 - Batch API, 22
 - caching beats downgrading, 89
 - model selection, 22
 - prompt caching, 22
- course correction
 - Esc, 14
 - rewind, 14
- delegation
 - when to delegate, 63
- durable artifacts, 38
- enterprise
 - certifications, 87
 - FedRAMP High, 87
 - pricing, 88
 - training data policy, 87
 - Zero Data Retention, 87
- extended thinking, 21
 - effort levels, 21
 - interleaved thinking, 21
- fast mode, 22
- GitHub Actions, 82
- headless mode, 78
 - allowedTools, 78
 - bare mode, 79
 - fan-out, 78
- hooks, 48
 - events (24), 49
 - handler types (4), 51
 - matchers, 51
- MCP, 52
 - permissions, 52
 - servers, 52
 - tools, 52
- memory

- auto-memory, 38
- microcompaction, see compaction,
 - two-phase
- model capabilities
 - comparison table, 23
- output styles, 60
- permissions
 - modes, 15
- Plan Mode, 14
- plugins, 53
- principles
 - fail fast, 6
 - immutability, 6
 - never fail silently, 5
 - simplicity, 6
- projects
 - brownfield, 68
 - greenfield, 67
 - refactoring cycle, 68
- prompting
 - be specific, 13
 - precision vocabulary, 19
 - step by step, 13
 - verification criteria, 13
 - XML tags, 20
- remote control, 82
- rich inputs
 - piping, 20
 - URL allowlisting, 20
- sandboxing, 54
- scheduled tasks
 - Cloud, 81
 - Desktop, 81
 - /loop, 81
- session
 - loop, 12
- sessions
 - /clear, 32
 - naming, 37
 - resume, 37
- skills, 45
 - budget (2%), 47
- status line, 33
- subagents, 62
- built-in types, 63
- frontmatter, 63
- SendMessage, 63
- team
 - managed settings, 84
 - onboarding, 85
 - settings.json, 84
 - settings.local.json, 84
- testing
 - 6-layer model, 41
 - phase-appropriate, 42
 - property-based, 41
 - test-first pattern, 42
- tool use, 5
- worktrees, 64
 - isolation, 64
- Writer/Reviewer pattern, 65